

Write your program!

Introduction to Programming

Michael Sperber

Herbert Klaeren

April 5, 2024

Copyright © 2007-2024 by Michael Sperber and Herbert

Klaeren. This book is licensed under the Creative Commons *Attribution-ShareAlike* 4.0 International License (CC BY-SA 4.0), available at <https://creativecommons.org/licenses/by-sa/4.0/deed.de> .

TABLE OF CONTENTS

0 How to get the most out of this book	11
0.1 Prior knowledge	11
0.2 Salutation	11
0.3 Software	11
0.4 Programming language	12
0.5 English	12
0.6 Excursions	12
0.7 tasks	12
0.8 Additional material	13
0.9 Code	13
0.10 Bureaucratic Didactics	13
0.11 Improvements	14
1 Elements of Programming 1.1	15
DrRacket	15
1.2 Expressions and the REPL	16
1.3 The definition window	19
1.4 Calculating without numbers	21
1.5 Abstraction and Application	26
1.6 Information and data	31
1.7 Systematically construct programs	32
1.8 Brief description	34
1.9 Signature	34
1.10 Tests	36
1.11 Construction instructions	38
1.12 The Power of Abstraction	42
1.13 Mantras for Programming	44
2 Case distinctions and branching 2.1 Calculating	51
with Boolean values	51
2.2 Branching	53
2.3 Cover	56
2.4 Enumerations	57
2.5 Number ranges	59

2.6 Data analysis and templates	73
2.7 Excursion: Branching in mathematics	77
2.8 Binary Branches	77
2.9 Syntactic sugar	80
2.10 Intercepting nonsensical data	81
3 Composite Data	87
3.1 Configure the computer	87
3.2 Constructing composite data yourself	94
3.3 Construction guidelines for composite data	100
3.4 Input and output of composite data	102
3.5 Alternatives in the construction instructions	106
4 Mixed Data 4.1	117
Mixed Data	117
4.2 Mixed data as output	123
4.3 The food traffic light system	125
4.4 Represent errors	132
5 Programming with self-references and combinators 5.1	141
Mapping flows	141
5.2 Modeling images	149
5.3 Mapping financial contracts	150
6 Programming with Lists 6.1	173
Representing Lists	173
6.2 Lists with other elements	180
6.3 Construction instructions	183
6.4 Built-in Lists	184
6.5 Parametric Polymorphism	184
6.6 Functions that produce lists	186
6.7 Minimum of a list and local variables	192
6.8 Functions on non-empty lists	196
7 Natural Numbers 7.1	205
Numbers that count	205
7.2 Natural Numbers and Lists	208

7.3 Counting the other way around	211
7.4 Excursion: Optimizing power	214
8 Excursus: Inductive Proofs and Definitions	219
8.1 Quantities	219
8.2 Statements about natural numbers	221
8.3 Inductive proofs lead to	225
8.4 Structure of the natural numbers	227
8.5 Finite Episodes	229
8.6 Notation for inductive definitions	233
8.7 Structural Recursion	234
8.8 Structural Induction	237
9 Higher-Order Programming 9.1	245
Determining Football Facts	245
9.2 Higher-order functions on lists	252
9.3 Converting lists	259
9.4 Fold lists	265
9.5 Schönfinkeln	270
9.6 Lexical binding	274
10 Programming with Accumulators 10.1	289
Carrying out intermediate results	289
10.2 Templates for functions with accumulator	295
10.3 Accumulating over natural numbers	300
10.4 Analyze stock prices	303
10.5 Context and tail recursion	306
10.6 The phenomenon of the reverse list	308
10.7 Iteratively fold lists	311
11. Programming video games	317
11.1 Images with image.rkt	317
11.2 Models and views	326
11.3 Modeling the highway	330
11.4 Street views	331
11.5 Moving images with universe.rkt 11.6	335
Cars and armadillos on the highway	337

11.7 The world is a highway	342
11.8 Run over, specifically	344
11.9 Reactive Animation	349
12 trees	357
12.1 Family trees	357
12.2 Binary Trees	361
12.3 Trees for searching	367
12.4 Produce sorted trees	372
12.5 Search trees	374
12.6 Sorted trees are more efficient than lists	378
12.7 Balancing search trees	381
12.8 Balancing by rotation	391
13 Properties of functions 13.1	401
Correctness and tests	401
13.2 How <code>check-property</code> works	404
13.3 More properties and inexact numbers	405
13.4 Relational Problems	409
13.5 Design guidelines for test cases?	416
13.6 Testing search trees	417
13.7 Algebraic properties	427
13.8 Properties of functions on lists	431
13.9 Excursion: Proving properties	432
14 Excursion: The λ-	439
Calculus 14.1 What does an old "calculus" have to do with programming ?	439
14.2 The language of λ -calculus	440
14.3 λ -intuition	444
14.4 How do variables work ?	444
14.5 Reduction in the λ -calculus	448
14.6 Normal forms	451
14.7 Evaluation strategies	452
14.8 The λ -calculus as a programming language	455
15 Excursion: The SECD machine	465
15.1 The applied λ -calculus	465

15.2 The SECD machine	467
15.3 Quotes and symbols	473
15.4 Data definitions for the λ -calculus	481
15.5 Data definition for machine language	484
15.6 A compiler for the SECD machine	486
15.7 Execute SECD code	489
15.8 The tail-recursive SECD machine	503
epilogue	509
Index for variables	513
index	519

FOREWORD

BY PROF. TORSTEN GRUST, UNIVERSITY OF TÜBINGEN

Writers fear the blank page. Painters fear the white canvas. For architects, it is
The green meadow. For programmers, it's the cursor, demanding attention in the empty editor window.
It's blinking... How do you approach a programming task when you don't start with a single line of code?
What exists when no function or data structure offers guidance, and when *everything* seems possible
but therefore difficult to grasp?

Mike Sperber and Herbert Klaeren provide good information on this topic in *their book *Write Your Program!** Programs can be constructed very systematically and—in some cases almost automatically—derived from the given problem. Scattered throughout the book are examples of this.
Construction manuals, whose program skeletons contain concrete indications of the structure of
This provides a solution to the problem. Once the skeleton is in place, its gaps can often be filled
immediately, partly by using further such constructions, partly because...
The solution arises directly from the problem. Programming using these construction instructions
comes closer to disciplined, engineering-based construction than anything else.

My *Computer Science I* lectures at the University of Tübingen are invariable for students who
haven't left without having gotten their hands thoroughly dirty. In the lecture...
self, the weekly exercise sheets and additional supervised programming lessons will
I wrote code like crazy. In doing so, I've already observed hundreds of students – typically complete
beginners who don't yet know any programming language – using
The design instructions have accepted and mastered the challenge of the empty editor.
The resulting programs are (mostly ;-) correct, compact, and, most importantly, readable.
and comprehensible. In this way, we can tackle surprisingly challenging tasks in *Computer Science I*
and, by the end of the second third of the lecture, are able to teach the students *Pac-Man*,
To have *Tetris* or *Asteroids* recreated.

Write Your Program! is also an introduction to functional programming.
with Scheme. Scheme itself, with its simple syntax, recedes into the background¹ and gives the view
freely focused on the essential ideas of thinking and programming with (mathematical) functions. Just
like Mike Sperber and Herbert Klaeren, I am convinced that this elegant programming paradigm is
"here to stay" and will, in the long run, define programming style and
-languages will have a far-reaching impact. Along with the construction instructions, the-

¹ Where we usually have a zoo of notations $x + 1$, $\sin(x)$, x ², $|x|$, $\bar{y}$, $\bar{x}$, $x!$, ... use, Scheme is satisfied
uniform with the notation (fx) .

This book therefore equips programming beginners with two superpowers. That's what I call...

A deal!

Torsten Grust

Tübingen, July 2019

ACKNOWLEDGMENTS

Peter Thiemann, Martin Gasbichler, Andreas Schilling, Torsten Grust and Michael Hanus held Lectures based on this book provided valuable feedback. Special thanks are due to the tutors and students of our *Computer Science I* lecture, who...

They provided a wealth of helpful criticism and excellent suggestions for improvement.

Since the book's development has been openly hosted on GitHub, many readers have also made suggestions for improvement and pull requests, including Nicolas Neuß, Joachim Breitner, Raphael Borun Das Gupta, Tobias Huttner, Nicolai Mainiero, Johannes Mai-er, Sibylle Hasse, Manuela Reinisch, Noah Haasis, mi-skam, uwffi, Aramís, TheKarcher, chessgarf, Beat Hagenlocher, Denis Maier, Philipp Kiersch, Simon Barth, Kaan Sahin, Erika Bor and Christina Zeller.

Before that, Robert Giegerich, Ulrich Güntzer, Martin Plümicke, Christoph Schmitz and Volker Klaeren, Eric Knauel, Marcus Crestani, Sabine Sperber, Jan-Georg Smaus and Mayte Fleischer made many corrections and improvements, especially to the previous book, *Vom Problem*.

Regarding the program: Katharina von Savigny played a crucial role in the linguistic design of the book.

We thank Sandra Binder, Peter Rempis and Axel Braun of Tübingen University Press for the great support and patience with us, and especially Susanne Schmid for the cover design.

Among our didactic role models were Matthias Felleisen, Robby Findler, Matthew Flatt and Sriram Krishnamurthi and their book **How to Design Programs** [FFFK14] are particularly important, from which we have adopted many ideas. The [book/book] developed within the framework of their PLT project Racket system is a crucial foundation for working with this book.

Michael Sperber, Herbert Klaeren Tübingen, July 2023

HOW TO GET THE MOST OUT OF THIS BOOK

The content of this book has been further developed over many years, which has resulted in it differing significantly from many other introductory programming books.

Before we get to the actual material, this chapter contains some tips on how to deal with it most effectively so that you can write your own programs as quickly as possible.

0.1 PRIOR KNOWLEDGE

This book is aimed at anyone who wants to learn programming. It can also serve as an introduction to so-called *functional programming* for those who already program in another paradigm (for example, object-oriented programming).

We have tried to minimize the required prior knowledge. Basic math skills are helpful, but only minimally necessary. Everything beyond that is explained in the book.

0.2 Salutation

This book uses "you" and "we" in its forms of address. "You" refers to you, the reader. "We" refers to us, the authors. We often use phrases like "First, we'll write a signature" when we write about the examples in the book that we've come up with for you. When the book addresses you with "you," we have a specific request for you.

0.3 SOFTWARE

The programming examples in this book are based on the DrRacket programming environment, which was specifically developed for programming education. DrRacket is available for free on the internet at

<https://www.racket-lang.org/>

available and runs on Windows, Mac and Unix/Linux computers.

Make sure you get at least version 8.6. This is especially important if you...
you choose a different installation method, such as a package manager for a Linux distribution.

0.4 PROGRAMMING LANGUAGE

Anyone learning to program has to deal with quite a lot of things: the concepts. a specific programming language, the notations used there, a programming environment, and the other tools needed for programming.

Most programming languages and their tools are intended for professional developers. They have been developed and therefore require a lot of knowledge that you still need to learn.

For this reason, this text uses a series of programming languages specifically developed for teaching purposes, based on *Racket* and *Scheme*, which are included with the Racket system. The development environment DrRacketen is included with Racket. which allows you to easily write and run programs in these languages.

0.5 ENGLISH

The book is of course in German, and the DrRacket programming environment can also be used in German. Display menus and other texts in German.

Programming languages use English words, and we use English names for all sorts of things in our programs. This doesn't require advanced English skills. We provide translations for less common words. (You're welcome to use German names in your programs.)

Furthermore, we haven't yet translated all error messages into German, but we are working on it. to it.

0.6 EXCURSIONS

In general, the book was written in such a way that the chapters and the sections within the chapters They build upon each other. Occasionally, however, there are sections (usually those with mathematics) that we find interesting, but which are not necessary to understand the subsequent material. Understand. These sections are marked with the word "Excursus".

0.7 TASKS

The book contains many exercises. It is essential that you complete at least some of them if you want to learn to program independently.

Each chapter contains exercises embedded in the text that relate to the immediately preceding material. (The symbol ¶ indicates when the exercise ends and the text continues.) You should complete these exercises to ensure you understand the material. You can choose the exercises at the end of each chapter that appeal to you.

0.8 ADDITIONAL MATERIAL

This book originated from the DeinProgramm project, which deals with effective programming education in general. The project's website can be found here:

<https://www.deinprogramm.de/>

There you can download the current version of the book and other material, such as publications on the didactic concept.

0.9 CODE

In this book, we work with many example programs, most of which are developed step by step. Our years of experience have taught us that simply reading about these programs can be tedious and perhaps even boring. You will likely get more out of it if you at least partially follow the development process on your computer and try out, extend, or experiment with the programs. You don't have to type in the code for these programs; you can simply download it. There are two ways to do this: The code is available for download on the book's website under the link "Supplementary Materials." We always indicate the exact file name in the text as follows:

[elements/tile.rkt](#)

code

If you are reading an electronic version of the book, the file name is also a link: Draufklicken and you'll get to the code.

0.10 BUREAUCRATIC DIDACTICS

Let's be honest: programming is a complex and therefore difficult activity. We – the authors – have already spent well over half our lives doing it and are still learning. But programming is also immensely rewarding when you successfully develop your own projects independently.

Pure thought can create something that works and communicates. So the learning effort is worthwhile.

But nobody enjoys getting stuck while solving a programming problem and ending up in a dead end. Therefore, with this book, we don't just want to present you with beautiful example programs, but rather teach you step by step *how* they were written and – most importantly – how to write your own.

The method we have developed and successfully applied to hundreds of novice programmers will seem enormously bureaucratic to you at first glance: This book deals, so to speak, with the German civil service method of programming.

Occasionally you'll feel restricted and perhaps even a little annoyed. But you will (hopefully!) gradually be able to use these techniques not only to solve other people's exercises, but also to implement your own ideas and give free rein to your own creativity: For this, you need the methods in this book, including the bureaucracy.

Two elements contribute particularly to learning success:

- The *construction instructions* prescribe steps you can take to resolve a problem-
The approach to finding a solution – the German civil service regulations, so to speak.
- *Mantras* are recurring programming principles that are worth keeping readily available.
to have. Luckily, there are only a handful.

0.11 IMPROVEMENTS

One last point about how we can get the most out of the book: If you notice an error in the book or have a suggestion on how we could improve it, we would appreciate your feedback. You can send this either by email to sdp@deinprogramm.de You can do this, or even better, submit an "issue" or "pull request" to access the book's source code. This source code, currently managed on the GitHub online service, can be accessed via the website for [website address].

Book:

<https://www.deinprogramm.de/sdp/>

1 ELEMENTS OF PROGRAMMING

This chapter provides an overview of the most important tools of programming. For You should have downloaded Racket to get started. Section 0.3 on page 11 explains where.

1.1 DRACKET

Racket includes a program called *DrRacket*: start it. A window will appear that should look approximately like the one in Figure 1.1. You can set the user interface to German.

to switch by selecting German user interface for DrRacket in the Help menu.

Pressing it. If you confirm your selection, DrRacket will then close and you won't have to do anything else. Start it once; then the menus should be in German.

DrRacket is a *development environment* that allows you to write and run programs.

You can. DrRacket doesn't just support a single programming language, but many. That's why

You still need to select the correct programming language for this book. To do this, select the menu item Language → Choose language (in the English version), which will open a new window with a dialog box.

In this dialog box, there is a section called Teaching Languages, and below that, a heading called YourProgram.

whose several entries appear, specifically tailored to the chapters of this book.

For the first part of the book, the level "Write Your Program! - Beginners" is responsible: Select this and then press the Start (or Run) button in the upper right corner to activate the selection. The result should then look like Figure 1.2.

The DrRacket window consists of two parts:

1. The program text is displayed in the upper half of the window (the *editor* or *definition window*). The editor functions similarly to a regular word processor. What is there
Once it's written, you can save it. When saving a program, use the .exe extension.
.rkt for «**Racket**».
2. The program's output is displayed in the lower half of the window – the *interaction window* or so-called REPL¹. You can also view individual program parts.
Targeted testing.

¹ «REPL» stands for «Read-Eval-Print-Loop» – we will show why later.

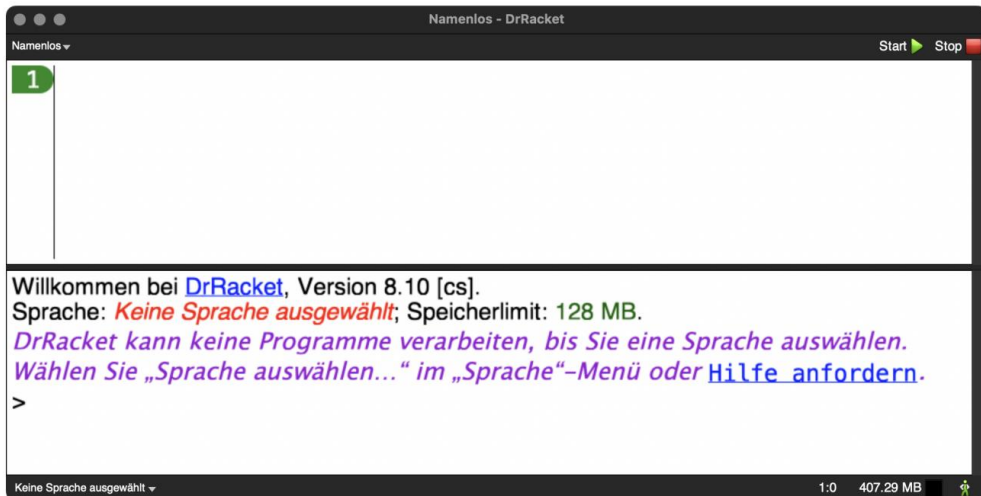


Figure 1.1: DrRacket after the first start

1.2 EXPRESSIONS AND THE REPL

Let's start with the REPL: If you've just pressed Start, the cursor will appear to the right of the > symbol: Here you can enter something and press Return. DrRacket will then display the result below.

For example, if you type `123`, DrRacket will display 123 directly below it. DrRacket can also do calculations. However, you need to write the problems slightly differently than usual.

For example, like this:

`(+12342)`

If you enter that into the REPL, DrRacket displays `165`, the *sum* of 123 and 42. This "calculation" is called an *expression*. Expressions that are meant to calculate something always have the same form in the languages taught:

`(Operator Operand ...)`

Parentheses *always* surround the operator (and they can't be placed anywhere else). Then comes the *operator*, which determines *what* is done (the *operation*). After that come the *operands*, which define the inputs for the operation.

To remember, it is helpful to read expressions aloud accordingly: So no longer "hundred-twenty-three plus forty-two", but "the *sum* of one hundred twenty-three and forty-two".

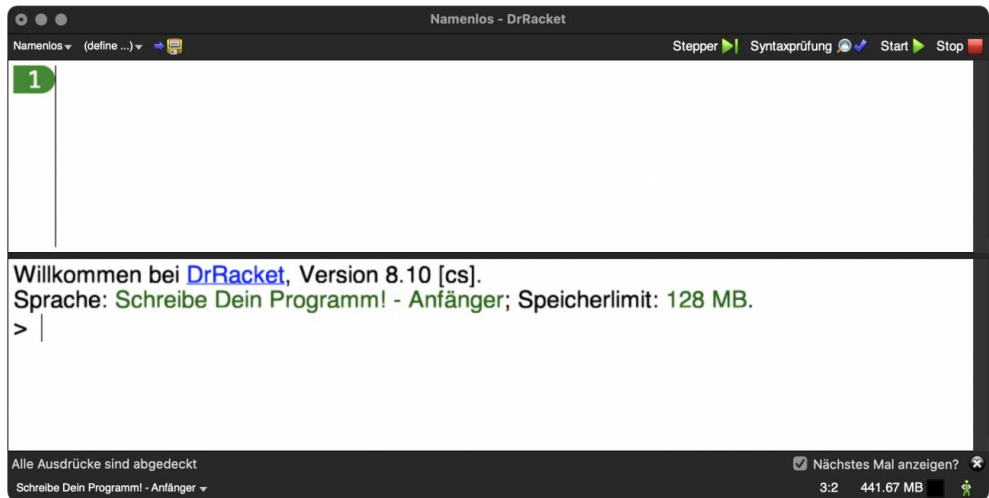


Figure 1.2: DrRacket with selected teaching language

If you forget parentheses, confusing results can occur. For example, if you type `+12342` into the REPL, it will look like this:

```
> +12342
#<function:+>
123
42
```

This is because, without parentheses, `+ 123 32` consists of *three* expressions, and therefore the REPL outputs three results: the `+` function, then `123`, then `42`. (The `+` operation is a so-called function – we will explain this in more detail later.) The same applies to `123 + 42` – try it out!

If you accidentally place the parentheses but write the operator in between, it will appear The REPL contains an error message:

```
> (123 + 42)
The operator cannot be a number, but it is 123.
```

Something similar happens if the space between the operator and the operands is missing – or between the operands themselves. It looks something like this:

```
> (+123 42)
The operator cannot be a number, but it is 123.
```

This is because `+123` together forms the number "plus one hundred and twenty-three" and is not split into the `+` sign and `123`.

If an expression like `(+ 123 42)` has a result like 165, we will write it like this in the book in the future, slightly differently than the DrRacket REPL:

```
(+ 123 42)
↳ 165
```

This allows you to calculate not only sums, but also differences, products (using the operator `*`) and quotients (using the operator `/`):

```
(- 123 42)
↳ 81

(* 123 42)
↳ 5166

(/ 123 42)
↳ 2.9285714
```

The last expression shows that decimal numbers are written with a period, not a comma. The overline in `2.9285714` represents a *repeating period*. Therefore, the number is actually...

`2.9285714285714285714285714285714...`

The REPL works as follows: It *reads* an expression, calculates or *evaluates* its value ("eval", short for "evaluate"), and displays or prints the result ("*print* ") – and then it starts again from the beginning, like in a loop ("loop").

The sequence *Read-Eval-Print-Loop* gives the REPL its name.

Expressions can also be combined, for example like this:

```
(* 123 (+ 20 22))
↳ 5166

(* 123 (+ (* 2 10) 22))
↳ 5166
```

When combining different expressions, it's important that each subexpression is enclosed in parentheses. If this isn't the case, error messages like this one may occasionally appear:

```
(* 123 (+ * 2 10 22))
+: Number expected as first argument, #<function:*> received
```

This is because the `*` here stands in for an operand. The word "*argument*" represents the value of a function's operand, so it essentially means that `+` would expect a number as the first argument, but instead the function `*` has been assigned to it. (We define the word "argument" more precisely in Section 1 on page 30.)

Saving is not possible with the REPL, and the REPL contents disappear every time you press Start or Run. However, you can recall previous entries by pressing Ctrl-Ÿ or Control-Ÿ (depending on your computer). (This also works down with Ctrl-Ÿ or Control-Ÿ.) You can also use this to correct an erroneous expression after you've already pressed Return.

Task 1.1

Write the following "mathematical" expressions in the notation of the teaching language into the REPL and let the REPL evaluate them:

$$\begin{aligned}
 &55 \text{ Ÿ } 27 \\
 &23 \text{ Ÿ } (44 + 27) \\
 &\frac{23}{44} + \frac{44}{23} \\
 &(23 + 42) \text{ Ÿ } (12 + (14 \text{ Ÿ } 2))
 \end{aligned}$$

Ÿ

1.3 THE WINDOW OF DEFINITION

Let's move on to the definition window above. This is where you write your program; you can then use the REPL to test it. Enter the following *definition into the definition window*:

```
(define everything (+ 20 22))
```

This definition states that the name "*everything*" refers to the result of `(+ 20 22)`. To try this out, click the Start or Run button in the upper right corner. The cursor will then return to the REPL, where you can test the program.

```
everything
```

```
Ÿ 42
```

Such a name in a program is called a *variable*. ²

² Although the term "variable" suggests that something about it can be changed, the value of a variable always remains the same.

Here are a few more examples of variable definitions:

```
(define one 1)
```

```
(define temperature 23) (define birgit-  
prinz 9)
```

You can also enter these names into the REPL and you will receive the corresponding information.

Value when you press Return:

```
one
```

```
↳ 1
```

```
temperature
```

```
↳ 23
```

```
birgit-prinz
```

```
↳ 9
```

There is considerable freedom in naming a program. Unlike other programming languages, hyphens are allowed in names. Only spaces are not permitted.

Case doesn't matter for names. So it also works:

```
Everything
```

```
↳ 42
```

```
ONE
```

```
↳ 1
```

```
tEmPeraTuRe
```

```
↳ 23
```

In a program, you can use line breaks and indentation to make your program more readable. For example, you can press the Return key after `everything`; the result looks like this:

```
(define everything
```

```
  (+ 20 22))
```

DrRacket indents the second line slightly to indicate that it still belongs inside the parentheses of the ``define` statement`. This is helpful for more complex expressions.

```
(define everything
```

```
  (+20)
```

```
    (* 11 2)))
```

Here, DrRacket places the operands of the sum directly below each other. DrRacket does not force you to... The indentation needs to be set exactly as intended, and sometimes changes in the program can throw it off. In that case, you can press the Tab key (on most keyboards it says Tab).

Pressing the Tab key (↵) corrects the indentation in the line where the cursor is located. If you select a section in the program and then press Tab, the entire text will be indented.

Section newly indented. There is also a menu item Racket ↵ Indent All.

(or Racket ↵ Reindent All), which does that for the entire program.

Another handy trick is to highlight a bracketed expression (and then

You can cut (or extract) the selected text by double-clicking the opening or closing bracket. In particular, you can then correctly format the selected text by pressing the Tab key.

move in.

Task 1.2

In a multi-line program, completely mess up the indentation, for example like this:

```
(define nr
  (+12)
  (- (* 42
    13)
    500)))
```

Then use the Tab key to correct the indentation again.

↵

You can save the contents of the definition window by pressing the button with the floppy disk symbol (3) . A dialog box will then  appear where you can enter a file name.

You can forgive.

1.4 Calculating without numbers

Computer programs can do more than just calculate with numbers. This section is about... Calculating with text and calculating with images.

³ Floppy disks were used to store data in the 20th century. Memory cards and USB sticks didn't exist back then.

Strings represent text. Literals for *strings* have the following form :

`"z1z2 . . . zn"`

The ``zi`` characters are any single characters, except for the quotation mark itself. Example:

`"Herbert was here!"`

The quotation mark (`"`) cannot occur "unprotected" because it marks the end of the string. It is represented as a "real" quotation mark within a string by `\` :

`"Herbert says \"Hello Mike!\""`

Figure 1.3: Strings

1.4.1 Calculating with Text

Text can be converted into a value using double quotation marks:

`"Mike Sperber"`

`"Herbert Klaeren"`

`"Write your program!"`

Such text values are called *strings*.

The simplest way to create a string is to write down the letters it consists of. Quotation marks are necessary to distinguish strings from other expressions. However, the quotation marks themselves are not part of the letters that make up the string – `"abc"` consists of the three letters

ABC.

Figure 1.3 describes the exact notation for such "fixed" strings. Fixed notations for values are generally called *literals*. We already know this from numbers: The string `123` stands for the number "one hundred and twenty-three". Boxes like Figure 1.3 will be used frequently in this book to introduce new language elements.

A program can also perform calculations with text, using the built-in function `string-append`, which concatenates two strings:

`(string-append "Herbert" "Mike")`

`⇒ "HerbertMike"`

`(string-append "Mike" " " "ist doof") ⇒ "Mike ist doof"`

The built-in function `string-length` returns the number of characters in a string:

```
(string-length "Herbert")
```

```
↳ 7
```

```
(string-length "Mike")
```

```
↳ 4
```

The names `string-append` and `string-length` look “different” at first glance than `+` and `*`, for example, but this impression is misleading: they are all names of predefined operations that programs can use without having to define them themselves.

The predefined functions `string->number` and `number->string` convert between numbers and the strings that represent them:

```
(string->number "23")
```

```
↳ 23
```

```
(number->string 23)
```

```
↳ "23"
```

Task 1.3

Make sure you understand the difference between the number `23` and the string `"23"`. Try to...

Example from:

```
(+ "23" 42)
```

```
(string-append 23 "42") (number-
```

```
>string "23")
```

```
↳
```

1.4.2 Calculating with Images

Programs can also process images. This requires an extension to DrRacket called a Teachpack. To activate it, select Language `↳ Add Teachpack`. In the dialog box that appears, select `image.rkt` and click OK. Then click Start again. If everything worked correctly, the REPL will show Teachpack: `image.rkt` at the bottom.

The Teachpack `image.rkt` contains additional predefined functions that we can use to create images. For example, `square`, `circle`, and `star-polygon`:

```
(square 40 "solid" "red")
```



```
ŷ
```

```
(circle 20 "solid" "green")
```



```
ŷ
```

```
(star polygon 20 10 3 "solid" "blue")
```



```
ŷ
```

Let's take a closer look at the built-in `star polygon` : It accepts five inputs, the first three of which are numbers – the side length, the number of sides, and the number of vertices to skip for each side. These are followed by two strings – "`solid`" means that the interior of the star is filled, and "`blue`" is the color. Instead of "`solid`," you can also write "`outline`," which makes it a bit clearer what "skipping" means.

```
(star polygon 20 10 3 "outline" "blue")
```



```
ŷ
```

Task 1.4

What do the numbers in the `square` and `circle` fields represent? Try different numbers!

There's also a built-in function called `rectangle`. Can you construct a working example of how to use `rectangle` ?

There's also a built-in function called `ellipse`, which is used in the same way as `rectangle` – try it out! ŷ

Images are values, just like numbers and strings. You can also use definitions to name them.

awarded for this:

```
(define s1 (square 40 "solid" "red")) (define c1 (circle 40  
"solid" "green")) (define p1 (star-polygon 20 10 3 "solid" "blue"))
```

Task 1.5

Try these definitions in the REPL by entering `s1`, `c1`, and `p1`. What happens when you enter `(s1)` ? Why? ŷ

Task 1.6

You can also insert image files or images from websites into your program, just like in a word processor. Here's how:

- To insert an image file, select the menu item Insert › Insert Image.
- To insert an image from a website, select Copy Image (or similar) from the browser's context menu. You can then insert the image into DrRacket via the menu item Edit › Paste.

Try it out and give the picture a name!

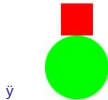
›

DrRacket can also calculate with images:

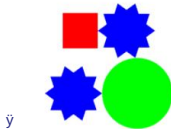
(beside s1 p1)



(above s1 c1)



(above (beside s1 p1) (beside p1 c1))



Task 1.7

Try the following expressions in the REPL:

```
(triangle 50 "outline" "red") (square 100
"solid" "blue")
```

Using the functions `triangle` and `square`, write an expression that calculates the construction of a very simple house.

›

This means that `above` accepts two (or more) images as input and combines them into a single image where the individual images are stacked on top of each other. The same applies to `beside`, except that the images are not...

They can be arranged next to each other. Of course, `above` and `beside` can also be combined. become.

1.5 ABSTRACTION AND APPLICATION

```
elements/tile.rkt
```

```
code
```

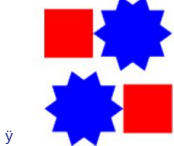
Here are two expressions that combine the images from the previous section into a pattern.

kidneys:

```
(above
```

```
  (beside s1 p1)
```

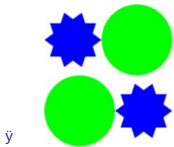
```
  (beside p1 s1))
```



```
(above
```

```
  (beside p1 c1)
```

```
  (beside c1 p1))
```



Both expressions follow the same pattern; they each "tile" two images in a square arrangement. The pattern could be written as follows:

```
(above
```

```
  (beside ab)
```

```
  (beside ba))
```

The first example is created by substituting `s1` for `a` and `p1` for `b`, and in the second example by substituting `p1` for `a` and `p1` for `b`. for `b` then `c1`.

You can put this pattern into a program. Then we don't have to do it `above` every time.

... `beside` ... Type `"beside"`. To do this, first write it exactly as written, with an `"a"` and...

b. When it starts, the following message appears:

a: Variable is not defined

This means that there is no definition for 'a'. You could write one down, for example:

```
(define a s1)
```

But then **a** would be fixed at **s1**. Instead, we want to generalize the pattern so that you can use it multiple times with different values for **a** and **b**. In programming, this generalization process is called *abstraction*. To do this, we need to add something to the pattern to indicate that we want to use different values for **a** and **b**:

```
(lambda (ab)
  (above
    (beside ab)
    (beside ba)))
```

The **lambda** is a kind of magic word; it says something like: "I want to assign values to the variables **a** and **b** later (and perhaps several times)." If you start the program now, the error message will disappear and the following will appear in the REPL:

```
#<function>
```

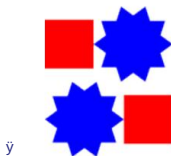
This indicates that the **lambda** output is a function. To do something with it, give it a name using **define**:

```
(define tile
  (lambda (ab)
    (above
      (beside ab)
      (beside ba))))
```

(Make sure everything is properly aligned! «To tile» means «kacheln» in German.⁴)

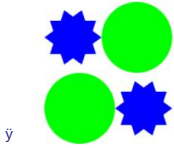
This has resulted in a new operation. You can use it directly:

```
(tile s1 p1)
```



⁴ We prefer English names for variables. However, this is purely a personal preference – you are welcome to use German variables for your own programs.

```
(tile p1 c1)
```



What happened there? The easiest way to see it is in a special tool, the *stepper motor*.

To use it, make sure the following program is in the definition window:

```
(define s1 (square 40 "solid" "red")) (define c1 (circle 40
"solid" "green")) (define p1 (star-polygon 20 10 3 "solid"
"blue"))
```

```
(define tile
  (lambda (ab)
    (above
      (beside ab)
      (beside ba))))
```

```
(tile s1 p1) (tile
p1 c1)
```

Then press the Stepper button (or Step in the English version) in the top right corner of the DrRacket window. A new window will appear that looks like this:

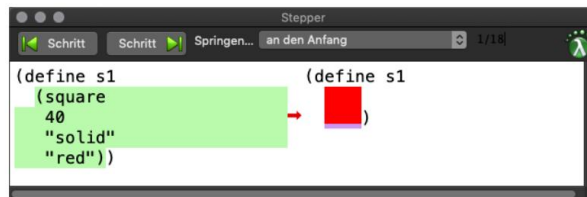


Figure 1.4: The stepper in action

If you now press the button with the forward or backward arrow, you can watch DrRacket execute your program. If you move forward a few steps, it will look like this:

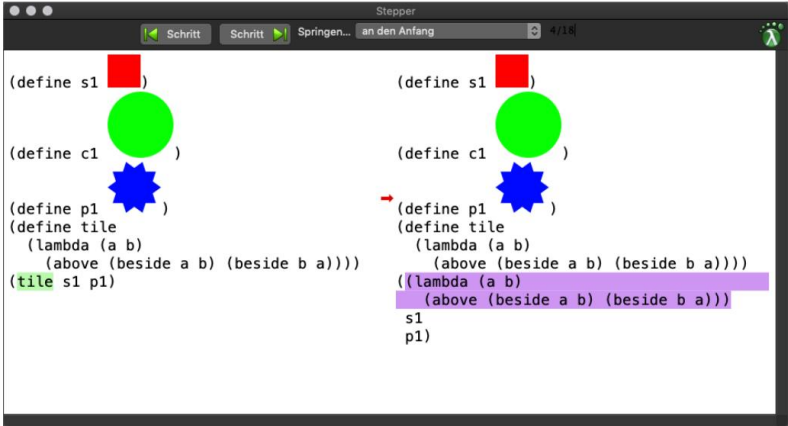


Figure 1.5: Stepper: Replacing variables

You can see how DrRacket evaluates one step at a time – shown in green on the left – and then replaces it with the result on the right. Above, you can see how the stepper replaces the variable `tile` with the lambda expression from the definition of `tile`. This does

the stepper also with the definitions of `s1` and `p1`:

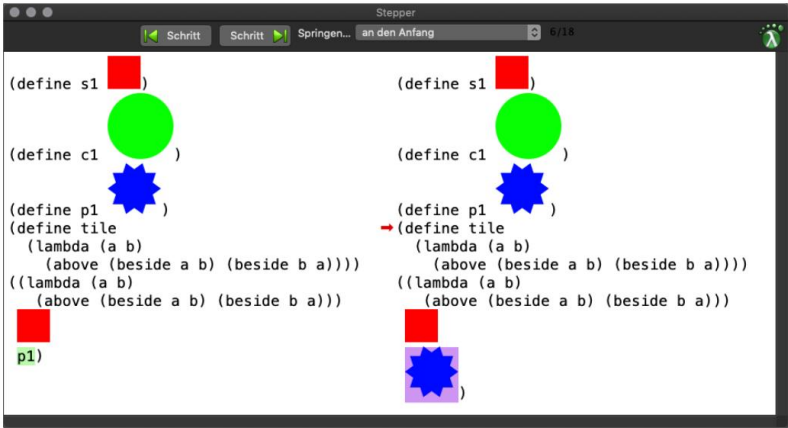


Figure 1.6: Stepper: Variable

Things get interesting after that. The variables from the definitions have all been replaced. Behind the lambda- The expression now consists of the square and the star, and these are now used for the variables from the

lambda expression inserted, i.e., the square for *a* and the asterisk for *b*:

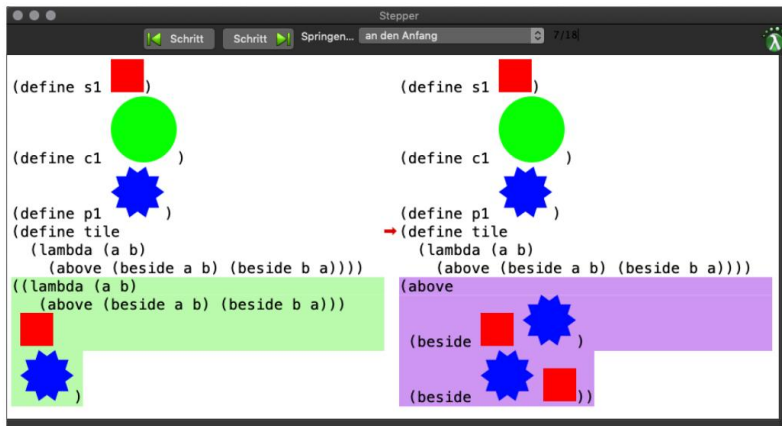


Figure 1.7: Stepper: Application

The lambda value also disappears .

Task 1.8

Try out the example in the stepper and click through to the end. Make sure you understand...

how the left side relates to the right side. y

`(tile s1 p1)` is a so-called *application*: an expression enclosed in parentheses. The operator and operands of the application are also expressions. The operator results in a function – either built-in or derived from a lambda expression. An application is also often called a *function call* or *function application* .

If the function is a lambda expression as in Figure 1.7, then it must have just as many Enter operands, such as the variables in the lambda expression. Then DrRacket evaluates the expression. as follows:

1. The operands are evaluated and provide the inputs for the function, the so-called-ten *arguments*.
2. The arguments are passed for the variables of the lambda expression – the so-called *parameters*-ter – inserted in the inner part of the lambda expression, the *trunk* .

Lambda expressions are *also called abstractions*. Figure 1.8 summarizes how abstractions work. and applications are built and evaluated.

An abstraction has the following form:

`(lambda (p1 . . . pn) e)`

The p_i are names, the *parameters*, and e is the *body*.

The p_i may occur in e .

The value of an abstraction is a *function* that expects an input for each parameter.

An *application* of a function has the following form:

`(f a1 . . . an)`

f is an expression that must result in a function; the a_i are also expressions, the *arguments*. When evaluating an application, f and the a_i are evaluated first; then the body of the function is evaluated, with the values of the arguments a_i being substituted for the parameters p_i .

Figure 1.8: Abstraction and Application

The difference between abstraction and function may be confusing: Abstraction-
tion is the lambda expression in the program, the function is the value that results from the program's execution. However, in this book we often use the term "function" for an abstraction because we are interested in the result of that abstraction.

Task 1.9

Can you use the result of one `tile` application as input for another `tile` application? Try it out! $\ddot{y}$

1.6 INFORMATION AND DATA

A definition such as

`(define value added tax 19)`

This suggests that the number 19 has a meaning "in the real world" at this point, for example, in a program that controls a cash register or helps with tax returns. The meaning could be the following statement: "The VAT rate is 19%." This sentence represents *information*, that is, a fact about the world, or at least the part of the world in which the program is supposed to operate. In computer programs, information is transformed into a simplified form that the program can use for calculations—in this case, the number 19. This simplified form is called *data*: data are *representations* of information.

One of the most important tasks in programming is choosing the right format for the data to represent the information relevant to the program and to make the information

A semicolon (;) denotes a *comment*. The comment extends from the semicolon to the end of the line and is ignored by DrRacket.

Figure 1.9: Comments

then to translate it into data.

It is not always obvious what information is represented by certain data.

The number 23, for example, could represent a range of information:

- the number of hairs Bruce Willis has
- the current outside temperature in °C in Tübingen • the maximum outside temperature on July 1, 2000 in °C in Tübingen • the size in m² of the bedroom • Alexandra Popp's jersey number. Which one is meant depends on

the context. Therefore, in order for others to be able to read our programs, we will have to repeatedly clarify what the context is and how information is to be translated into data and vice versa. We can (and should) add a comment to this using concrete examples:

```
(define bruce-willis 23) ; Number of hairs (define temperatur-  
tübingen 23) ; 23°C in Tübingen (define temperatur-tübingen-2000-07-01  
23) ; 23°C in Tübingen on January 7, 2000 (define bedroom  
23) ; Bedroom is 23m² (define alexandra-popp  
23) ; Back number
```

2

The semicolon marks the rest of the line as a *comment*. DrRacket ignores this line when evaluating the output, but it is useful for human readers. Figure 1.9 summarizes how the semicolon works.

1.7 SYSTEMATICALLY DESIGNING PROGRAMS

elements/electricity tariff.rkt code

In section 1.5, we showed how abstraction and application work in program evaluation. Understanding this is important, but you don't need to constantly think about how DrRacket evaluates your program while writing it. Accordingly, in this book, we mainly focus on how to construct programs systematically. Let's look at this using a small but realistic example:

Consider the following electricity tariffs. Both tariffs consist of a monthly basic charge and a component that depends on the kilowatt hours (kWh) consumed.

	Basic fee per month	Consumption price per kWh
"Cheap electricity" tariff	0.19 €	8.20 €
Tariff «Watts for little»	0.16 €	4.90 €

1. Write a program that accepts the monthly consumption in kilowatt hours and calculates the monthly invoice amount. The monthly invoice amount is calculated based on the "Cheap Electricity" tariff.
2. Write a program that accepts the monthly consumption in kilowatt hours and calculates the monthly invoice amount payable in the "Watts for little" tariff.

Let's start with the first part of the task!

Function First of all: It says "Write a program...". In the teaching language of this book, this is always "Write a *function* ...". Later we will write programs that have "auxiliary functions" in addition to the "main function", but the principle always remains the same.

If it says "that accepts the monthly consumption in kilowatt hours", then it means This means that the function should accept the monthly consumption as input – i.e., as an argument.

Similarly, "and calculates the monthly invoice amount payable" means that The function should return this invoice amount as a result.

Functions that solve a problem within a given task should always have a name – we should come up with it right at the beginning. The name "*cheap-electricity*" is a good choice. We already know that the function accepts consumption in kilowatt-hours. If we also come up with a name for kilowatt-hours, we can write an unfinished program directly without having to think too much about it.

```
(define cheap electricity
  (lambda (kWh)
    ...))
```

Important: Be sure to write down this *outline* , even if you don't yet have any idea what comes next. Always. Every single time.

To calculate the invoice amount, we can write a **formula** : It is the sum of the basic charge and the product of energy (in kilowatt hours) and the price of one kilowatt hour. Accordingly, we need to complete the formula:

```
(define cheap electricity
  (lambda (kWh)
    (+ 4.90 (* 0.19 kWh))))
```

You can now access the `cheap electricity` function :

```
(cheap electricity 10)
↳ 6.8

(cheap electricity 20)
↳ 8.7

(cheap electricity 30)
↳ 10.6
```

The ``cheap-electricity`` function is a relatively small program. As programs grow larger, it's often no longer immediately clear what the individual program components do or whether they are actually correct. Therefore, it makes sense to supplement the function with some program elements that improve its clarity and also increase the likelihood that the function is correct. In the following sections, we will examine these program elements individually.

1.8 SUMMARY

To improve readability, we preface each function with a short description, the so-called *descriptor*, which explains what the function does. A single concise line is sufficient. It could look like this:

Calculate the monthly bill for the cheap electricity tariff

```
(define cheap electricity ...)
```

1.9 SIGNATURE

The readability of the program is further supported by the *signature*, which describes what kind of data the function expects as arguments and what kind it provides.

The number of kilowatt-hours on the meter reading slip is usually a non-negative integer. The output, however, can contain two decimal places and is therefore a fraction. To write the signature, we use mathematical terms to describe these *types*. A non-negative integer is also called a *natural number*. A fraction is also called a fraction.

A *signature* describes a set or type of values.

The signature `string` describes a string of characters.

The following number signatures are built in:

`natural` numbers (0, 1, 2 ...) `integers` whole
 numbers (0, 1, $\sqrt{1}$, 2, $\sqrt{2}$, ...) `rational` fractions or rational
 numbers real numbers any numbers
`real`
`number`

This list forms a "tower": Every natural number is also an integer (negative numbers are added), every integer is also a rational number (fractions with denominators other than 1 are added), every rational number is a real number (non-rational numbers like $\sqrt{2}$ are added), and every real number is also a number (complex numbers are added).

Signatures for functions have the form

`(s1 . . . sn -> s)`

The signatures `s1 . . . sn` are the signatures for the arguments of the function and `s` is the signature for the result.

Figure 1.10: Signatures

A *signature declaration* for a function has the following form:

`(: f (s1 . . . sn -> s))`

It states that the behavior of the function corresponds to the signature: if the inputs of the function `f` correspond to the signatures `s1 . . . sn`, then the output corresponds to the signature `s`.

Figure 1.11: Signature declarations

rational number. The signature declaration for `cheap electricity` should come directly after the short description and looks like this:

`(: cheap electricity (natural -> rational))`

The signature "`natural`" is responsible for natural numbers, "`rational`" for fractions.

This signature declaration consists of the form `(: ...)`, the name of the function, and its signature, `(natural -> rational)`. This signature has an arrow `->` in the middle and therefore represents a function. To the left of the arrow is what the function accepts as input, and to the right of the arrow is the output. The signature declaration can thus be read as: "`Cheap-electricity` is a function that accepts a natural number as input and produces a rational number as output."

A test has the following form:

```
(check-expect te)
```

When the program is running, DrRacket evaluates the test expression `t` and checks whether it matches the value of the expression `e`. If not, the test fails and is logged by DrRacket.

Figure 1.12: Tests

Figures 1.10 and 1.11 describe the format of signatures and signature declarations.

more precisely.

1.10 TESTS

We tested the `cheap-electricity` function in the REPL above by having DrRacket evaluate several calls with different inputs. Using these examples, we can also check (perhaps with a calculator) whether the function produced correct results.

These example calls are even more helpful for readers when they appear in the program code. Paired with the desired results, they facilitate understanding. In the program, we write it as follows, immediately after the code signature:

```
(check-expect (billig-strom 10) 6.8) (check-expect
(billig-strom 20) 8.7) (check-expect (billig-strom
30) 10.6)
```

If we now run the program, the REPL will say:

```
All 3 tests passed!
```

The new program element `check-expect` (see Figure 1.12) performs a so-called *test case* or *test*. In this case, DrRacket reports that the tests were successful.

But the program might contain a bug. If we "accidentally" write `5.90` instead of `4.90` in the definition of `cheap-electricity`, the output will look different:

Check error:

```
The actual value 7.8 is not the expected value 6.8.
```

```
in stromtarif.rkt, line 5, column 0
```

```
The actual value of 9.7 is not the expected value of 8.7.
```

in stromtarif.rkt, line 6, column 0

The actual value of 11.6 is not the expected value of 10.6.

in stromtarif.rkt, line 7, column 0

(We saved the code under the name stromtarif.rkt, which is why this filename appears in the error messages.)

The tests thus point out potential errors. This is so valuable that we will *always* write tests in the future, specifically after the signature. In particular, we will write the tests *before* we even begin the definition. This can be annoying at times, but it makes psychological sense so that we think about the correct behavior of the function beforehand. If we write the tests afterward, there's a strong temptation to simply call the function in the REPL and copy the actual result into the test without checking it for correctness.

For the three tests above, we might need to use a calculator. But there are
There is almost always at least one possible test, which is very simple. In this case, it is:

```
(check-expect (billig-strom 0) 4.9)
```

We can directly obtain the desired result from the table. The test only checks the base amount and not the price per kWh – so further tests are needed – but it is easy to write.

How many tests should you write? That depends on the complexity of the function. But
There should be three for simple functions, more for more complicated ones.

Another criterion is that every part of the program we've written should be evaluated at least once during testing. DrRacket can help you with this. If you reduce the program to just the function definition (i.e., remove all tests and example calls) and then run it, the body of `billig-strom` will be highlighted in blue. (Parts of the signature of `billig-strom` will also be highlighted.) This highlighting indicates the so-called *coverage*: The blue-highlighted program parts have never been run and therefore haven't been tested. Tests are still missing. In the future, we'll make sure that nothing remains blue – otherwise, we'll have to write more tests. We'll return to this topic in section 2.3.

For programs that generate images, it's usually not possible to write the test cases in advance. However, you can try out examples you've thought of beforehand and check them visually. If a result appears correct, you can copy the image from the REPL into the program. In the REPL, it might look like this:

```
(square 40 "solid" "red")
```



You can then copy the square from the REPL, for example by selecting it with the mouse.
and then copy with Ctrl-C or Command-C and then paste into the program with Ctrl-V or Command-V and
then perform the following test:

```
(check-expect (square 40 "solid" "red")
```



Task 1.10

Write two tests for `tile` !

ÿ

Incidentally, a signature declaration is also a kind of test case. We see this when we intentionally make a mistake like this one:

```
(check-expect (cheap-electricity "30") 13.0)
```

Try it out! Along with the test results, a message about a... will appear.

Signature violation as follows:

Signature violations:

received "30" in stromtarif.rkt ..., signature in stromtarif.rkt ...

The message contains clickable links that show you where the signature violation occurred and which signature was violated.

If signature violations appear, you should read them carefully because they indicate a problem. point to your program and give valuable hints for its solution.

1.11 CONSTRUCTION INSTRUCTIONS

In the previous section, we learned about the following work steps that lead to a function.
lead:

- Scaffolding
- Brief description
- Signature declaration
- Tests
- Hull

This order is due to the order in which they were introduced in this chapter. Going forward, it makes sense to always perform these steps in the same order, namely as follows:

- Short description •

- Signature •

- Tests

- Scaffolding

- Hull

This also corresponds to the order in which the relevant elements appear in the program: If you follow this order, you can simply "write down" your program.

In this book, we refer to such instructions for constructing programs as "*construction instructions*." Since many more construction instructions will be added, they will be numbered. The first construction instruction defines the process and is therefore given the special number 0. We will add two more elements shortly—data analysis and templates—which we will explain in later chapters.

Construction Instructions 0 (Procedure)

When constructing a function, proceed in the following order: 1. Short description

2. Data analysis 3.

Signature

declaration 4. Tests 5.

Scaffolding

6. Templates

7. Hull

For each of these steps, there is a set of construction instructions, and for some, several depending on the situation. It might seem overly bureaucratic to always follow the same sequence and always proceed according to the respective construction instructions. It is bureaucratic, certainly, but it's also a valuable aid that prevents you from getting stuck. If you ever reach a dead end while solving a problem, the first thing you should check is whether you followed the construction instructions. Often, the cause of the problem is that one of the steps is missing.

First, we will introduce construction guidelines for the steps of the process that we already on the list:

Construction Instructions 1 (Short Description)

First, write a comment for the function that briefly describes its purpose. One sentence, which should fit on one line, is sufficient. Example:

`Calculate the monthly bill for the cheap electricity tariff`

The quick guide to data analysis, which you might be missing at this point, will only be available later. in Chapter 2. It is not needed at this point.

Construction instructions 2 (signature declaration)

Write a signature declaration for the function directly below the short description. First, think of a name that is as descriptive as possible. Then consider what types of inputs and outputs the function will have and write a signature that describes them as precisely as possible. Example:

`(: cheap electricity (natural -> rational))`

When using number signatures, pay particular attention to ensuring they are as precise as possible. For example, the signature `(number -> number)` would also be correct for the function " `billig-strom` ", but not in this way. Exactly.

Construction Instructions 3 (Tests)

Add tests for the function below the signature. Think of simple yet interesting examples of function calls and define the desired outcome. Turn these examples into tests using ``check-expect``. Example:

`(check-expect (billig-strom 0) 4.9) (check-expect (billig-strom 10) 6.8) (check-expect (billig-strom 20) 8.7) (check-expect (billig-strom 30) 10.6)`

Make sure that the tests ensure that the code of your function is fully covered by the tests.

The last sentence of the construction instructions doesn't make sense with regard to the "cheap-electricity" example , since each of the tests already ensures that addition and multiplication are evaluated in "cheap-electricity" . However, in section 2.3 , we will program examples where it's not a given that the tests will evaluate everything.

Construction instructions 4 (scaffolding)

Write a framework for the function below the tests. For this, you take the name from...

the signature declaration in a function definition such as:

```
(define cheap-electricity
  (lambda (...)
    ...))
```

Think of names for the function's inputs. There must be as many of these as there are inputs in the signature. Then write these names as inputs into the lambda abstraction.

Example:

```
(define cheap electricity
  (lambda (kWh)
    ...))
```

For the fuselage, first write three dots ... (also called *an ellipse*) as a reminder that you will need to fill this in later.

Construction instructions 5 (hull)

As a final step, use your knowledge of the problem to fill in the body of the function.

```
(define cheap electricity
  (lambda (kWh)
    (+ 4.90 (* 0.19 kWh))))
```

We will demonstrate this process using the second part of the electricity tariff task. It involved... about the tariff "Watts for Little".

Brief description Let's start with the brief description:

Calculate the monthly billing amount for the Watt tariff for a low price

Signature We can also formulate the signature declaration analogously to "cheap electricity" – we just need to choose a new name:

```
(: watt-for-little (natural -> rational))
```

Tests: We need to reformulate the tests. It's always a good idea to start with the simplest example:

```
(check-expect (watt-for-little 0) 8.2)
```

Otherwise, we will again use progressively larger consumption values and calculate the cost.

The amount of the amount in the head is:

```
(check-expect (watt-for-little 10) 9.8) (check-expect
(watt-for-little 20) 11.4) (check-expect (watt-for-little 30)
13.0)
```

Framework The framework results directly from the signature:

```
(define watt-for-little (lambda
(kWh) ...))
```

Hull Finally, we still need to complete the hull by adding the appropriate Write in the formula:

```
(define watt-for-little (lambda
(kWh)
(+ 8.20 (* 0.16 kWh))))
```

Complete!

1.12 THE POWER OF ABSTRACTION

We wrote completely independent solutions for the two parts of the problem regarding electricity tariffs. These differ only in details – in both cases, the electricity tariff is calculated using the same formula, consisting of a basic charge and a consumption price per kWh. The definitions are therefore similar:

```
(define cheap electricity
(lambda (kWh)
(+ 4.90 (* 0.19 kWh))))
```

```
(define watt-for-little (lambda
  (kWh)
  (+ 8.20 (* 0.16 kWh))))
```

This “doubling” is unsatisfactory and can also cause problems later: If an error is discovered, two functions may need to be corrected.

It would therefore be good if we could somehow summarize the similarities between the two functions, thus *abstracting them* from “cheap electricity” and “watts for little”. To do this, we copy the function one last time and rename it. We also replace all the parts where the two functions differ with a variable, for example, “basic charge” and “per kWh”:

```
(define electricity tariff-invoice amount
  (lambda (kWh)
    (+ basic charge (* per kWh))))
```

The two new variables are not yet bound; we need to add them to the **lambda** parameters :

```
(define electricity tariff-invoice amount
  (lambda (basic charge per kWh)
    (+ basic charge (* per kWh))))
```

We'll add a short description and signature. Both new parameters are also rational numbers, so it looks like this:

```
; Calculate monthly bill amount for electricity tariff (: electricity tariff-billing
amount (rational rational natural -> rational))
```

Furthermore, we should formulate tests. We can copy these from the tests for *cheap electricity* and *watts for little*. We only need to add the basic fee or the price per kWh to each argument:

```
(check-expect (stromtarif-rechnungsbetrag 4.90 0.19 0) 4.9)
; Cheap electricity
(check-expect (electricity tariff-invoice amount 4.90 0.19 10) 6.8)
; Cheap electricity
(check-expect (electricity tariff-invoice amount 4.90 0.19 20) 8.7)
; Cheap electricity
(check-expect (electricity tariff-invoice amount 4.90 0.19 30) 10.6)
Cheap electricity
```

```
(check-expect (stromtarif-rechnungsbetrag 8.20 0.16 0) 8.2)
```

Watts for little

```
(check-expect (stromtarif-rechnungsbetrag 8.20 0.16 10) 9.8)
```

Watts for little

```
(check-expect (stromtarif-rechnungsbetrag 8.20 0.16 20) 11.4)
```

Watts for little

```
(check-expect (stromtarif-rechnungsbetrag 8.20 0.16 30) 13.0)
```

Watts for little

Finally, we can also change the definitions of `cheap-electricity` and `watt-for-little` so that they no longer calculate the invoice amount "themselves", but instead use the function `electricity-tariff-invoice-amount` :

```
(define cheap-electricity
```

```
  (lambda (kWh)
```

```
    (electricity tariff-invoice amount 4.90 0.19 kWh)))
```

```
(define watt-for-little (lambda
```

```
  (kWh)
```

```
    (electricity tariff invoice amount 8.20 0.16 kWh)))
```

We will use this technique frequently in this book. It not only saves time and makes our programs easier to use, but sometimes leads to entirely new insights – as can be seen, for example, in Chapter 9.

1.13 MANTRAS FOR PROGRAMMING

Following the simple programs we have covered so far, in the following chapters we will write increasingly interesting functions and introduce further construction instructions for specific situations.

However, there are some overarching principles of programming that we will revisit repeatedly. Not all of them are relevant right now, but we're listing them here so they're gathered in one place and we can refer to them in the following chapters.

The first mantra is intended to counteract a phenomenon we have often observed in our students, who sometimes give up on an assignment before they have written down everything they know.

Mantra 1

Make sure that everything you know is actually covered in the program.

The next mantra sounds like child's play:

Mantra 2

Read what it says there.

We find reading carefully surprisingly difficult. For this book, two aspects of reading are particularly important. Most importantly, read each task step by step – just enough to understand the next step – instead of trying to do everything at once. Furthermore, DrRacket (or other programming environments) often provides useful feedback that is worth reading, even if it is sometimes tedious.

The design instructions help you to read the task in a structured way and to write what you know in your program. From this follows the next mantra:

Mantra 3:

Follow the construction instructions. If you find yourself in a dead end, check if you have applied all relevant construction instructions.

The construction instructions help us to make progress – even if we don't understand everything yet – by reading what is written there and writing down what we know.

We already encountered the following mantra in section 1.12 on page 42 :

Mantra 4: If

you see two program parts that differ only in a few places and are related in content, abstract!

So far, we have only written small programs with one, two, or three functions: Larger Programs consist of many functions, and this mantra names the principle by which we divide them into functions:

Mantra 5: If

your program calculates multiple things that you can give different names to, write a separate function for each one.

We will often use mathematical insights in the form of equations to improve our programs:

Mantra 6

Use equations to simplify your program.

Sometimes it's fun to piece together a complicated program like a puzzle. Unfortunately, this often leads to undesirable complexity: Complex programs are prone to errors, difficult to understand, and difficult to develop further. We should beware of this.

Mantra 7

If your function seems complicated to you, it is likely that it contains errors: either because it should actually be simpler or because the task is complicated and therefore its solution is prone to errors.

The next mantra describes how you can develop particularly good representations. The details are in Chapter 5 on page 141.

Mantra 8

Look for combinators for your data that make bigger things out of small things – and then make even bigger things out of those.

TASKS

Task 1.11

Let C be a temperature in degrees Celsius. The temperature F in degrees Fahrenheit is calculated as:

$$F = \frac{9}{5}C + 32$$

1. Write a function `from Celsius to Fahrenheit` that accepts a temperature in °C as input and returns a temperature in °F. Use the construction guide for this.
2. Write a function `Fahrenheit->Celsius` that accepts a temperature in °F as input and returns a temperature in °C. Use the construction instructions for this as well!

Task 1.12

Multiplying strings:

- Write a `double-string` function that accepts a string and "doubles" it, that is, for an input `"Sparrowhawk"` the return value `"SparrowhawkSparrowhawk"` delivers.
- Write a function `quadruple-string` that accepts a string and quadruples it `facht»`.
- Write a function `octuple-string` that accepts a string and `«disdains-facht»`.
- Write a function `sixteentuple-string` that accepts a string and `«ver-sixteen times over.»`

Task 1.13

Internet time is an alternative concept of time measurement, in which the day is not divided into 24 hours. It is not divided into 60 minutes of 60 seconds each, but into 1000 so-called .beats. One such .beat It is therefore 1 minute 26 seconds and 4 tenths of a second long; for further subdivision there are two decimal places.

The time is indicated by an @ symbol before the value. @0.00 is equivalent to 0:00.

Central European Time.

For example, the time 03:02, 9 seconds and 6 tenths of a second corresponds to @126.50 .beats.

Write a function ``time->beats`` that calculates the internet time in beats from a given time (hours, minutes, seconds, and tenths of a second). The function takes four inputs: hours, minutes, seconds, and tenths of a second.

Use abstraction and write helper functions where they solve subproblems solve!

The built-in functions `floor` and `round` are helpful in solving this task :

The first function rounds down, the second rounds up or down, depending on whether the decimal part below or above 0.5.

Task 1.14

In the USA and in Europe, there are different measures for the energy efficiency of motor vehicles.

witnesses:

- In Europe, the common measure is *fuel consumption* in liters per 100 km (l/100km);

- In the USA, the common measure is *range* in miles per gallon (mi/gal).

Write functions that convert between the two units of measurement. Proceed as follows:

When writing any function, follow this process: First, write the short description and signature.

Next, write some test cases. Then, derive the framework from...

from the signature and complete the body of the function.

1. Write a function ``liters-per-hundred-kilometers`` that accepts a quantity of gasoline in liters and the range of that gasoline in kilometers, and calculates the fuel consumption in liters.
Liters per 100 km calculated.
2. Write a function ``miles-per-gallon`` that accepts a distance in miles and the fuel consumption for that distance in gallons, and calculates the range in miles.
calculated per gallon.
3. Define a constant `kilometers-per-mile` (one US mile is approximately 1.61 kilometers) and write two functions `kilometers->miles` and `miles->kilometers`.
each accept a distance in one unit of measurement and convert the distance into the respective
Convert to another unit of measurement.
4. Define a constant `liters-per-gallon` (one gallon is approximately 3.79 liters)
and write two functions `liters->gallons` and `gallons->liters`, each accepting a quantity in one unit of measurement and converting the quantity to the other unit of measurement.
convert.
5. Write the function `l/100km->mi/gal` that accepts a consumption in liters per 100 km.
and converts the range into miles per gallon. Use the functions that
You created the other subtasks. Should you encounter further sub-problems,
Abstract these sub-problems into their own functions.
6. Write the function `mi/gal->l/100km` that accepts a range in miles per gallon and converts it to fuel consumption in liters per 100 km. Use the functions for this,
that you created in the other subtasks. Should you encounter further sub-problems,
Abstract these sub-problems into their own functions.
7. Find out the fuel consumption of various vehicles that you use daily.
You see this in road traffic in Germany. Compare this consumption with the range specifications of typical vehicles for the US market.

Task 1.15

A time clock is installed in a company, which records the time spent at the start and end of the workday. stamps the current time on a card. A time is described by hours, minutes, and seconds. Through skillful negotiations by the works council, night shifts have been avoided.

offered. Work starts no earlier than 6 a.m. and ends no later than 10 p.m.

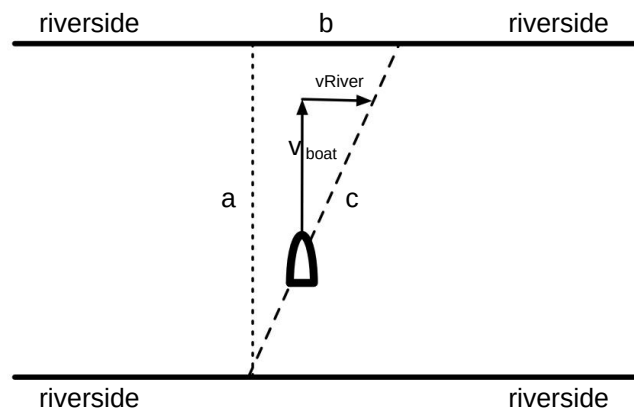
Each worker in this factory earns €72 per hour.

Write a function called `money-earned` that calculates how much money was earned from two given times (start and end of work). It is sufficient to specify the amount in cents. First, convert the times to seconds. The function has six inputs: start hour, start minute, start second, end hour, end minute, end second.

Use abstraction and write helper functions where they solve subproblems!

Task 1.16

A boat crosses a river with a current and is blown off course by the current. This increases the distance the boat actually travels.



Given the width of the river a , the flow velocity of the river v_{Fluss} , and the speed of the boat v_{Boot} , calculate the length of the distance the boat actually travels. To do this, program functions that solve the following sub-problems: 1. Write a function `crossing-time` that calculates the time it takes to cross the river.

of the river is needed:

$$t = \frac{a}{v_{\text{Boot}}}$$

2. Then write a function called `other-shore-offset` that calculates the length of the line segment the boat is carried away (i.e. the offset on the other bank, in the diagram the distance `b`).
3. To calculate `c`, you need the *Pythagorean theorem*:

$$a^2 + b^2 = c^2$$

Write a function `hypotenuse` that calculates `c` in the equation above.

To calculate the square root of a number, you can use `sqrt`, which is the following:

Signature declaration has:

```
(: sqrt (number -> number))
```

Should you encounter further sub-problems, abstract these sub-problems into their own functions. no.

4. Finally, write a function called `boat-travel-distance` that calculates the actual distance the boat travels. Use the functions you have written so far to do this.

2. CASE DISTINCTIONS AND BRANCHES

Computer programs sometimes have to differentiate between various possibilities when processing data: Is the water temperature warm enough for bathing? Which of five Tupperware containers is large enough for a specific amount of potato salad? Which is the correct turn to Dortmund? Such decisions are based on the fact that a value can belong to one of several different categories – this is known as a *case distinction*; functions operate on case-distinguished data through *branching*.

In this chapter, we will first discuss two particularly simple types of case distinctions: *enumerations* and *number ranges*. In Chapter 4, we will revisit case distinctions and program them with so-called *mixed data*.

2.1 Calculating with Boolean values

For the programs in this chapter, we need a new type of data. This data is generated when we compare, for example, two numbers:

```
(< 0 5)
```

```
→ #t
```

(< 0 5) is the notation for $0 < 5$. The `#t` stands for "true" because the statement "is 0 less than 5" is indeed true. Conversely, the result is not `#t`:

```
(< 5 0)
```

```
→ #f
```

The `#f` stands for «false», because this statement is not true.

"True" and "false" together are called *Boolean values* or *truth values*.¹ An expression-

A pressure that produces a boolean value is accordingly also called a *boolean expression*.

We will often refer to Boolean expressions as *conditions*. If a condition returns `#t`, we will also use the terminology that the condition is *true* – or, if it returns `#f`, that it is *not true*.

¹ Boolean values are named after *George Boole* (1815–1864), who was the first to formulate an algebraic approach to the treatment of logic using the values "true" and "false".

`<` is a built-in function that tests for "less than", thus corresponding to the mathematical operator `<`. Similarly, there are `>` for "greater than" (mathematics: `>`), `=` for "equal to" (mathematics: `=`), `<=` for "less than or equal to" (mathematics: `≤`) and `>=` for "greater than or equal to" (mathematics: `≥`).

Analogous to the equals sign (`=`) for numbers, strings can be compared using `string=?`:

```
(string=? "Mike" "Mike")
⇒ #t

(string=? "Herbert" "Mike")
⇒ #f
```

`#t` and `#f` are literals, like numbers. Therefore, they can also appear in programs:

```
#t
⇒ #t
#f
⇒ #f
```

Programs can also perform calculations with Boolean values. An expression of the form

```
(and e1 e2 . . . en)
```

The operation `(and e1 e2)` always results in `#t` if all `ei` result in `#t`, otherwise `#f`. With two operands `e1` and `e2`, `(and e1 e2)` always results in `#t` if `e1` and `e2` result in `#t`.

```
(and #t #t)
⇒ #t

(and #f #t)
⇒ #f

(and #t #f)
⇒ #f

(and #f #f)
⇒ #f
```

Corresponding to the "and" expressions with "and", there are also expressions of the form that use "or".
implement:

```
(or e1 e2 . . . en)
```

which always result in `#t` if *either* of the operands results in `#t`, otherwise `#f`. With two operands `e1` and `e2`, `(or e1 e2)` always results in `#t` if either `e1` or `e2` results in `#t`:

```
(or #t #t)
```

```
ŷ #t
```

```
(or #f #t)
```

```
ŷ #t
```

```
(or #t #f)
```

```
ŷ #t
```

```
(or #f #f)
```

```
ŷ #f
```

The ``or`` function thus calculates an *inclusive* OR: `(or e1 e2)` means "e1 or e2 or both", in contrast to the *exclusive* OR "either e1 or e2".²

Furthermore, there is a built-in function ``not`` that inverts a Boolean value .

or its *negation* is calculated, i.e., it behaves as follows:

```
(not #f)
```

```
ŷ #t
```

```
(not #t)
```

```
ŷ #f
```

Boolean values have the signature `boolean`.

Task 2.1

Write a function that calculates the exclusive OR of two Boolean values.

ŷ

2.2 BRANCHES

```
case distinctions/say-number.rkt
```

code

To demonstrate case distinctions, let's consider the following example problem: We write a function that is supposed to "recite" a number. It should behave as follows:

```
(say-number 0)
```

```
ŷ "zero"
```

```
(say-number 1)
```

```
ŷ "one"
```

² In computer science, "or" without any further specification almost always refers to an inclusive or. Those who mean an exclusive or say "exclusive or".

For the sake of simplicity, we will initially limit ourselves to the numbers from zero to three. The function has the following brief description:

Convert number to text

The function converts a natural number into a string and has the following signature:

```
(: say-number (natural -> string))
```

Unfortunately, the signature doesn't tell us that the function only works up to three – we'll describe how to make the signature more precise later. Here, however, we want to focus on the function definition. But first, let's turn the examples above into test cases:

```
(check-expect (say-number 0) "zero") (check-expect
(say-number 1) "one")
```

The framework is derived directly from the signature:

```
(define say-number (lambda
  (n) ...))
```

But how do we proceed from here? The input can be broken down into four cases – 0, 1, 2, and 3 – thus forming a *case distinction*. To process a case distinction in the input of a function, we need a new program element: the *conditional statement*. Conditions begin with the word ``cond`` and are among the most complex program elements we will use in this book. But don't worry, it won't be that difficult. To write a conditional statement, we need to know *how many* cases there are. In our example, there are four. The conditional statement for this has the following form:

```
(define say-number (lambda
  (n)
    (cond
      (... ...) (... ...)
      (... ...)
      (... ...))))
```

Each of the program segments `(... ...)` is a so-called *branch*. The first part of a branch is always a condition, which should return `#t` for the corresponding case and for the other cases.

#f. In this case, the condition of each branch must identify one of the numbers from zero to three:

```
(define say-number (lambda
  (n) (cond

    ((= n 0) ...)
    ((= n 1) ...) ((= n
2) ...)
    ((= n 3) ...))))
```

The second part of each branch represents the desired result for the corresponding case. If we fill it in, the result looks like this:

```
(define say-number (lambda
  (n) (cond

    ((= n 0) "zero")
    ((= n 1) "one") ((= n 2)
"two")
    ((= n 3) "three"))))
```

Complete!

Task 2.2

Find out what happens when you call the function with a number it wasn't designed for ÿ is.

As mentioned above, the signature of ``say-number`` suggests that it can recite any natural number, even though it only works up to three. Fortunately, we can clarify the signature as follows:

```
(: say-number ((integer-from-to 0 3) -> string))
```

The `integer-from-to` function thus provides a signature that describes natural numbers between a lower and an upper limit.

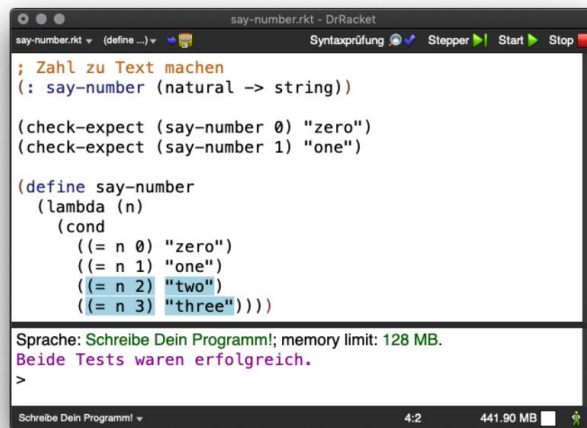


Figure 2.1: Display of coverage in DrRacket

2.3 COVER

Perhaps you noticed with the `say-number` function from the previous section that... Immediately after the program has run, two lines are highlighted in color, as shown in Figure 2.1. This is because these two lines were never evaluated, as the tests only check the first two cases. Theoretically, in the last two branches, for 2 and 3, there could still be a bug, and the tests wouldn't detect it.

If you add the following test cases, the colored background will disappear:

```

(check-expect (say-number 2) "two")
(check-expect (say-number 3) "three")

```

Try it out – ideally, try one first, then the other!

The colored background indicates the *coverage*, as we briefly discussed in section 1.10 on page 37: An expression in the program that is evaluated by the tests, this means *covered*. The expressions highlighted in color are therefore not covered. The last sentence of Design guideline 3 on page 40 states exactly that: A function should be fully covered by its tests. Coverage is therefore a minimum requirement for our tests.

2.4 LISTS

case distinctions/pet.rkt

code

Next example task: We want to divide pets into cute and not-so-cute.

In this world, there are only three different types of pets: cats, dogs, and snakes.

We could describe pets in a commentary in the program like this:

A pet is one of the following:

; - Cat

; - Dog

; - Snake

Such a short comment, describing the data that appear in our task, is called

Data definition. More on this in section 2.6. The wording of the data definition "one of the following" indicates that this type of data also falls into several cases – it is a matter of

So, once again, it's about distinguishing between cases.

Before we solve the actual task (cute or not), we first need to consider how to represent pets in the program: For the purposes of this task, it is sufficient to

We need to know *which* of the three possibilities we are dealing with. We will use the simplest possibility: strings containing the corresponding text, namely "cat", "dog", and "snake".

Such a list of values constitutes a so-called *enumeration* – a special case of a Case distinction.

There is no finished signature yet for the list of pets; we still need to create that. Define. The *signature definition* looks like this:

```
(define pet
  (signature (enum "cat" "dog" "snake")))
```

There are two new program elements included:

- We always write "signature" when we create a new signature.
- The ``enum` function` (which only works within a ``signature`` expression) is responsible for enumerations. An ``enum`` expression contains the values that belong to the enumeration.

Task 2.3

The function `say-number` from the previous section has the following signature declaration:

```
(: say-number ((integer-from-to 0 3) -> string))
```

Write explicit data definitions for the function's input and output, and make its signature more precise using enumerations! ÿ

Back to our cuteness analysis function. We can now use the defined signature `pet` in the signature declaration of our function. Together with the short description, it looks like this:

```
Is pet cute? (: cute? (pet -> boolean))
```

The question mark is part of the function name and is a convention we use for the names of functions that return a Boolean value – that is, functions that answer a yes/no question.

Since there are only three input options, we can write a test for each of the three:

```
(check-expect (cute? "cat") #t) (check-expect
(cute? "dog") #f) (check-expect (cute? "snake")
#f)
```

Let's turn to the function itself. First, the framework:

```
(define cute?
  (lambda (p)
    ...))
```

Since the input is an enumeration, i.e., a case distinction, we need a branch in the body. Because there are three cases in the enumeration, the branch also needs three branches:

```
(define cute?
  (lambda (p)
    (cond
      (... ...)
      (... ...)
      (... ...))))
```

Next, we need to write the conditions, which should compare `p` with `"cat"`, `"dog"`, and `"snake"`, respectively. It might be tempting to write `(= p "cat")` etc. However, this will result in an error message something like this:

=: Number expected as first argument, "cat" received

The function = therefore only handles numbers; for strings we must use the function `string=?`:

```
(define cute?
  (lambda (p)
    (cond
      ((string=? p "cat") ...)
      ((string=? p "dog") ...)
      ((string=? p "Snake") ...))))
```

So far, everything follows purely from the definition of `pet`. Finally, we still need the answers.

Add to that. Cats and dogs are cute, snakes are not:

```
(define cute?
  (lambda (p)
    (cond
      ((string=? p "cat") #t)
      ((string=? p "dog") #t)
      ((string=? p "Snake") #f))))
```

Complete!

2.5 NUMBER RANGES

`case distinctions/heat-water.rkt`

code

Another common type of case distinction occurs with number ranges.

To demonstrate this, let's consider the following task: We write a function that "heats water." That is, we add a certain amount of heat to a certain amount of water at a certain temperature. We want to calculate how hot the water becomes.

After that. For the sake of simplicity, we measure the added heat – just like the temperature – in degrees Celsius (°C).³ We write three versions of the function:

- The first, naive version simply adds the added heat to the initial temperature.
- The second version takes into account that water boils at 100°C and cannot get any hotter.

³ We are simplifying this considerably, but temperature increase and heat are related via heat capacity.

- The third version additionally takes into account that frozen water at 0°C needs to have 80°C of heat added to melt it and still be at 0°C .

We are therefore gradually taking into account that the internal energy of water – the heat – is not the same as its temperature.

Naive Version: We'll start with the naive version and follow the construction instructions. First, a brief description:

Calculating water temperature after heating is naive.

Next comes the signature declaration. The initial temperature and the added heat are the inputs; we represent them as real numbers. The output is the final temperature, also a real number. Since we already know that this version is somewhat simple, we append a -0 to the name:

```
(: heat-water-0 (real real -> real))
```

Now we will write down some simple examples as test cases:

```
(check-expect (heat-water-0 -10 20) 10) (check-expect (heat-  
water-0 10 20) 30) (check-expect (heat-water-0 90 20) 110)
```

Next up is the scaffolding:

```
(define heat-water-0  
  (lambda (temp heat) ...))
```

The hull is no longer rocket science; the two inputs are simply added together:

```
(define heat-water-0  
  (lambda (temp heat) (+ temp  
                        heat)))
```

Just run the tests and we're done!

Boiling Water. Next, we wanted to take into account that water cannot get hotter than 100°C under normal pressure conditions. We only slightly adjust the short description; the signature also remains almost unchanged – we only change the name and replace the 0 with a 1.

Calculate water temperature after heating, taking boiling into account (: heat-water-1 (real real -> real))

For the tests, we can of course copy the tests from `heat-water-0` , but we need to adapt the last one:

```
(check-expect (heat-water-1 -10 20) 10) (check-expect
(heat-water-1 10 20) 30) (check-expect (heat-water-1
90 20) 100)
```

When testing, it is always useful to also test borderline cases – does the function actually switch over? when the 100 are reached.

The following two test cases serve this purpose:

```
(check-expect (heat-water-1 99 1) 100) (check-expect
(heat-water-1 99 2) 100)
```

Since the signature is the same, the framework is also identical:

```
(define heat-water-1
  (lambda (temp heat) ...))
```

From 100°C onwards , the function must calculate its result differently. Or, put another way, the inputs fall into two different classes: those where the sum is less than 100 and those where it is greater.

Therefore, we need a **cond function** with two branches:

```
(define heat-water-1
  (lambda (temp heat)
    (cond
      (... ...)
      (... ...))))
```

We now need a condition to determine whether the sum of `temp` and `heat` is below 100 (more precisely: less than *or equal to* 100, because 100 is just within reach) and another for when the sum is above 100.

These could look like this:

```
(<= (+ temp heat) 100) (> (+
temp heat) 100)
```

If we place these two conditions at the corresponding locations in the hull, it looks like this.

out of:

```
(define heat-water-1
  (lambda (temp heat) (cond
    ((<= (+ temp heat) 100) ...) ((> (+ temp heat)
    100) ...))))
```

In the places following the conditions, we need to insert expressions that yield the correct result in each case. That would then be:

```
(define heat-water-1
  (lambda (temp heat) (cond
    ((<= (+
    temp heat) 100) (+ temp heat)) ((> (+ temp heat) 100) 100))))
```

Complete!

Task 2.4

Do the two conditions absolutely have to use `<=` and `>`? What happens if you replace `<=` with `<` and then run the program? What happens if you then also replace `>` with `>=`? Why does the program still work then? ÿ

Boiling Water and Ice. Let's move on to the "full version". As a reminder: we also need to take into account that frozen water at 0°C requires the addition of 80°C of heat to melt and still remain at 0°C .

We'll start again with the short description:

Calculate water temperature after heating, with ice & boiling

Since this is the last version, it no longer has a number. Otherwise, the signature is unchanged:

```
(: heat-water (real real -> real))
```

We can't simply use the tests unchanged. The very first one no longer works:

```
(check-expect (heat-water -10 20) 10)
```

Heating the water from -10°C to 0 °C requires only 10°C of heat input, but then an additional 80°C of heat must be added before the process continues. Therefore, we need to modify the test case:

```
(check-expect (heat-water -10 20) 0)
```

The other test cases, `heat-water-1`, can remain as they are:

```
(check-expect (heat-water 10 20) 30) (check-expect
(heat-water 90 20) 100) (check-expect (heat-water
99 1) 100) (check-expect (heat-water 99 2) 100)
```

However, a few more tests should clarify exactly what happens around zero and where exactly it is crossed:

```
(check-expect (heat-water -10 5) -5) (check-expect
(heat-water -5 60) 0) (check-expect (heat-water
-5 90) 5) (check-expect (heat-water -1 81) 0)
(check-expect (heat-water -1 82) 1)
```

Again, we should consider the different scenarios into which our inputs decompose. There are three obvious cases: 1.

The initial temperature is below 0 °C, so ice is being heated.

2. Heating would raise the water temperature to over 100°C.

3. Everything else – the water starts out liquid and remains liquid due to the heating.

The first case also has three "sub-cases":

- The heating remains below 0 °C. •

The heating gets "stuck" at 0 °C. • The heating increases the temperature above zero.

Such complex case distinctions are relatively rare. When they do arise, particular care is required.

Therefore, we will work through this as an example.

Let's start again with the framework for the function:

```
(define heat-water
  (lambda (temp heat)
    ...))
```

We already know from the case analysis that there are three cases. Therefore, we also need a **cond** with three branches.

```
(define heat-water
  (lambda (temp heat)
    (cond
```

```
(... ...)
(... ...) (... ..))
```

Now we add conditions that correspond to the three cases. To make the assignment of cases and conditions clear in this complicated case, they are each provided as a comment above:

```
(define heat-water
  (lambda (temp heat)
    (cond
      (The initial temperature is below 0 °C, so ice is being heated. ((< temp 0) ...))
      (Heating would increase the water temperature to over 100°C. ((>= (+ temp heat) 100) ...))
      (Water starts as a liquid and remains liquid when heated. ((and (>= temp 0) (< (+ temp heat)
100)) ...))))
```

Now we can start filling in the branches. Since the first branch (the ice) is more complicated, we'll put that one aside for now. The second branch simply gives 100°C. The third branch, where the water remains liquid, is just as easy – the answer is there (+ temp heat). The current result looks like this:

```
(define heat-water
  (lambda (temp heat) (cond
    (The initial temperature is below 0 °C, so ice is being heated. ((< temp 0) ...))
    (Heating would increase the water temperature to over 100°C. ((>= (+ temp heat) 100)
100)
    (Water starts out liquid and remains liquid when heated. ((and (>= temp 0) (< (+ temp heat)
100)) (+ temp heat)))))
```

That we tackled the easy cases first might seem like laziness. And it is – but it's also a sensible strategy that stems from Mantra 1 :

Mantra 1

Make sure that everything you know is actually covered in the program.

We also know something about the first branch, namely that it itself is a case distinction with three cases. So we can (once again) write down the **cond** :

```
(define heat-water
  (lambda (temp heat)
    (cond ;
      Initial temperature is below 0 °C, so ice is heated. ((< temp 0) (cond (... ...))

      (... ...)
      (... ...)))
    Heating would increase the water temperature to over 100°C. ((>= (+ temp heat) 100) 100)

    Water starts out liquid and remains liquid when heated. ((and (>= temp 0) (< (+ temp
    heat) 100)) (+ temp heat))))
```

Here too, it makes sense to write the case descriptions above the branches. Then we add the conditions with the following result:

```
(define heat-water
  (lambda (temp heat)
    (cond ;
      Initial temperature is below 0 °C, so ice is heated. ((< temp 0) (cond ; Heating remains
      below 0 °C. ((<
      (+
        temp heat) 0) ...)

      ; Heating gets "stuck" at 0 °C. ((and (>= (+ temp heat)
      0) (< (+ temp heat) 80))

      ...)
      Heating raises the temperature above zero. ((and (>= (+ temp heat) 0) (>= (+
      temp heat) 80))

      ...)))
```

Heating would increase the water temperature to over 100°C. $((\geq (+ \text{temp heat}) 100) 100)$

Water starts out liquid and remains liquid when heated. $((\text{and } (\geq \text{temp } 0) (< (+ \text{temp heat}) 100)) (+ \text{temp heat}))))$

The conditions are now even more complicated, but will hopefully become clearer through closer examination:

- "The warming

remains below 0 °C."

This means the sum of the initial temperature and the heating is less than 0 °C. • "The heating gets stuck at 0°C."

In this case, the sum of temperature and heating must be between 0 °C and 80°C .

- "The warming raises the temperature above zero."

The sum of temperature and heating goes not only above 0 °C but also above 80°C .

The answers under these conditions are relatively easy to complete:

(define heat-water

(lambda (temp heat) (cond

The initial temperature is below 0 °C, so ice is being heated. $((< \text{temp } 0) (\text{cond}$

; Heating remains below 0 °C. $((< (+ \text{temp heat}) 0) (+ \text{temp heat}))$

; Heating gets "stuck" at 0 °C. $((\text{and } (\geq (+ \text{temp heat}) 0) (< (+ \text{temp heat}) 80))$

0)

Heating raises the temperature above zero. $((\text{and } (\geq (+ \text{temp heat}) 0) (\geq (+ \text{temp heat}) 80)) (- (+ \text{temp heat}) 80))))$

Heating would increase the water temperature to over 100°C. $((\geq (+ \text{temp heat}) 100) 100)$

Water starts out liquid and remains liquid when heated. ((and (>= temp 0) (< (+ temp heat) 100)) (+ temp heat))))

Done! 4

Almost *finished*. A closer look at the program reveals some room for improvement. Let's start with the condition:

```
(and (>= (+ temp heat) 0) (>= (+
temp heat) 80))
```

A temperature above 80°C is also above 0°C. We can therefore simplify this to:

```
(>= (+ temp heat) 80)
```

This is mathematically plausible, but to be on the safe side we should run the tests again: But they all still run successfully.

Before we simplify the function even further, it makes sense to explain how the output works.

To explore the meaning of conditional expressions:

Task 2.5

Run the [heat-water function](#) in the stepper motor and observe how – and especially in what order – the conditions are evaluated in a condition expression. ⁴

Once you've completed this task, you'll have observed that DrRacket evaluates the conditions in a `cond` expression sequentially. As soon as one of them evaluates to `#t`, DrRacket continues with the expression of that branch. The remaining branches are no longer considered, regardless of whether they could evaluate to `#t`. We can exploit this to further simplify the function. Let's consider the first two conditions in the "inner" `cond`:

```
(cond
  ((< (+ temp heat) 0) (+ temp heat)) ((and (>= (+
temp heat) 0) (< (+ temp heat) 80))

  0)
  ...)
```

⁴ However, that's not quite right yet: You might notice that something isn't quite right. We'll come back to that later.

If the first condition in this section returns `#f`, meaning it's false, then `(+ temp heat)` is greater than or equal to 0. This means the subexpression `(>= (+ temp heat) 0)` in the *next* condition *always* returns `#t`. Therefore, we can simplify:

```
(cond
  ((< (+ temp heat) 0) (+ temp heat)) ((and #t
                                             (< (+ temp heat) 80))
                                         0) ...)
```

An expression `(and #te)` always yields the same result as `e` – look again at page 52 to see for yourself! We can therefore further simplify the inner **cond** to:

```
(cond
  ((< (+ temp heat) 0) (+ temp heat)) (< (+ temp
                                           heat) 80) 0) ...)
```

So we use mathematics (more precisely, *algebra*, the study of equations) to simplify our program. Algebra is a very powerful tool in programming, and we will use it frequently. This is often easier if we don't think about the concrete meaning of the expressions but simply use equations, as is often the case in mathematics and as described in Mantra 6 on page 46.

But we're not finished simplifying yet. Let's now look at the three "outer" ones.

Conditions:

```
(cond
  (< temp 0) ...) (>= (+
                       temp heat) 100) ...) ((and (>= temp 0)
                                                    (< (+ temp heat) 100)) ...))
```

If the first condition results in `#f`, then `(>= temp 0)` automatically applies. We can therefore simplify the last condition to:

```
(and #t (< (+ temp heat) 100))
```

If the second condition also results in `#f`, then `(< (+ temp heat) 100)` also holds. We can therefore simplify to `(and #t #t)` and from there to `#t`.

With this insight, we can simplify the function and write `#t` as the condition.

(Try it out!) However, some people find it ugly: That's why it's also possible to use `#t` instead of this.

Add the special word **"else"** ("otherwise" in English). The "else" branch comes into play when all others have "failed." The result then looks like this:

```
(define heat-water
  (lambda (temp heat) (cond ;
    Initial
    temperature is below 0 °C, so ice is heated. ((< temp 0) (cond ; Heating remains below 0 °C.
      (< (+ temp
        heat)
        0) (+ temp heat))

    ; Heating gets stuck at 0 °C. ((< (+ temp heat) 80) 0)

    Heating raises the temperature above zero. (else (- (+ temp heat) 80))))

    Heating would increase the water temperature to over 100°C. ((>= (+ temp heat) 100) 100)

    Water starts out liquid and remains liquid when heated.
    (else
      (+ temp heat))))))
```

We have now applied the complete functionality of **cond** . Figure 2.2 summarizes it again.

You can use ``else`` from the outset if you are certain that all other cases are already covered in the previous branches. However, if in doubt, we always recommend using the mathematical approach to ensure the function is correct.

Speaking of correct: Is **"heat-water"** correct? Here's another test case:

```
(check-expect (heat-water -1 191) 100)
```

Of the 191°C, 1°C is needed to reach 0°C , then another 80°C to go above 0°C , leaving 110°C. But of course, it still won't go above 100°C. Unfortunately, the function sees it differently:

Check error:

The actual value 110 is not the expected value 100.

Why is that? If we mentally trace the branches, we see that initially the first branch of the outer **condition** takes effect because the condition `(< temp 0)` has the value `#t` . In the inner

In the languages taught, branching is represented using the special form ``cond``. A ``cond`` expression has the following form:

```
(cond
  (b1 a1)
  (b2 a2)
  ...
  (bný1 aný1)
  (else an))
```

Here, ``b1`` and ``ai`` are themselves expressions. The ``cond`` expression evaluates all conditions ``bi`` sequentially; as soon as a condition ``bk`` results in ``#t``, the ``cond`` expression is replaced by the corresponding ``ak``. If all conditions fail, ``ak`` is replaced by ``an``. The pairings (``bi`` and ``ai``) are called *branches* of the ``cond`` expression, and the branch containing ``else`` is called *the `else` branch*. The ``else`` branch can also be omitted – in which case, one of the conditions should always result in ``#t``. If, however, all ``bi`` results in ``#f``, DrRacket terminates the program and displays an error message.

Figure 2.2: Branching

Finally, since the first two conditions each result in `#f`, the else branch remains and adds the heat "blindly" to the initial temperature.

We therefore need to correct our program because this case has not yet been taken into account: The initial temperature is below 0°C, but the heating would raise the temperature above 100°C. The else branch of the inner **condition** of the outer branch is therefore triggered (`< temp 0`), which is incorrect in this case. The first two branches are responsible for final temperatures down to 0°C, so we need to insert the new branch immediately before the else branch. It looks like this:

```
Heating would raise the temperature to over 100°C ((>= (- (+ temp heat) 80)
100) 100)
```

The program is finally finished and correct!

But how could this error have gone unnoticed, at least initially by us? This is because the case distinction underlying the task is quite complex. Complex programs *always* carry a high risk of containing errors. With complex case distinctions, this means that almost always one case is missing from the program, at least on the first attempt.

We should take this opportunity to remember Mantra 7 on page 46 :

Mantra 7:

If your function seems complicated to you, it is likely that it contains errors: either because it should actually be simpler or because the task is complicated and therefore its solution is prone to errors.

Therefore, when faced with complicated case distinctions, test thoroughly and assume that you have forgotten branches.

Heating water more simply: From Mantra 7, one can draw an even better conclusion than simply carefully examining and testing. If the case distinctions are so complicated, then you should thoroughly investigate whether there isn't a simpler way.

Unfortunately, we neglected this principle when writing this book. Fortunately.

Nicolas Neuß from the University of Erlangen read an advance copy of this book. He was struck by He also noted the complexity of the function and made the following suggestion:

In my opinion, the whole thing becomes much prettier if you consider the physical aspects. This concerns. Therefore, when one writes conversion routines $E \rightarrow T$ and $T \rightarrow E$ (E stands for internal energy (i.e., heat) and T for temperature), so firstly, the problem becomes clear and secondly, the temperature increase can be easily calculated.

Let's try to follow Dr. Neuß's idea. We'll start with the $E \rightarrow T$ function he mentioned. In our terminology, this function could be called **heat->temperature**, with

The following short description and signature:

Calculate temperature from heat
 (: heat->temperature (real -> real))

As explained at the beginning of the section, below 0°C the following apply:

Temperature and heat are related. Between 0°C and 80°C , the temperature remains constant at 0°C .

Above 80°C , each degree increase also leads to an increase in temperature.

until a temperature of 100°C or a heat of 180°C is reached. From then on, the temperature remains constant. at 100°C , even as the temperature increases.

The test cases could therefore look like this:

```
(check-expect (heat->temperature -50) -50)
(check-expect (heat->temperature 0) 0)
(check-expect (heat->temperature 20) 0)
(check-expect (heat->temperature 80) 0)
(check-expect (heat->temperature 81) 1)
(check-expect (heat->temperature 180) 100)
(check-expect (heat->temperature 200) 100)
```

The input heat decomposes into four cases, as described above, which we refer to as four branches. write a **condition** :

```
(define heat->temperature (lambda
  (heat) (cond

    (... ...)

    (... ...) (... ...)

    (... ...))))
```

The conditions and expressions follow directly from the description:

```
(define heat->temperature (lambda
  (heat) (cond

    ((<= heat 0) heat)

    ((and (< 0 heat) (<=
      heat 80))

    0)

    ((and (< 80 heat) (<=
      heat 180))

    (- heat 80))

    (< 180 heat) 100))))
```

We need another function in the other direction, which we'll call `temperature->heat`. This function is simpler: temperatures up to 0°C correspond to the heat, temperatures above that require 80°C more heat. So there are only two cases and therefore two branches:

```
; Calculate heat from temperature (:
temperature->heat (real -> real))
```

```
(check-expect (temperature->heat -20) -20) (check-expect
  (temperature->heat -1) -1) (check-expect (temperature-
  >heat 1) 81) (check-expect (temperature->heat 100)
  180)
```

```
(define temperature->heat (lambda
  (temp) (cond

    (< temp 0) temp)
```

```
((and (> temp 0) (<=
      temp 100)) (+ temp
      80))))
```

Dr. Neuss suggested using these two functions to write the body of `heat-water`. We'll try that and write a new function, `heat-water-2`, before discarding the old one:

Calculate water temperature after heating, using ice and boiling (: `heat-water-2` (real real
-> real))

```
(define heat-water-2
  (lambda (temp heat)
    ...))
```

(We're using the same test cases as with `heat-water`.) How can we incorporate `heat->temperature` and `temperature->heat`? Well, we can calculate the corresponding heat for `temp`, add `heat`, and then convert the whole thing back into a temperature:

```
(define heat-water-2
  (lambda (temp heat)
    (heat->temperature (+
      (temperature->heat temp) heat))))
```

The function passes all test cases. Even better, the two functions `heat->temperature` and `temperature->heat` are much easier to understand than the behemoth `heat-water`.

For complicated branching – generally for all complicated programs – it is worthwhile to investigate whether there is an easier way.

2.6 DATA ANALYSIS AND TEMPLATES

A *data definition* first appeared in section 2.4 on page 57 :

A pet is one of the following:

- ; - Cat
- ; - Dog
- ; - Snake

This data definition is the result of a thought process, the so-called *data analysis*.

This term already appeared in section 1.11 on page 38, as the second step of the construction guide: From now on, we will always carry out this step because it is central to the systematic construction of programs.

The general procedure for data analysis looks like this:

Design Guide 6 (Data Analysis)

Search the problem statement for quantities relevant to the problem; candidates are always the nouns.
Write a data definition for each of these quantities, unless it is obvious from the context.

If there is no signature for the data definition yet, write a signature definition.
In addition, write down examples and add a comment for each one that describes the relationship between data and information.

This book contains several specialized construction guides on *how* to write the signature definition. The relationship between data and information is discussed in section 1.6 on page 31.

In this case, "pet" is a problem-relevant variable for which we have written a data definition. However, when discussing the heating of water, we did not explicitly state that the water temperature is a real number, because this is generally known.

There are a number of typical patterns in the formulation of data definitions that recur. One such pattern is evident in the definition of "pet," namely "one of the following," which indicates that "pet" is a case distinction.

Design guide 7 (Case distinction: Data analysis)

Try to find a formulation for the data definition that says "... is one of the following".
If possible, your data definition describes a *case distinction*. Then write a list of all cases, each case on a separate line:

```
; An X is one of the following:
; - Case 1
; - Case 2
; - ...
; - Case n
```

For the special case of enumeration, we have the following construction instructions:

Construction Guide 8 (Enumeration: Data Analysis)

If your data definition describes a case distinction and each of the cases consists of only a single value, it is an *enumeration*.

Write a signature definition for each list in the following form:

```
(define s (signature (enum ...)))
```

Make sure that the number of cases in the signature definition matches the number of cases in the data definition.

Data analysis is the most important method for implementing mantras 1 and 2:

Mantra 1

Make sure that everything you know is actually covered in the program.

Mantra 2

Read what it says there.

Data analysis initially focuses on the data itself, *not* on what is done with it. (That comes later.)

Furthermore, it always leads to elements in our program. For the `cute?` function, we first derived a signature definition from the data definition:

```
(define pet
  (signature (enum "cat" "dog" "snake")))
```

In lists, all examples are already included in the signature definition. Therefore, we do not need to write out separate examples.

Since the data definition for "pet" has three cases, the signature definition must also have three cases. A branch in the body arises directly from the signature definition, which results solely from the signature (`pet -> boolean`) or the signature definition of `pet` as an enumeration with three cases:

```
(define cute?
  (lambda (p)
    (cond
      (... ...) (... ...)

      (... ...))))
```

Da *cute?* expects an input for the signature *pet* and *pet* a list signature with *three*

In such cases, a branching with its own *three* branches *must* appear in the trunk, regardless of what the specific task. We emphasize the word "*three*" because many errors result from it.

It can happen that the number of branches in the *cond* of such a function does not correspond to the number of cases. corresponds.

Such an unfinished program, in which some elements emerge from the analysis of the data. Having a template without specific consideration of the concrete task is called a *template*. Certain types of data definitions correspond to specific templates.

Construction instructions 9 (template)

Once you have completed the framework, use the signature and the associated data definitions to select construction instructions with one or more templates. and transfer the elements of the templates into the body of the function.

The template is the next tool for implementing Mantra 1:

Mantra 1

Make sure that everything you know is actually covered in the program.

For each type of data definition, we write not only the construction guide for data analysis but also a template. Here is the template for case distinctions:

Construction instructions 10 (Case distinction: Template)

When writing a function that takes a case distinction as input, write a branching statement in the body as a template, with as many branches as there are cases in the case distinction, following this pattern:

```
(define f
  (lambda (a)
    (cond
      (... ...)
      ...
      (... ...))))
```

Next, write conditions in the branches that differentiate the individual cases from each other.

This construction manual may seem rather bureaucratic to you, and it is.

You might find it boring to blindly follow them, and that's certainly the case from time to time.

The construction instructions save you a lot of thinking, and you can free up your brain. then use it to solve the actual problem, once the template is available. It stands. So, at the latest when you are faced with a seemingly difficult task, first turn to... Read the assembly instructions before you worry about whether everything will work out in the end. What's at What remains in the end is often surprisingly simple.

2.7 EXCURSUS: BRANCHES IN MATHEMATICS

In mathematics, there are also branching statements. These are usually written with a large curly bracket. However, in mathematics, it is common to state all conditions. "to spell out," meaning their meaning is not dependent on the order of evaluation. We will do as we did with [heat-water](#). Following mathematical conventions, we will write the function as a single-letter function h with parameters t for [temp](#).

and h for [heat](#). The $\stackrel{\text{def}}{=}$ stands for "is defined as" and the mathematical equivalent for [define](#):

$$h(t, h) \stackrel{\text{def}}{=} \begin{cases} t + h & \text{if } t + h < 0 \\ 0 & \text{if } t + h \geq 0 \text{ and } t + h < 80 \\ -100 & \text{if } t < 0 \\ t + h - 80 & \text{if } t + h \geq 80 \\ t + h - 80 & \text{if } t + h \geq 80 \text{ and } t + h < 100 \\ 100 & \text{if } t + h \geq 100 \\ t + h & \text{if } t \geq 0 \text{ and } t + h < 100 \end{cases}$$

It's somewhat difficult to see in this notation: The condition $t < 0$ is the prerequisite for the first four cases.

2.8 BINARY BRANCHES

In some case distinctions, the last category is defined by the fact that a value does not belong to any of the other categories. In such cases, using an else branch in the [condition](#) is appropriate.

Let's assume you received a pet as a birthday present (as described in section [2.4](#) on page 57). The gift included an offer to exchange it for another pet if you didn't like it. Let's further assume that you only like cute pets.

You're interested in pets. So we'll write a function that swaps a pet for a cute one. It swaps it if it's not cute – and otherwise doesn't swap it.

The function could have the following short description and signature:

```
(: exchange-for-cute (pet -> pet))
```

The old pet is therefore the input for the function, and the pet after the exchange is the output. The following are suitable test cases:

```
(check-expect (exchange-for-cute "cat") "cat") (check-expect (exchange-for-cute "dog") "dog") (check-expect (exchange-for-cute "snake") "cat")
```

The scaffolding looks like this:

```
(define exchange-for-cute (lambda (p)

  ...))
```

Next up is the construction of the template. We could start with the definition of

Let's orient ourselves to `pet` and use a case distinction with three cases as a basis. However, it's quite obvious that the result of `exchange-for-cute` depends on whether `cute?` returns `#t` or `#f`. `#t` and `#f` here form an enumeration with only two cases. Therefore, we could construct a template with only two cases:

```
(define exchange-for-cute (lambda (p)

  (cond

    ((cute? p) ...) (else ...))))
```

The gaps are easy to fill:

```
(define exchange-for-cute (lambda (p)

  (cond

    ((cute? p) p) (else
    "cat"))))
```

This Boolean case distinction represents a frequently occurring special case: The second case is defined precisely by the fact that it is not the first; the corresponding `cond` expression therefore always has two points: one with a condition and one with `else`. Because such binary branching occurs so frequently, there is a somewhat shorter special form for the "binary `cond` with `else`" called `if`. This allows us to write `exchange-for-cute` more concisely:

A binary branch has the following form:

```
(if b
  k
  a)
```

Here, *b* is the condition and *k* and *a* are the two branches: the *consistent* *k* and the *alternative* *a*.

Depending on the outcome of the condition, the value of the branch is either the value of the consequence or the value of the alternative.

Figure 2.3: Binary branching

```
(define exchange-for-cute
  (lambda (p)
    (if (cute? p)
        p
        "Cat")))
```

The `if` statement saves a pair of parentheses and the `else` clause, but it can only handle binary branching. Figure 2.3 shows the general form of `if` expressions. There is a separate construction guide for Boolean conditional statements.

Construction instructions 11 (boolean case distinction: template)

If the result of a function depends on a Boolean quantity that the function can calculate using the inputs, use a binary number as a template in the body.

Branch:

```
(define f
  (lambda (e)
    (if ... ; here the boolean value is calculated)
        ...
        ...)))
```

Task 2.6

In mathematics, there is the "absolute value" of a number: For a negative number, it yields the corresponding positive number (so -5 becomes 5), all other numbers remain unchanged. Write a function that calculates the absolute value of a number!

2.9 SYNTACTIC SUGAR

In fact, **if** is more "primitive" than **cond**: every **cond** form can be translated into an equivalent **if** form, according to the following scheme:

```
(cond (t1 a1) (t2 a2) . . . (tnŷ1 anŷ1) (else an))
ŷ (if t1 a1 (if t2 a2 . . . (if ... (if tnŷ1 anŷ1 an). . .)))
```

The nested **if** form on the right side of the translation evaluates, just like the **cond** form, It sequentially evaluates all conditions until it returns a **#t**. The right-hand side of the **cond** branch is then That's precisely the consequence of the **if** statement. Only if all conditions have failed is the alternative... the last **if** expression, namely the one from the **else** branch.

Because with the help of this translation, every **cond** form can be replaced by nested **if** forms. Strictly speaking, **`cond`** is not necessary. Therefore, **`cond`** is a so-called *derived form*. However, since **`cond`** and other derived forms are still practical and convenient to use... Derived forms that sweeten the programmer's work are also called *syntactic sugar*.

Task 2.7

Rewrite the function **`cute?`** from section 2.4 so that it uses **`if`** instead of **`cond`**. ŷ

Task 2.8

Could it also be conceivable that **`if`** is syntactic sugar for **`cond`** – so can you provide a translation? How to specify an **if** expression within a condition expression? ŷ

"And" and "or" are also essentially syntactic sugar: It is always possible to use an "and-"

The expression needs to be translated into **if statements**. The following translation rules apply:

```
(and e) ŷ e
(and e1 e2 . . .) ŷ (if e1 (and e2 . . .) #f)
```

An AND expression with multiple operands is thus translated step by step into a cascade of **if** expressions:

```
(and abc)
ŷ (if a (and bc) #f)
ŷ (if a (if b (and c) #f) #f)
ŷ (if a (if bc #f) #f)
```

Similarly, or expressions can always be translated into if expressions:

```
(or e) ÿ e (or e1
e2 . . .) ÿ (if e1 #t (or e2 . . .))
```

Example:

```
(or abc)
ÿ (if a #t (or bc))
ÿ (if a #t (if b #t (or c))) ÿ (if a #t (if b #tc))
```

Task 2.9

Regarding the syntactic sugar for ``and`` and ``or``, we could also define what ``(and)`` and ``(or)`` are translated into (i.e., without operands). We could then omit the rules for ``(and e)`` and ``(or e)``. What would be the correct translation in each case? You should work through some examples to find this out. (Warning: The correct translation might surprise you.) Translate ``(and abc)`` and ``(or abc)`` using your new rules as if the rules for ``(and e)`` and ``(or e)`` didn't exist.

ÿ

2.10 INTERCEPTING NONSENSE DATA

Let's return to the ``heat-water`` function from section 2.5: It already has many branches, but still doesn't cover everything, for example, initial temperatures above 100°C:

```
(heat-water 150 0)
ÿ 100
```

This is because there's no meaningful result if the initial temperature is above 100°C. The function merrily calculates one anyway. It would be helpful if we could report when the function has received nonsensical input.

To do this, we first expand the branching, right at the beginning:

```
(define heat-water
  (lambda (temp heat)
    (cond
      ; Initial temperature is above 100°C, therefore inadmissible (> temp 100) ...
      (> temp 100)
      ...)))
```

But what happens in case of an error? There's a built-in function called ``violation`` that accepts an error message as a string and, when called, terminates the program and prints the error message. The complete function looks like this:

```
(define heat-water
  (lambda (temp heat)
    (cond ;
      (Initial temperature is above 100°C, therefore inadmissible ((> temp 100)
        (violation "initial
          temperature above 100°C"))
      (The initial temperature is below 0 °C, so ice is being heated. ((< temp 0) (cond
        ; Heating remains below 0 °C. ((< (+
          temp heat) 0) (+ temp heat))
        ; Heating gets stuck at 0 °C. ((< (+ temp heat) 80) 0)
        ; The heating would raise the temperature to over 100°C ((>= (- (+ temp heat) 80)
          100) 100)
        (Heating raises the temperature above zero. (else (- (+ temp heat) 80))))

      (Heating would increase the water temperature to over 100°C. ((>= (+ temp heat)
        100) 100)
      (Water starts out liquid and remains liquid when heated.
        (else
          (+ temp heat))))))
```

Of course, we should also test for errors – this isn't possible with ``check-expect``, which expects a test expression to return a valid value. For error cases, there's ``check-error``, which generates test cases that pass if the evaluation returns an error.

```
(check-error (heat-water 150 0)) ; Initial temperature above 100°C
```

The simpler function ``heat-water-2`` also passes this test, without using any ``violation``. Why is that? Here is the error message:

```
cond: all conditions resulted in #f
```

Furthermore, DrRacket identifies the `temperature->heat` function as the culprit. Its conditions explicitly exclude temperatures above 100°C. Here too, the simpler version is clearly superior.

Task 2.10

Try the expressions `(heat-water 0 0)` and `(heat-water-2 0 0)`. They produce different results. Why? Which version is correct?

Ÿ

TASKS

Task 2.11

Section 4 of the Road Traffic Act links the following sanctions to the "Flensburg" points, which are awarded for violations of traffic regulations:

0 points No sanctions

1 to 3 points: Preliminary note; 4

to 5 points: Reprimand; 6 to 7

points: Warning

8 points deducted

Write a function that calculates the consequence of a given score!

Task 2.12

Write a function `card-type` that accepts credit card transaction data and returns a corresponding category as a string. Use the construction guidelines: write the short description, perform data analysis, and write the signature.

Then create the test cases and the framework. Next, complete the function body and ensure that the tests run successfully.

Revenue < 15,000 Ÿ White

15,000 Ÿ revenue < 50,000 =Ÿ Gold 50,000 Ÿ revenue Ÿ 150,000

=Ÿ Platinum

150,000 < revenue

Ÿ Black

Task 2.13

1. Write a function `min-2` that takes two numbers as arguments and returns the smaller of the two. Also write a function `min-3` that takes three numbers as arguments and returns the smallest of the three. Use the construction guide: explicitly write a short description and signature, then create the skeleton and test cases. Finally, complete the function body and verify that the tests run successfully.
2. Write a similar function `max-2` and `max-3`.
3. Write a function `min-3` that outputs the smallest of three given numbers.
4. Write a function `min-3?` that checks if the first of three given numbers is the minimum is.
5. Write a function `valid-value?` that checks whether the first of three given numbers lies between the other two. "Between" means that the first number could also be the same as the second or the third.
6. `clamp`, which is defined as follows:

$$\text{clamp}(x, \text{min}, \text{max}) = \begin{cases} \text{min } x & \text{if } x < \text{min} \\ x & \text{if } \text{min} \leq x \leq \text{max} \\ \text{max } x & \text{if } x > \text{max} \end{cases}$$

Task 2.14

In football, a player's jersey number often provides clues about her position.

We make the following assumptions:

- A *goalkeeper* wears the number 1 on her back.
- A *defender* wears the number 2, 3, 4, or 5.
- A *midfielder* wears the number 6, 7, 8, or 10.
- A *striker* wears the number 9 or 11.
- A *substitute* wears a number between 12 and 19.
- All other numbers are invalid.

First, write a data definition for the positions and then a suitable signature definition named `position`.

Now write a function with the following signature:

```
(: number->position (natural -> position))
```

The function should calculate the corresponding position for a given jersey number.

When writing the function, use the construction guidelines for functions and conditional statements.

Test the function ``number->position`` with at least six test cases to ensure all cases are covered.

Task 2.15

Write a program that automatically calculates fines and warnings.

1. Program a function called "parking for too long" to assess whether someone has parked for too long in a paid parking space. This function receives a time period and returns the corresponding fine.

These warnings are defined as follows:

Exceeding the maximum parking time by up to 30 minutes e5
up to one hour e10
up to two hours e15
up to three hours e20
longer than three hours e25

2. Running a red light can result in a higher fine, penalty points, and driving bans, depending on the level of danger. Write three functions: one for the fine (red-light-fine), one for penalty points (red-light-points), and one for the driving ban (red-light-ban) that indicates whether a driving ban will be issued. Assign the functions the length of time the light had been red and whether there was any danger or property damage.

The fines are defined as follows: •

Running a red light within the first second: €50 and 3 points. • Running a red light within the first second that endangers others or causes property damage: €125, 4 points and a 1-month driving ban.

• Running a red light after the first second: €125, 4 points and a 1-month driving ban. • Running a red light after the first second that results in endangerment or property damage: €200, 4 points and a 1-month driving ban.

3 COMPOSITE DATA

Many pieces of information represented in programs have several components: • A feast consists of an appetizer, a main course, and dessert. • A time consists of hours and minutes.

- A door consists of a door leaf and a door handle.

Several things are *combined to form one*. Another way of looking at it is that a single thing has *several properties* : • A felt-tip pen has thickness and color.

- A cat has an age and a gender.
- A loudspeaker has a minimum and a maximum frequency.

To represent such information, we introduce a new type of data in this chapter: *composite data*.

3.1 Configuring the Computer

`composite-data/computer.rkt`

code

Many computer stores allow you to select certain components of a new computer, for example, processor, hard drive and size of RAM:



In other words, a computer *consists of*:

- Processor
- RAM
- Hard drive

Of course, a computer also consists of other parts, but these are (at least in this example) always the same or irrelevant. Therefore, an order only needs to include these three components. We assume that for the processor, only the name ("Athlon", "Xeon", "Cell", etc.) matters, for the RAM, only the size in gigabytes, and for the hard drive, only the size in gigabytes.

We can turn this into an official data definition:

A computer consists of:
; - Processor
; - Main memory capacity in GB
; - Hard drive capacity in GB

The important phrase here is "consists of," which indicates composite data. The data resulting from such a composite data definition can be presented as a table:

	Field	Component
computer	Processor	"Cell"
	RAM	8
	Hard drive	250

This table therefore represents a computer with a Cell processor, 8 gigabytes of RAM, and a 250-gigabyte hard drive. It thus has several components and is therefore a composite. The computer company will ship many computers according to this scheme, each order specifying a processor, RAM, and hard drive. Such information, all following the same scheme, can be sorted by "type," where the type defines the kind of information and the components it comprises. In the table above, the *field* is the generic term for a component that all computers have. The *component* is the specific part of an individual computer.

Composite data forms the basis of the teaching languages through *so-called records*. Each record belongs to a specific *record type*, which defines what kind of information is represented and which fields the records of that record type contain.

The record type for computers provides fixed fields ("Processor", "RAM", "Hard disk"). A single record of this type consists of components, one per field. (In this case, "Cell", 8 and 250.)

Record types must be explicitly defined in a program using a *special record definition* that begins with ``define-record`` . Here is the record definition for defining data for computers:

```
(define-record computer make-
  computer (computer-
    processor string) (computer-ram (computer-
      hard-drive natural))          (natural))
```

We will explain this somewhat complicated form step by step. Because it defines several functions related to records, it is called ``define-record``. Following ``define-record`` is the name of the signature for the values of the record type, ``computer``. From now on, we will refer to this signature as the *record signature*.

Next in the record definition is ``make-computer``, the name of a function responsible for *constructing* a computer record. Analogous to assembling a computer with a Cell processor, 8 gigabytes of RAM, and a 250-gigabyte hard drive, the corresponding record is created using the ``make-computer`` function as follows:

```
(make-computer "Cell" 8 250) ÿ
#<record:computer "Cell" 8 250>
```

Make-computer has the following signature:

```
(: make-computer (string natural natural -> computer))
```

The three inputs are, in order, processor, RAM, and hard drive. ``Make-computer`` creates a value of the type ``computer`` from the computer records. Because ``Make-computer`` "constructs" a ``Computer=Record``, the function is also called a *constructor*.

With the spelling

```
#<record:... ...>
```

The record type and its components become visible in the REPL.

Here are some examples of computer records, with comments that establish the relationship between data and information (see section 1.6 on page 31) :

```
; Cell, 4 GB RAM, 1000 GB hard drive (define gamer (make-
  computer "Cell" 4 1000))
```

```
gamer
```

```
ÿ #<record:computer "Cell" 4 1000>
```

```
; Xeon, 2 GB RAM, 500 GB hard drive (define workstation
  (make-computer "Xeon" 2 500))
```

workstation

```
γ #<record:computer "Xeon" 2 500>
```

Conversely to building a computer, some hobbyists remove individual components, for example, to install them in another computer. The functions ``computer-processor``, ``computer-RAM``, and ``computer-hard-drive``, defined by the Record Definition, are responsible for this removal process.

```
(computer-processor gamer)
```

```
γ "Cell"
```

```
(computer-ram gamer) γ 4
```

```
(computer-hard-drive gamer)
```

```
γ 1000
```

These three functions are called *selectors*. They have the following signatures:

```
(: computer-processor (computer -> string)) (: computer-ram (computer  
-> natural)) (: computer-hard-drive (computer -> natural))
```

Strictly speaking, these signatures are redundant: The record definition already states that the fields have the signatures `string`, `natural`, and `natural`. Each selector takes `computer` as input and the respective field signature as output. The signature declaration for the constructor is also redundant.

Task 3.1

What is the relationship between the record definition and the constructor signature? γ

Nevertheless, it is helpful, at least initially, to understand how constructors and selectors work by looking at their signatures. Therefore, we will write them down in this chapter, but not again thereafter.

Using the constructor and selectors, we can define more advanced functions. To begin with, this could be a function that calculates the total memory of a computer, i.e., main memory and hard disk storage combined. Such a function would have a short description and signature as follows:

```
; Calculate total memory (: total-memory
(computer -> natural))
```

Here are our expectations for `total-memory`, formulated as test cases:

```
(check-expect (total-memory workstation) 502) (check-expect (total-
memory gamer) 1004)
```

The framework is as follows:

```
(define total-memory (lambda
  (c)...))
```

To perform a calculation from the record, ``total-memory`` (and virtually every other function) must consider its components. Therefore, it makes sense to populate the template with the calls to the selectors.

```
(define total-memory (lambda
  (c)
    ... (computer processor c) ... ... (computer
    ram c) ... ... (computer hard drive
    c) ...))
```

Now that the template is finished, we can turn our attention to the content of the task: The

The processor has nothing to do with the amount of memory, so we can delete the corresponding selector call:

```
(define total-memory (lambda
  (c) ... (computer-
    ram c) ... ... (computer-hard-drive
    c) ...))
```

The total storage capacity is the sum of the two components:

```
(define total-memory (lambda
  (c)
    (+ (computer-ram c)
      (computer-hard-drive c))))
```

`Total memory` is an example of a function that accepts a record. Conversely, there are also functions that produce records. Let's assume our computer dealer offers several standard models in addition to individual configurations of processor, RAM, and hard drive.

Let's say there's a budget model, a model for professionals (whatever "professional" might mean), and a model for computer gamers. Depending on which model the customer chooses, the corresponding configuration must be assembled. There are three fixed options for the standard configuration; this is simply a list.

For the purposes of this list, we will first proceed – according to construction instructions 8 on page 75 – a data definition and a signature definition:

One model is one of the following:

```
; - Budget model
; - Professional model
; - Gamer model (define
model (signature
```

```
(enum "cheap" "professional" "gamer"))))
```

A function that creates the appropriate computer for a standard configuration could therefore have the following short description and signature:

```
; Assemble a standard computer (: standard-
computer (model -> computer))
```

The test cases should cover all three standard configurations:

```
(check-expect (standard computer "cheap") (make computer
      "Sempron" 2 500))
(check-expect (standard computer "professional") (make computer
      "Xeon" 4 1000))
(check-expect (standard computer "gamer") (make computer
      "Quad" 4 750))
```

Here is the framework:

```
(define standard computer (lambda
  (k)...))
```

For the template, we again follow construction instructions 8 on page 75. Since the argument from `standard-computer` is a case distinction – a list with *three* alternatives – we can use the corresponding template – a branching with *three* branches:

```
(define standard computer (lambda (k)
  (cond

    (... ...)

    (... ...) (... ...))))
```

When testing the branches, we need to compare `k` with the elements of the enumeration. Since these are strings, we use ``string=?`` for this purpose.

```
(define standard computer (lambda (k)

  (cond

    ((string=? k "cheap") ...) ((string=? k
    "professional") ...) ((string=? k "gamer") ...))))
```

In each branch, we now need to ensure that the corresponding computer is manufactured.

The constructor ``make-computer`` is responsible for creating computer records . Accordingly, we must place a call to ``make-computer`` in each branch , each call taking three arguments:

```
(define standard-computer
  (lambda (k)
    (cond

      ((string=? k "cheap") (make-
        computer ... ..))

      ((string=? k "professional")
        (make-computer ... ..)) ((string=? k
        "gamer") (make-
        computer ... ..))))
```

Now we need to provide the arguments for the ``make-computer`` commands . For each command, these are the processor, the size of the main memory, and the size of the hard drive. We can find this information, for example, in the test cases. The result is as follows:

```
(define standard-computer
  (lambda (k)
```

```
(cond
  ((string=? k "cheap") (make-
    computer "Sempron" 2 500))
  ((string=? k "professional") (make-computer
    "Xeon" 4 1000)) ((string=? k "gamer")

    (make computer "Quad" 4 750))))
```

Complete!

Task 3.2

Write a function that classifies a computer as "fat", "average", or "tired". Define the criteria yourself. ÿ

3.2 CONSTRUCTING COMPOSITE DATA YOURSELF

[composite-data/wallclock-time.rkt](#)

code

For another example, we refer back to the following sentence from the introduction, which we can already interpret as a data definition:

A time consists of an hour and a minute.

To develop the appropriate record definition, we need to come up with a name for the record type. Then we can already write a basic outline:

```
(define-record wallclock-time
  make-wallclock-time
  ...)
```

Next, we need to determine *how many* components the records should have. In this case ("hour and minute"), there are two. We can therefore extend the record definition framework by two fields:

```
(define-record wallclock-time
  make-wallclock-time
  (... ...)
  (... ...))
```

Paying attention to the number of components helps us to implement Mantra 1 on page 45 :

Mantra 1

Make sure that everything you know is actually covered in the program.

Next come the names of the selectors. We follow a convention of starting all selector names with "wallclock-time-". We've also applied a convention to the constructor name, which is derived from "make-" and the name of the record type. So:

```
(define-record wallclock-time
  make-wallclock-time
  (wallclock-time-hour ...)
  (wallclock-time-minute ...))
```

The fact that we write the selectors one below the other is merely for the sake of clarity; it is also a convention.

The signatures for `wallclock-time-hour` and `wallclock-time-minute` are still missing .

Not only that: We first need data definitions for both fields.

```
An hour is a whole number between 0 and 23.
; A minute is a whole number between 0 and 59.
```

Since we are dealing with integers starting from 0, we could use `natural` , but it is more precise to use `integer-from-to` from section 2.2 on page 55 and introduce our own signature definitions:

```
; An hour is an integer between 0 and 23. (define hour (signature (integer-from-
to 0 23)))
; A minute is an integer between 0 and 59. (define minute (signature (integer-
from-to 0 59)))
```

We can now use these signatures in the record definition for `wallclock-time` :

```
(define-record wallclock-time
  make-wallclock-time
  (wallclock-time-hour hour)
  (wallclock-time-minute minute))
```

We write out the signatures of the defined functions. These result directly from the record definition:

A record definition has the following general form:

```
(define-record t
  c
  (sel 1 sig1)
  . . .
  (sel n sign))
```

This form defines a record type with n fields. Here, t , c , $\text{sel } 1 \dots \text{sel } n$ are variables for which all `define-record` definitions are created:

- t is the name of the record signature.
- c is the name of the constructor created by `define-record`. The constructor has the following Signature:

```
(: c (sig1 . . . sign -> t))
```

- $\text{Sel } \text{sel } 1, \dots, \text{sel } n$ These are the names of the selectors for the fields of the record type. The selector has the following signature:

```
(: sel i (t -> sigi))
```

Figure 3.1: `define-record` (simple)

```
(: make-wallclock-time (hour minute -> wallclock-time))
(: wallclock-time-hour (wallclock-time -> hour))
(: wallclock-time-minute (wallclock-time -> minute))
```

The constructor accepts one argument for each field – the signatures of the Fields before the arrow. The constructor always outputs a record, which contains the name. of the record type.

With selectors, it's the other way around: the record signature is always at the front (they accept (yes, each a record) and after the arrow is the signature of the respective field.

Figure 3.1 summarizes the form of record definitions.

Here are three examples of times as data, with comments explaining what they represent. Describe information:

```
(define wt1 (make-wallclock-time 11 55)) ; five to twelve
(define wt2 (make-wallclock-time 0 0)) ; midnight
(define wt3 (make-wallclock-time 1 1)) ; 1 o'clock 1
```

First, we calculate the number of minutes since midnight for a given time. Here are short-description, signature, test cases and framework:

Calculate minutes since midnight (: minutes-since-midnight

(wallclock-time -> natural))

(check-expect (minutes-since-midnight wt1) (+ (* 11 60) 55)) (check-expect (minutes-since-midnight wt2) 0) (check-expect (minutes-since-midnight wt3) 61)

(define minutes-since-midnight (lambda (wt)

...))

`Minutes-since-midnight` is intended to be a function that accepts times as input, i.e., composite data. A function that performs calculations on composite data usually needs to use its components, which it can access via selectors. As a next step, we'll add calls to both selectors:

(define minutes-since-midnight (lambda (wt)

... (wallclock-time-hour wt) ...

... (wallclock-time-minute wt) ...))

Now we add some knowledge about time and thus complete the hull:

(define minutes-since-midnight (lambda (wt)

(+ (* 60 (wallclock-time-hour wt))

(wallclock-time-minute wt))))

Task 3.3

Write a function that returns whether a given time refers to the morning or afternoon (i.e., before or after 12 noon).

We can also reverse the minutes-since-midnight function :

; Calculate the time from minutes since midnight (: minutes-since-midnight->wallclock-time (natural -> wallclock-time))

The arrow in minutes-since-midnight->wallclock-time is part of the name and represents the conversion of one quantity into another.

The test cases are reversed compared to `minutes-since-midnight` :

```
(check-expect (minutes-since-midnight->wallclock-time
                (+ (* 11 60) 55)) wt1)
```

```
(check-expect (minutes-since-midnight->wallclock-time 0) wt2) (check-expect
```

```
(minutes-since-midnight->wallclock-time 61) wt3)
```

Here is the framework:

```
(define minutes-since-midnight->wallclock-time (lambda (msm)

  ...))
```

This function produces a time – therefore it must call the constructor for ``wallclock-time`` . This results in the following template:

```
(define minutes-since-midnight->wallclock-time (lambda (msm)

  (make-wallclock-time ... ...)))
```

To complete the template for the hull, we first need to calculate the hour from the minutes since midnight (`msm`) . For this, we need a function that performs integer division. Unfortunately, the built-in function ``/`` doesn't do this.

```
(/ 61 60)
⇒ 1.0166666666666667
```

But the `quotient` function helps us further:

```
(quotient 61 60)
⇒ 1
```

We can use this in the template:

```
(define minutes-since-midnight->wallclock-time (lambda (msm)

  (make-wallclock-time (quotient msm 60) ...)))
```

We're still missing the minute – for that we need the remainder of the division. The built-in function ``remainder`` calculates that:

```
(remainder 67 60)
ȳ 7
(remainder 125 60)
ȳ 5
```

This allows us to complete the hull:

```
(define minutes-since-midnight->wallclock-time (lambda (msm)

  (make-wallclock-time (quotient msm 60) (remainder msm 60))))
```

Task 3.4

Write a function `make-wallclock-time-12h` that constructs a time from a time given in 12-hour format. For example:

```
(make-wallclock-time-12h 6 30 "AM")
ȳ #<record:wallclock-time 6 30>
(make-wallclock-time-12h 6 30 "PM")
ȳ #<record:wallclock-time 18 30>
(make-wallclock-time-12h 12 0 "PM")
ȳ #<record:wallclock-time 12 0>
(make-wallclock-time-12h 12 00 "AM")
ȳ #<record:wallclock-time 0 0>
(make-wallclock-time-12h 12 30 "PM")
ȳ #<record:wallclock-time 12 30>
```

(There is no clear assignment for the two cases 12:00 AM and 12:00 PM; we have defined this arbitrarily.) ȳ

Task 3.5

Does `minutes-since-midnight->wallclock-time` work for all numbers as input? ȳ

Task 3.6

Write a function `wallclock-time-add-minutes` that adds a specific number of minutes to a given time. Use the existing functions `minutes-since-midnight` and `minutes-since-midnight->wallclock-time`. You can assume that the result will also fit within the 24 hours of a day.

3.3 DESIGN GUIDELINES FOR COMPOSITE DATA

This section summarizes the findings from the composite data examples in the form of construction guidelines. We begin with data analysis and the associated record definition.

Construction Guide 12 (Composite Data: Data Analysis)

You can recognize composite data by phrases like "an X consists of ...", "an X is characterized by ...", or "an X has ...". Sometimes the phrase is...

The implementation is slightly different. The resulting data definition is a comment in the program in the following form:

An X has / consists of / is characterized by:

; - Component / Property 1

; - Component / Property 2

; ...

; - component / property n

This is followed by a corresponding record definition. For this, think of names for the record type T and for the fields, f1 ... fn. Each field has a signature sig_i :

```
(define-record T
```

```
  make-T
```

```
  (T-f1 sig1)
```

```
  ...
```

```
  (T-fn sign)
```

The name of the record type T is the record signature, `make-T` is the constructor, and T-fi are the selectors. The fact that the constructor name begins with `make-` and that the selector names are composed of the type name and the fields is purely a convention.

You should only deviate from it for very good reasons.

This includes the signatures for the constructor and the selectors:

```
(: make-T (sig1 ... sign -> T))
```

```
(: T-f1 (T -> sig1))
```

```
...
```

```
(: T-fn (T -> sign))
```

If you have enough practice using constructors and selectors, you can also omit the signatures (which are redundant): The relevant signatures for the fields are already in the record definition.

Once you have completed the data analysis and record definition for composite data, you should determine from the function signature whether the composite data will be used as input or output. Depending on this, you can select the appropriate template from the following two construction guides.

Construction instructions 13 (Composite data as input: template)

If your function accepts composite data as input (this is clear from the signature), proceed as follows after writing the skeleton: 1. For each component, write (sel r)

into the template, where sel is the selector of the component and r is the name of the record parameter, for example:

(wallclock-time-hour wt)

2. Complete the template by constructing an expression in which the selector applications occur.
3. It is possible that not all selector applications will be used in the hull: In this case, delete the selector application again.

With a little practice, you can omit unnecessary selector applications from the outset. However, occasionally a missing one indicates an error; therefore, it's often advisable to list it initially.

Functions that output composite data must construct a corresponding record and therefore call the constructor. Here is a template for that:

Construction instructions 14 (Composite data as output: Template)

If your function outputs composite data, write a call to the appropriate record constructor in the body, starting with an ellipsis for each field of the record.

Records, for example:

(make-wallclock-time)

3.4 INPUT AND OUTPUT OF COMPOSITE DATA

```
composite-data/dillo.rkt
```

```
code
```

In this section, we combine input and output of composite data into a single function.

The example involves armadillos in Texas: They particularly like to cross the highways. and are unfortunately often run over – consequently, there are many armadillos on the roadside. see. Furthermore, friendly drivers occasionally feed the armadillos. With these two We want to examine the following aspects: What happens when an armadillo is run over? What happens when an armadillo is fed? Accordingly, we are interested to know if an armadillo at Life is and what its weight is. We can translate this directly into a data definition according to construction instructions 12 on page 100 :

An armadillo has the following characteristics:

; - Weight (in g)

; - alive or dead

Again, this involves composite data, as is evident from the wording "has". Here, we will limit ourselves to the two properties that are relevant to the task. From the data definition, we can directly create a suitable record definition:

```
(define-record dillo
  make-dillo
  (dillo-weight natural)
  (dillo-alive? boolean))
```

(«Dillo» is short for «Armadillo», the English word adopted from Spanish for Armadillo.)

For the field "alive?" we could choose different representations: A list would be possible; we have opted for a Boolean value that answers the question "Is this alive?" Armadillo?» answered. Here are the signatures for the record functions:

```
(: make-dillo (natural boolean -> dillo))
(: dillo-weight (dillo -> natural))
(: dillo-alive? (dillo -> boolean))
```

Here are some examples with data and information:

```
(define dillo1 (make-dillo 55000 #t)) ; 55 kg, alive
```

```
(define dillo2 (make-dillo 58000 #f)) ; 58 kg, dead
(define dillo3 (make-dillo 60000 #t)) ; 60 kg, alive
(define dillo4 (make-dillo 63000 #f)) ; 63 kg, dead
```

Let's start by feeding the armadillos. The standard food portion is 500g, and that Armadillos gain weight by the corresponding amount through feeding. Here is a brief description and signature:

```
Feed the armadillo 500g of food.
(: feed-dillo (dillo -> dillo))
```

Here is the first, obvious test case:

```
(check-expect (feed-dillo dillo1) (make-dillo 55500 #t))
```

With `feed-dillo`, what it does with dead armadillos is relevant: Dead armadillos don't eat, Accordingly, they do not gain weight when offered food:

```
(check-expect (feed-dillo dillo2) dillo2)
```

Here is the framework of the function:

```
(define feed-dillo
  (lambda (dillo)
    ...))
```

We also use `dillo` for the parameter name, not to be confused with the signature, which is also called `dillo`. The **lambda** ensures that `dillo` positions itself within its hull on refers to the parameter, not to the signature located further out.

Task 3.7

To clarify which `dillo` belongs to which **lambda** or definition, you can press the [Syntax Check](#) button in DrRacket and then move the mouse pointer over the different occurrences of `dillo`.

More on this topic can be found in section 9.6 on page 274.

`Feed-dillo` accepts composite data as both input and output. Accordingly, The templates are used for both situations.

First, the template for composite data is used as input from the design instructions. Next up is item 13 on page 101. We'll write down the selector calls:

```
(define feed-dillo
  (lambda (dillo)
    ... (dillo-weight dillo) ...
    ... (dillo-alive? dillo) ...))
```

In addition, there is the template for composite data as output from construction instructions **14** on page **101**, i.e., the call to the constructor:

```
(define feed-dillo
  (lambda (dillo) (make-
    dillo... ...)
    ... (dillo-weight dillo) ... ... (dillo-alive?
    dillo) ...))
```

The second test case shows that, with regard to **feed-dillo**, the armadillos fall into two different groups: **feed-dillo** behaves differently for living armadillos than for dead ones: a case distinction. Accordingly, we need a branch in the trunk, based on the value of **(dillo-alive? dillo)**, which is already in the template. Since **dillo-alive?** returns a Boolean value, it is a Boolean case distinction. Therefore, we can apply construction guide **11** on page **79** and use a binary branch:

```
(define feed-dillo
  (lambda (dillo)
    (if (dillo-alive? dillo)
        ...
        ...)
    (make-dillo ... ...) ... (dillo-
    weight dillo) ...))
```

Now we need to add the two branches. The simplest is "armadillo dead," because then the same armadillo that went in comes out of the function. So we insert **dillo** as an alternative branching option:

```
(define feed-dillo
  (lambda (dillo)
    (if (dillo-alive? dillo)
        ...
        dillo)
    (make-dillo ... ...)
    ... (dillo-weight dillo) ...))
```

In the first branch, we finally need to calculate a new armadillo value that takes the increase into account. This consumes the constructor call and the second selector call from the template:

```
(define feed-dillo (lambda
  (dillo)
    (if (dillo-alive? dillo)
        (make-dillo ... ...) dillo)))
```

When calling the constructor `make-dillo`, we must specify the weight of the freshly fed armadillo and whether it is still alive. The weight increases by the amount of the feed.

Weight of the food. Furthermore, the armadillo is still alive because the constructor call is in the branch where this is the case:

```
(define feed-dillo
  (lambda (dillo)
    (if (dillo-alive? dillo)
        (make-dillo (+ (dillo-weight dillo) 500)
                    #t)
        dillo)))
```

Now we come to the unpleasant part: being run over, which turns a living armadillo into a dead one. Here's a brief description and signature:

```
; Armadillo run over
(: run-over-dillo (dillo -> dillo))
```

From the example `dillo1` we can create the first test case:

```
(check-expect (run-over-dillo dillo1) (make-dillo 55000 #f))
```

But we should also consider what `run-over dill` does to dead armadillos. They remain dead even after being run over.

```
(check-expect (run-over-dillo dillo2) dillo2)
```

Here is the framework of the function:

```
(define run-over-dillo (lambda
  (dillo)
    ...))
```

[Run-over-dillo](#) takes composite data as both input and output. Accordingly, the templates for both situations are used again. First, the

Template for composite data as input; we write down the calls to the selectors:

```
(define run-over-dillo (lambda
  (dillo)
    ... (dillo-weight dillo) ... ... (dillo-alive?
    dillo) ...))
```

In addition, there is the template for composite data as output, i.e., the call to the constructor:

```
(define run-over-dillo
  (lambda (dillo) (make-
    dillo ... ...) ... (dillo-weight
    dillo) ... ... (dillo-alive? dillo) ...))
```

Since running over it does not change the weight, the expression for the weight takes the weight of the input armadillo from the template:

```
(define run-over-dillo
  (lambda (dillo) (make-
    dillo (dillo-weight dillo) ...) ... (dillo-alive? dillo) ...))
```

The armadillo is definitely dead after being run over. Since it doesn't matter whether the armadillo was alive beforehand or not, we can discard the selector call `(dillo-alive? dillo)` :

```
(define run-over-dillo
  (lambda (dillo)
    (make-dillo (dillo-weight dillo) #f)))
```

Complete!

3.5 ALTERNATIVES IN THE CONSTRUCTION INSTRUCTIONS

Perhaps you noticed in the series on templates for [feed-dillo](#) that we arranged the templates for constructing composite data and the template for binary data.

We arranged the case distinctions rather arbitrarily by placing the branching "outside" the constructor call.

We could just as easily have applied the construction instructions the other way around, into which we would have placed the branch "inside" in the constructor call of `make-dillo` :

```
(define feed-dillo
  (lambda (dillo)
    (make-dillo (if (dillo-alive? dillo)
                    ...
                    ...))
    ...
    (dillo-alive? dillo) ...
    ... (dillo-weight dillo) ...))
```

In this procedure, we first fill in the ellipses for the two possible weights:

```
(define feed-dillo (lambda
  (dillo)
    (make-dillo (if (dillo-alive? dillo)
                    (+ (dillo-weight dillo) 500) (dillo-weight
dillo))
    ...)
    ... (dillo-alive? dillo) ...))
```

Regarding the second argument of `make-dillo` , namely whether the armadillo is alive or dead, the value there is the same as before, i.e., corresponding to the result of `dillo-alive?`:

```
(define feed-dillo
  (lambda (dillo) (make-
    dillo (if (dillo-alive? dillo)
              (+ (dillo-weight dillo) 500) (dillo-weight
dillo))
    (dillo-alive? dillo))))
```

This version is just as correct as the first, and neither is obviously "better" than the other.¹ Thus, when combining construction instructions, it is often possible to create several different versions.

¹ The first version, in the case of "dead armadillo," avoids generating a new armadillo value. The second version, however, is shorter. In practice, the difference is insignificant.

There are many paths to take. Most of them all work, but they occasionally differ in length and elegance.

Another alternative in the design of `feed-dillo` would be possible at this point. been:

```
(define feed-dillo
  (lambda (dillo)
    (if (dillo-alive? dillo) (make-
      dillo ... ...)
      dillo)))
```

Here we rather hastily wrote "`dillo`" in the alternative of the `if`-expression because it seemed "obvious" to us that the condition of a dead armadillo after feeding is the same as before. It would have been more consistent to first apply the templates completely:

```
(define feed-dillo (lambda
  (dillo)
    (if (dillo-alive? dillo)
      (make-dillo... ...) (make-
        dillo... ...))))
```

Then we would also have answered the questions for the second branch ("armadillo dead"), which The armadillo's weight and whether it is still alive:

```
(define feed-dillo (lambda
  (dillo)
    (if (dillo-alive? dillo)
      (make-dillo (+ (dillo-weight dillo) 500) #t) (make-dillo (dillo-
        weight dillo) #f))))
```

This version is also correct. We could further refine this version by exploiting the observation that the value of the ``dillo-alive?`` field in the result corresponds to the result of ``(dillo-alive? dillo)`` and inserting this into the second ``make-dillo`` call :

```
(define feed-dillo
  (lambda (dillo)
    (if (dillo-alive? dillo)
      (make-dillo (+ (dillo-weight dillo) 500) #t) (make-dillo (dillo-
        weight dillo) (dillo-alive? dillo)))))
```

The program has become longer, but it also offers some insight if we take a closer look at the second `make-dillo` call. This call creates a `dillo` record where each field is populated from the corresponding field of `dillo`. It is therefore equivalent to `dillo`. An equation expresses this:

$$(\text{make-dillo } (\text{dillo-weight } \text{dillo}) (\text{dillo-alive? } \text{dillo})) = \text{dillo}$$

We can therefore replace the `make-dillo` call with `dillo` and thus simplify the program using the equation:

```
(define feed-dillo (lambda
  (dillo)
  (if (dillo-alive? dillo)
      (make-dillo (+ (dillo-weight dillo) 500) #t) dillo)))
```

Now the first version of `feed-dillo` has been created without us having to hastily consider that in the case of "armadillo dead" the input is the same as the output.

In this example, the resulting insight and simplification may seem trivial. However, we can often use equations in programming to gain useful insights or to make our programs shorter, faster, or more elegant. This corresponds to Mantra 6 on page 46:

Mantra 6

Use equations to simplify your program.

Task 3.8

Formulate equations for the record types `computer` and `wallclock-time` from this chapter, following the same principles as for `dillo`.

TASKS

Task 3.9

Write a data definition and a record definition for *fractions* and various functions for the Fractions:

- Reducing a fraction

- Test for equality of rational numbers represented by two fractions •

Addition, subtraction, multiplication and division of fractions

Note: The following built-in function is helpful for solving the task:

```
(: gcd (natural natural -> natural)),
```

which calculates the greatest common divisor of two natural numbers.

Task 3.10

Each Qux has a name. Experts are also interested in how many Bas a Qux has. A further distinction is made between Arg-Quxen, Foo-Quxen, and Bla-Quxen.

1. Write a data definition for Quxe and a corresponding record definition. Also note the signatures of the selectors.
2. Write a signature, skeleton, and template for a function that accepts a Qux and returns a string. Identify the construction guidelines used. Be sure to also apply the construction guidelines for the components of Qux records.
3. Assume that you have defined the following signature for a function to be written, `quxop2`:
lays:

```
(: quxop2 (natural (enum "Hx" "Bx" "Px") -> qux))
```

(It is assumed that the record definition for a Qux has the name `qux`.) Develop the framework and template of the function to be written using the appropriate construction instructions.

Task 3.11

Write a program to manage weekly room reservations at the university!

1. Design a data and record definition for an entry in an administrative system for Lecture and seminar rooms.
Each entry includes the following information: the name of the room (as a string), the day of the week, the time (only hours are counted), and the name of the lecturer occupying the room.
2. Write a function called `reserve`` that accepts an entry and a lecturer's name as arguments and returns an entry. If the room has not yet been booked (i.e.,

The lecturer's name is listed in the entry as ""), the room is to be reserved and thus a new
 The entry will be returned where the lecturer's name is set. Otherwise, the
 Entry returned unchanged.

Task 3.12

Write out additional functions for the computers from section 3.1:

- Consider how you would calculate a suitable price for a computer depending on its configuration.
 Write a function that implements your method.
- Write a function that expands a computer's memory. It accepts a computer and a number and returns
 a new computer with the main memory expanded by the number of units.
 The number is increased.

Task 3.13

It's about baking cakes.

1. Create a data definition called "dough" for the dough. Every dough consists of eggs, flour, and sugar.
 and water, and has a total weight. Think about suitable units for the ingredients.
2. Create a data definition called "cake" for a cake. This contains a dough, a baking time, and a time in...
 Minutes and the final weight of the cake.
3. Write a function `ingredients->dough` which returns a number of eggs, a quantity
 Flour in grams, a quantity of sugar in grams, and a quantity of water in milliliters are obtained.
 and makes a dough from it. Assume that each egg weighs 64 g.
4. Write a function called `bake-cake``. This function receives a batter, a baking time in minutes, and
 Makes a cake. Assume that after baking, 80% of the water will still be in the cake-
 chen are.

Task 3.14

Write a data definition for `appointments`, consisting of date, time, and duration.

(in minutes) and location. For the date and time, use further data definitions consisting of day, month,
 and year, and hour and minute, respectively.

1. Write a function called `date-ok?` that determines whether a date value corresponds to an actual
 calendar date, i.e., distinguishes correct dates like 1/1/1970 from nonsensical ones like 34/17/2006.
 Ignore leap years. Consider months with 28, 30, and 31 days.
2. Write a function called `date-equal?` that compares whether two date values are equal.

3. Write a function ``time-ok?`` that determines whether a time value corresponds to an actual time.
corresponds.
4. Write a function ``time-overlap?`` that checks if two times of a given duration (in minutes) overlap.
Assume that these times are on the same day.
5. Write a function `called overlap` that checks whether two given dates overlap.
Note the duration of the appointments. Assume that the appointments do not extend past midnight.

Note: The built-in function can be used to solve the task.

(: remainder (natural natural -> natural))

It can be helpful. It calculates the remainder of an integer division.

Task 3.15

Create a data definition and a record definition for a vehicle registration certificate (see Figure 3.2). Organize the vehicle registration document fields logically into subgroups and create separate data and record definitions for these subgroups. Use descriptive names for records and fields! Provide an example by generating a vehicle registration document value with all entries.

Task 3.16

Write a program for Tübingen's city traffic that checks whether a vehicle is allowed to drive in the environmental zones.

1. Define a data type for vehicles. This data type should represent the type, the license plate number, and include the vehicle's emissions class.
2. Create example vehicles for the vehicle types "city bus", "coach", "diesel car", and "gasoline car". Assume that the buses belong to emissions class 2, the diesel car to emissions class 3, and the gasoline car to emissions class 4.
3. Write a function `called "driving ban?"` that checks whether a given vehicle is still allowed to drive given a minimum emissions class. Design the function signature so that it only accepts minimum emissions classes from 1 to 4.
4. Mühlstraße in Tübingen is closed in one direction to all vehicles except city buses.
Write a function `sonderrecht?` that checks whether a given vehicle is allowed to drive on Mühlstraße in the closed direction.

5. The city council proposes to boost tourism by allowing tour buses to use Mühlstrasse in the restricted direction on Sundays. To achieve this, extend the "special right" function to include weekdays and permit tour buses on Sundays as well.

When writing the function, use the construction guidelines for functions and for case distinctions. Write test cases that cover all possible case distinctions.

Task 3.17

Write a program for a parcel delivery service that calculates the price of a package!

1. Write a data definition and a record definition for *addresses*. An address includes the Name, street address with house number, postal code, city and country.
2. The parcel service charges a surcharge for shipments sent internationally.
Write an *international* function that takes an *address* as an argument and determines whether the address is located abroad.
3. The parcel service has a special rate for shipments sent within the same postal code area. Write a function *`same-zip-code?`* that takes two *addresses* as arguments and determines whether the postal codes and countries of the two addresses are the same.
4. A package is classified according to its dimensions. Write a data definition and a record definition for *dimensions*. Dimensions consist of length, width, and height.
5. Package prices depend on the size of the package being sent. The parcel service uses three size categories— *Small*, *Medium*, and *Large*— to calculate the cost. The decisive criterion for the package size is the sum of the longest and shortest sides of the package.

Write a function *`add-longest-and-shortest-side`* that takes a *dimension* as its argument. The return value of *`add-longest-and-shortest-side`* should be the sum of the longest and shortest sides of the dimension. Outsource the sub-problems to two helper functions: *`longest-side`* and *`shortest-side`*.

6. Write a data definition and a record definition for *packets*. A packet has a sender and subscriber address. A recipient address. Use the existing record definition for the addresses. A package also has other properties: dimensions (use the existing record definition for this), weight, delivery time, and an optional cash-on-delivery option. The delivery time should be *normal*, *next-day*, or *next-morning*.
7. Write a function *`parcel-size-class`* that takes a *parcel* as an argument and returns the size class. The parcel size is determined by its dimensions (see above).
The following table shows the relationship between package size and dimensions:

Package size	dimensions
S	0–50 cm
M	>50–100 cm
L	>100 cm

8. Write a function ``calculate-base-postage`` that takes a *package* as an argument. and the basic postage costs for this package are calculated.

Use the following basic tariff table of the parcel service as a basis:

Package size	Weight		
	0-5kg	>5-10kg	>10kg
S	3.00	6.00	9.00
M	6.00	10.00	14.00
L	9.00	15.00	21.00

9. Write a function ``transportation-time-factor`` that takes a *package* as an argument and returns the surcharge factor for the transit time. Use the following surcharge factors as a basis:

Transport distance	Transport duration		
	normal	next-day	next-morning
same postcode -25%		+0%	+25%
inland	+0%	+50%	+100%
Abroad	+100%	+200%	+300%

10. Write a function called ``cash-on-delivery-surcharge`` that takes a *package* as an argument and returns the cash-on-delivery surcharge. Include the following surcharges. underlying:

Transport distance, cash	on delivery fee
inland	+3.00
Abroad	+9.00

11. Write a function ``calculate-postage`` that takes a *package* as an argument and that Postage costs have been calculated. Use the already programmed solutions for the various sub-problems for this.

Figure 3.2: Front and back of a vehicle registration document

4 MIXED DATA

Sometimes, at one point in our problem, different classes of the same type come together.

Data before:

- An animal can be an armadillo or a parrot. • A coordinate can be a Cartesian coordinate or a polar coordinate.
- A meal can be breakfast, lunch, or dinner.

Such data are called *mixed data*, and they represent another form of case distinction, in addition to the enumerations and number ranges known from Chapter 2.

Although the data are diverse, they support common operations: The weight of an animal can be calculated for both armadillos and parrots, the distance from the origin can be calculated for both coordinate representations, and the number of courses can be determined for each type of food.

4.1 MIXED DATA

[mixed-data/animal.rkt](#)

code

The introduction mentioned parrots: we use them to introduce mixed data.

Before we can do that, however, we need to define parrots using the usual methods. To do this, we'll extend the file containing the armadillo from the previous chapter.

Just as with armadillos, we are interested in the weight of parrots, but since parrots are not usually run over on Texas highways, we assume they are always alive. Furthermore, we are only considering talking parrots, each capable of saying a single sentence. Here is the data definition:

A parrot has the following characteristics:

; - Weight in grams

; - sentence he says

Here is the corresponding record definition:

```
(define-record parrot make-  
  parrot (parrot-  
    weight natural) (parrot-sentence  
    string))
```

...and the corresponding signatures:

```
(: make-parrot (natural string -> parrot))
(: parrot-weight (parrot -> natural))
(: parrot-sentence (parrot -> string))
```

Here are two examples of parrots with comments describing the relationship between data and information:

```
(define parrot1 (make-parrot 10000 "The gardener did it."); 10kg,
  Miss Marple
(define parrot2 (make-parrot 5000 "I love you."); 5kg,
  Romantics
```

You can probably imagine parrots and armadillos meeting in a program, so appearing *mixed together*. Parrots and armadillos belong to the same general category.

Animal. A description could look like this:

An *animal* is one of the following:

- an armadillo
- a parrot

The phrase “one of the following” is already familiar to you from section 2.4 and construction instructions 7 on page 74: It indicates a case distinction.

However, since armadillos and parrots are not each represented as just one value, it This is not a list: The two cases of the case distinction describe different *classes* of animals, each with its own signature. This results in a new organization of data: *mixed data*. Accordingly, it makes perfect sense to search by the “weight of an animal”.
to ask or “to feed an animal”, which is what we intend to do below.

The description of the term “animal” is already suitable as a data definition and must be used for Inclusion in the program can only be reformatted as a comment:

```
An animal is one of the following:
; - Armadillo
; - Parrot
```

For composite data, the data definition can be converted into a record definition. In this case, there is already a record definition for each individual class, *Animal*.

If we want to process animals in general into functions, we need a signature.

for animals. For example, we want to write a function that determines the weight of each animal.

This function could have the following signature:

```
(: animal-weight (animal -> natural))
```

We therefore need a definition for the signature `animal`. This looks like this:

```
(define animal
  (signature
    (mixed dillo parrot)))
```

We are familiar with the `signature` from the case distinctions in section 2.4 on page 57. The `mixed` is new and stands for "mixed data," meaning case distinctions where each case has its own signature. You can read the definition above as "Animals are a mix of armadillos and parrots"; however, this sounds rather clunky in German, which is why we will stick with the formulation "one of the following" for the data definition. With this definition, the signature `animal` is available. We have already previewed the signature for `animal-weight`. Here it is again, along with a brief description:

Determining an animal's weight

```
(: animal-weight (animal -> natural))
```

This function should work for armadillos *and* parrots as well, so we need Test cases for both:

```
(check-expect (animal-weight dillo1) 55000) (check-expect
(animal-weight dillo2) 58000) (check-expect (animal-weight
parrot1) 10000) (check-expect (animal-weight parrot2) 5000)
```

The scaffolding looks like this:

```
(define animal-weight
  (lambda (animal)
    ...))
```

Animals also form a case distinction in the data, with two cases: armadillos and parrots. Therefore, in the body of the function, we need a branch with two branches:

```
(define animal-weight (lambda
  (animal)
    (cond
      (... ...)
      (... ...))))
```

Next, we need two conditions – one that identifies armadillos and one that identifies parrots. To achieve this, we extend the record definitions by adding a new element: the *predicate*. The pre-

Dikate will allow us to write the terms. The record definitions will then look like this:

like this:

```
(define-record dillo
  make-dillo
  dillo? ; ÿ= predicate (dillo-
    weight natural) (dillo-alive?
    boolean))

(define-record parrot
  make-parrot
  parrot? ; ÿ= Predicate (parrot-
    weight natural) (parrot-sentence
    string))
```

Two new names have been added: `dillo?` and `parrot?`. These are the predicates, and they have the following signatures:

```
(: dillo? (any -> boolean)) (: parrot?
  (any -> boolean))
```

The ``any` predicate` is also new and serves as a signature for any value: a predicate can therefore be applied to absolutely anything. These two predicates distinguish armadillos and parrots from other values:

```
(dillo? dillo1)
ÿ #t

(dillo? parrot1)
ÿ #f

(parrot? dillo1)
ÿ #f

(parrot? parrot1)
ÿ #t

(dillo? 5)
ÿ #f

(parrot? "foo")
ÿ #f
```

In general, a predicate is a function with the signature `(any -> boolean)` that returns `#t` for a certain type of value, but `#f` for everything else. In this case, the

A `define-record` form has the following general form:

```
(define-record t
  c
  p
  (sel 1 sig1)
  ...
  (sel n sign))
```

This form defines a record type with n fields. Here, t , c , $\text{sel } 1 \dots \text{sel } n$ are variables for which all `define-record` definitions are created:

- t is the name of the record signature.
- c is the name of the constructor with the following signature:
 $(: c (\text{sig1} \dots \text{sign} \rightarrow t))$
- p is the name of the predicate that distinguishes records of this type from other values.
 The predicate can also be omitted.
 The predicate has the following signature:
 $(: p (\text{any} \rightarrow \text{boolean}))$
- $\text{sel } 1, \dots, \text{sel } n$ These are the names of the selectors for the fields of the record type. The selector $\text{sel } i$ has the following signature:
 $(: \text{sel } i (t \rightarrow \text{sigi}))$

Figure 4.1: `define-record` (with predicates)

The predicates ``dillo?`` and ``parrot?`` each represent the values belonging to a specific record type.

Predicates do not necessarily have to belong to a record type; occasionally, we will define some ourselves. Write. A number of predicates are predefined – more on this on page [128](#).

Task 4.1

Write a predicate "`animal?`" for animal, `dillo`, or `parrot` records.

Ÿ

We can use the predicates `dillo?` and `parrot?` in the template:

```
(define animal-weight
  (lambda (animal)
    (cond
      ((dillo? animal) ...)
      ((parrot? animal) ...))))
```

In the first branch – the branch for armadillos – `dillo-weight` is now used, in the second branch – for parrots – `parrot-weight` is responsible:

```
(define animal-weight
  (lambda (animal)
    (cond
      ((dillo? animal) (dillo-weight animal)) ((parrot? animal)
      (parrot-weight animal))))))
```

Figure 4.1 is an updated description of the form of `define-record`, which also takes predicates into account.

This example provides a construction guide for mixed data. First, the data analysis:

Design Guide 15 (Mixed Data: Data Analysis)

Mixed data consists of case distinctions where each case is a separate class of data with its own signature. For mixed data, write a signature definition of the following form below the data definition:

```
(define sig
  (signature
    (mixed sig1 sig2 . . . sign)))
```

Sig is the signature for the new data type; sig1 to *sign* are the signatures from which the new data type is mixed.

The construction instructions for the template are similar to those in construction instructions 10 for case distinctions on page 76 :

Construction instructions 16 (Mixed data as input: template)

You can create a template for a function and its test cases that accepts mixed data.

You can construct it as follows:

- Write tests for each of the cases. •

Write a conditional branch as the body in the template that has exactly *n* branches – that is, exactly as many branches as there are cases in the data definition or signature.

- For each branch, write a condition that identifies the corresponding case. • Complete the branches by performing a data analysis for each individual case.

you take and use the corresponding help functions and construction instructions.
 The clearest programs usually arise when separate helper functions are defined for each case.

A construction guide or template for mixed data as *output* is unnecessary – you simply use the template for the relevant case.

Note the difference between `enum` and `mixed`, which are easily confused: `enum` stands for “one of the following *values*”, while `mixed` stands for “belonging to one of the following *signatures*”.

Task 4.2

Think of meaningful data definitions for breakfast, lunch, and dinner (a ... eat). consists of ...), so it makes sense to ask the question "What kind of vegetable is included?" for each type of food. First, write a function for each type of food that describes the vegetable it contains. provides. Then write a data definition for the general term "food"; a meal can be breakfast, lunch, or dinner. Write a function that returns the contained
 Provides vegetables for a meal. y

4.2 MIXED DATA OUTPUT

We can feed a parrot similarly to an armadillo – only the portion is smaller, let's assume 50 g. Brief description and signature:

Feed the parrot 50g of food.

(: feed-parrot (parrot -> parrot))

Test cases:

```
(check-expect (feed-parrot parrot1)
  (make-parrot 10050 "The gardener did it."))
(check-expect (feed-parrot parrot2)
  (make-parrot 5050 "I love you."))
```

Scaffolding:

```
(define feed-parrot
  (lambda (parrot)
    ...))
```

The template is created from the combination of the templates for composite data as input (construction guide 13 on page 101) and as output (construction guide 14 on page 101):

```
(define feed-parrot (lambda
  (parrot) (make-
    parrot ... ...) ... (parrot-weight
    parrot) ... ... (parrot-sentence parrot) ...))
```

...and finally the complete hull:

```
(define feed-parrot (lambda
  (parrot) (make-parrot
    (+ (parrot-weight parrot) 50)
    (parrot-sentence parrot))))
```

Now we can write a function that accepts any animal and feeds it. Here is a short description and signature:

Feed the animal

```
(: feed-animal (animal -> animal))
```

The function should behave exactly like `feed-dillo` or `feed-parrot`. We can express this directly as tests:

```
(check-expect (feed-animal parrot1) (feed-parrot parrot1)) (check-expect (feed-animal
parrot2) (feed-parrot parrot2)) (check-expect (feed-animal dillo1) (feed-dillo dillo1))
(check-expect (feed-animal dillo2) (feed-dillo dillo2))
```

The scaffolding looks like this:

```
(define feed-animal (lambda
  (animal)
  ...))
```

Since the input is mixed, we need a branch with as many branches as `animal` has cases, i.e., two, which we will immediately equip with the appropriate conditions:

```
(define feed-animal
  (lambda (animal)
```

```
(cond
  ((dillo? animal) ...)
  ((parrot? animal) ...)))
```

Finally, we need to add the answers for which we use the functions we have already written:

```
(define feed-animal
  (lambda (animal)
    (cond
      ((dillo? animal) (feed-dillo animal))
      ((parrot? animal) (feed-parrot animal))))))
```

This example shows that we do not have a specific design guide for mixed data as output. What we need: We just need to make sure we deliver the correct case in each instance.

Task 4.3

Write a `run-over-animal` function that runs over an animal!

ÿ

4.3 THE FOOD TRAFFIC LIGHT SYSTEM

`mixed-data/sugar-traffic-label.rkt`

code

In this section, we will look at another example of mixed data, this time from the outset using the construction instructions from the previous section.

The traffic light labeling system is a labeling system on food packaging that indicates the content of health-relevant nutrients using traffic light colors: green for low levels.

Content: yellow for medium content and red for high content. For sugar, for example, the

This is how the traffic light system looks, based on 100g of a food product:

green,	low content less than 5 g
yellow	average content between 5 g and 22.5 g
red	high salary more than 22.5 g

Despite the efforts of the European Union, the terminology is inconsistent. Technically The traffic light system is redundant if the sugar content is given in grams. Sometimes it is However, the sugar content is also given separately for fructose and glucose.

However, a program could simplify handling by transferring the information from a food package – total sugar in grams, fructose and glucose separately, and the traffic light rating – into the It brings a uniform traffic light design.

Let's write such a program. First, the data analysis: • The amount of sugar in grams is a (rational) number. • Sugar fractions consist of the amount of fructose and glucose. • A sugar traffic light is red, yellow, or green. • The sugar content can be expressed either as the amount of sugar, sugar fractions, or a sugar traffic light. become.

Here is the data definition for the compound sugar fractions:

```
; Sugar components consist of:
; - Amount of fructose (in g)
; - Amount of glucose (in g)
```

This directly results in the record definition with two components, each a rational number:

```
(define-record sugars make-
  sugars sugars?
  (sugars-
    fructose-g rational) (sugars-glucose-g
    rational))
```

Here are some examples, along with comments that describe the relationship between data and information:

```
(define s1 (make-sugars 1 1)) ; 1 g fructose, 1 g glucose (define s2 (make-sugars 2 3)) ;
2 g fructose, 3 g glucose (define s3 (make-sugars 5 5)) ; 5 g fructose, 5 g glucose (define
s4 (make-sugars 10 2.5)) ; 10 g fructose, 2.5 g glucose (define s5 (make-sugars 10 13)) ;
10 g fructose, 13 g glucose (define s6 (make-sugars 15 10)) ; 15g fructose, 10g glucose
```

The traffic light itself is a simple case distinction:

```
A traffic light is one of the following values:
; - red
; - yellow ; -
green
```

This is a list; we assign a name according to the corresponding signature:

```
(define traffic-light (signature
  (enum "red"
    "yellow" "green")))
```

The information about the sugar content can take any of the three forms mentioned above:

; A sugar content is one of the following:
 ; - Weight in grams
 ; - sugar content
 ; - Traffic light designation

The wording again indicates that the data is mixed, specifically with three cases. We translate this into a signature definition with "mixed" and also three cases:

```
(define sugar-content (signature
  (mixed rational
    sugars
    traffic light)))
```

This example shows that the cases in a definition for mixed data do not all have to be records. However, it is important that the cases are *disjoint*, meaning that each value can be uniquely assigned to one of the cases: otherwise, it would not be possible to write a meaningful branch that distinguishes between the cases.

Now to our function for determining the traffic light label for sugar content. Here is a brief description and signature:

```
; Determine traffic light designation for sugar content (: sugar-traffic-
light (sugar-content -> traffic-light))
```

We need quite a few test cases to cover all cases of sugar content as well as the extreme cases of the table above.

```
(check-expect (sugar-traffic-light 2) "green") (check-expect (sugar-
traffic-light 5) "yellow") (check-expect (sugar-traffic-light 10) "yellow")
(check-expect (sugar-traffic-light 12.5) "yellow") (check-expect (sugar-
traffic-light 23) "red")
```

```
(check-expect (sugar-traffic-light s1) "green") (check-expect (sugar-traffic-
light s2) "yellow") (check-expect (sugar-traffic-light s3) "yellow") (check-
expect (sugar-traffic-light s4) "yellow") (check-expect (sugar-traffic-light
s5) "red")
```

The following predicates are built in:

- `number?` tests whether a value is a number.
- `real?` tests whether a value is a real number.
- `rational?` tests whether a value is a rational number.
- `natural?` tests whether a value is a natural number.
- `string?` tests whether a value is a string.
- `boolean?` tests whether a value is a Boolean value.

Figure 4.2: Built-in predicates

```
(check-expect (sugar-traffic-light s6) "red")
```

```
(check-expect (sugar-traffic-light "green") "green")
(check-expect (sugar-traffic-light "yellow") "yellow")
(check-expect (sugar-traffic-light "red") "red")
```

Next up, as always, is the scaffolding:

```
(define sugar-traffic-light (lambda
  (sugar-content) ...))
```

Next, we apply the template for functions that accept mixed data. We need a branch with as many branches as `sugar-content` has cases, which is three:

```
(define sugar-traffic-light (lambda
  (sugar-content) (cond
```

```
  (... ...) (... ...)
```

```
  (... ...))))
```

Next, we need tests for the three cases. The second case is simple, since it involves `sugars` records: there's the predicate `sugars?`. However, the first case involves a rational number, and the third a string – both built-in data types.

These include the built-in predicates `rational?` and `string?` – Figure 4.2 lists even more built-in predicates. Armed with this information, we can complete the tests:

```
(define sugar-traffic-light (lambda
  (sugar-content) (cond
```

```
((rational? sugar-content) ...) ((sugars?
sugar-content) ...) ((string? sugar-
content) ...))))
```

The first branch is not only a rational number, but also a case distinction with three cases according to the table from the beginning:

```
(define sugar-traffic-light (lambda
(sugar-content) (cond

((rational? sugar-content) (cond

(... ...)
(... ...)
(... ...)))

(sugars? sugar-content) ...) ((string?
sugar-content) ...))))
```

Next, we will add tests according to the table:

```
(define sugar-traffic-light (lambda
(sugar-content) (cond

((rational? sugar-content) (cond ((<
sugar-
content 5) ...) ((and (>= sugar-
content 5) (<= sugar-content 12.5)) ...) ((> sugar-content 12.5) ...)))

(sugars? sugar-content) ...) ((string?
sugar-content) ...))))
```

Finally, we need to enter the answers:

```
(define sugar-traffic-light (lambda
(sugar-content) (cond

((rational? sugar-content) (cond

((< sugar-content 5) "green")
```

```
((and (>= sugar-content 5) (<= sugar-content 12.5)) "yellow") ((> sugar-content 12.5) "red"))
```

```
((sugars? sugar-content) ...) ((string? sugar-  
content) ...)))
```

The nested ``cond`` is somewhat confusing: Construction Manual 16 , page 123 , already notes that it is advisable to move this branch into a separate helper function. Here is a short description and signature for this helper function:

```
; Convert sugar content in g to traffic light (: sugar-weight-  
>traffic-light (rational -> traffic-light))
```

The test cases can be derived from the test cases for `sugar-traffic-light` by simply copying and renaming them:

```
(check-expect (sugar-weight->traffic-light 2) "green") (check-expect (sugar-weight-  
>traffic-light 5) "yellow") (check-expect (sugar-weight->traffic-light 10) "yellow") (check-  
expect (sugar-weight->traffic-light 12.5) "yellow") (check-expect (sugar-weight->traffic-  
light 23) "red")
```

Scaffolding:

```
(define sugar-weight->traffic-light (lambda (sugar-  
weight) ...))
```

We've already written the hull; we just need to move it here and then rename ``sugar-content`` to ``sugar-weight`` . In DrRacket, after clicking "Syntax Check," you can right-click on the ``sugar-content`` parameter to open a menu that includes the option "Rename sugar-content ." DrRacket will then ensure that all related occurrences of ``sugar-content`` are renamed in the same way.

```
(define sugar-weight->traffic-light (lambda (sugar-  
weight) (cond  
  
((< sugar-weight 5) "green") ((and (>=  
sugar-weight 5) (<= sugar-weight 22.5)) "yellow") ((> sugar-weight 12.5) "red"))))
```

Back to the [sugar-traffic-light](#) function . There we use the newly defined auxiliary radio...
tion:

```
(define sugar-traffic-light (lambda
  (sugar-content) (cond ((rational?
    sugar-
      content) (sugar-weight->traffic-light
        sugar-content))
    ((sugars? sugar-content) ...) ((string?
      sugar-content) ...))))
```

The next branch deals with the case of [sugars](#). This involves composite data, so we'll write the template with the selector calls in this branch:

```
(define sugar-traffic-light (lambda
  (sugar-content) (cond
    ((rational? sugar-content) (sugar-
      weight->traffic-light sugar-content)) ((sugars? sugar-content) ...
    (sugars-fructose-g sugar-
      content) ... ... (sugars-glucose-g sugar-content) ...) ((string?
      sugar-content) ...))))
```

We need to add the fructose and glucose content and convert the sum into a traffic light color according to the table from the beginning. This can again be done using the [sugar-weight->traffic-light](#) function :

```
(define sugar-traffic-light (lambda
  (sugar-content) (cond ((rational?
    sugar-
      content)
    (sugar-weight->traffic-light sugar-content)) ((sugars? sugar-
      content)
    (sugar-weight->traffic-light
      (+ (sugars-fructose-g sugar-content)
        (sugars-glucose-g sugar-content)))) ((string? sugar-
      content) ...))))
```

That leaves the last case – which is thankfully trivial, since it's already a color that `sugar-traffic-light` just needs to return:

```
(define sugar-traffic-light (lambda (sugar-
  content)
  (cond
    ((rational? sugar-content) (sugar-weight-
      >traffic-light sugar-content)) ((sugars? sugar-content) (sugar-weight-
      >traffic-light (+ (sugars-fructose-g
        sugar-content)

          (sugars-glucose-g sugar-content)))) ((string? sugar-
        content) sugar-content))))
```

Complete!

4.4 REPRESENTING ERRORS

`mixed-data/slope.rkt`

code

This section describes another common application for mixed data.

This concerns situations where a function does not know a meaningful answer.

As an example, we will write a function that calculates the slope of a line on the plane using the coordinates of two points on it. The function accepts four arguments: the x- and y-coordinates of the first point, and then the x- and y-coordinates of the second.

Full stop. We'll start with a short description, signature, some tests, and a framework:

```
; Calculate the slope of a straight line (: slope (real
real real real -> real))
```

```
(check-expect (slope 0 0 1 1) 1) (check-expect
(slope 0 0 2 1) 1/2) (check-expect (slope 1 2 3 5)
3/2)
```

```
(define slope
  (lambda (x1 y1 x2 y2) ...))
```

The formula collection states that to calculate the slope, we must divide the difference in the Y-coordinates by the difference in the X-coordinates:

```
(define slope
  (lambda (x1 y1 x2 y2) (/ (- y2
                                y1)
                             (- x2 x1))))
```

Finished, one might think. But there are lines that have no gradient. In the REPL it looks like this.

out of:

```
> (slope 0 0 0 1) /:
divided by 0
```

The message is rather unfriendly and presents several problems:

- While the message states what the program did (divide by zero), it doesn't identify the cause of the problem: that the line has no slope.
- The program simply terminates without giving it a chance to resolve the issue.
- to deal with it.

- The fact that the program can simply abort is not apparent from the signature, which suggests that the function always returns a real number.

We can solve all three problems at once with mixed data. To do this, we first need to ensure that the function returns a meaningful value before it would divide by zero.

This divides the inputs into two classes – there is either a slope or there isn't.

We use binary branching to distinguish between these two classes:

```
(define slope
  (lambda (x1 y1 x2 y2) (if (= (-
                                x2 x1) 0)
                              ...
                              (/ (- y2 y1) (- x2
                                            x1)))))
```

We simply need to replace the ellipse with something else in the program. We can't just use any real number, as that could also represent a legitimate slope. Instead, we need a value that we can distinguish from legitimate numbers.

We could use a special string or `#f`. (See exercise 4.10 on page 139.) However, we recommend defining a custom record type instead. This record type has no fields, meaning it can only be used to create a single value, namely (`make-no-slope`):

There is no slope (define-record
no-slope make-no-slope no-
slope?)

(This is therefore a degenerate version of composite data without components.) However, we can distinguish the value of (make-no-slope) from all other values by the predicate `no-slope?` and use it when there is no slope:

```
(define slope
  (lambda (x1 y1 x2 y2) (if (= (-
    x2 x1) 0)
    (make-no-slope) (/ (-
    y2 y1) (- x2
    x1)))))
```

Now we can define a test case:

```
(check-expect (no-slope? (slope 0 0 0 1)) #t)
```

However, DrRacket reports a signature violation for this test case: The signature still pretends that it will always produce a real number. Therefore, we extend it as follows:

```
(: slope (real real real real -> (mixed real no-slope)))
```

This signature clearly shows what it delivers, and the names also indicate the meaning of the two cases. Of course, we could define a custom signature as we would for mixed data, but that's hardly worthwhile here.

TASKS

Task 4.4

In the `sugar-weight->traffic-light` function on page 130, simplify the conditions as described in section 2.5 on page 68.

Task 4.5

A supermarket wants to manage its goods using a program. There are three product categories:

Food described by a name, the price per piece, the best-before date, and the current stock in the supermarket

Beverages are described by name, unit price, best-before date, and stock level. It must also be noted whether a deposit is required.

Others described by a name, unit price and stock level.

1. Conduct a data analysis and create data and record definitions.
2. Write a function `unitprice` that accepts a product class and assigns the unit price. returns.
3. Write a function called `"book"` that accepts a product class and the number of units to be booked, reduces the stock of the product class, and returns the product class.
If more copies are requested than are available in the product category, the stock level should be set to 0.
4. Write a function called `"still usable?"` that accepts a product category and a date and returns `#t` if the best-before date (BBD) has not been exceeded. If no BBD is known, return `#f`.

Task 4.6

On his travels around the world, Dr. Sperber encounters many interesting and eccentric characters. Among them is Dr. Knaubichler, an expert in the field of breeding mystical creatures. There are three classic basic creatures: • The Garnolaf,

which possesses strength. • The

Ronugor, which possesses knowledge.

• The Tschipotol, which possesses a willingness to take risks.

The creatures' characteristics vary in intensity. The Knaubichler creature characteristic scale ranges from 0 to 100. Each basic creature has only one characteristic; none possesses a characteristic of another basic creature.

Now Dr. Knaubichler crossbreeds the basic creatures. Thus, new creatures are created.

• The Ronulaf, with knowledge and strength. •

The Chigor, with knowledge and a willingness to take risks. •

The Lapotol, with strength and a willingness to take risks. •

The Chirgaronu, with knowledge, strength, and a willingness to take risks.

Unfortunately, the hybrids do not inherit the full traits of both parent creatures.

When different basic creatures are crossed, the following applies: When Ro-nugor and Tschipotol are crossed, the inherited knowledge is reduced by 10%; when Tschipotol and Garn-nolaf, however, risk tolerance decreases by 5%, but strength increases by 8%. Garnolaf

Ronugor and the base creatures increase strength by 5%. If you cross all three base creatures, each one gains a certain amount of strength. Property decreases by 3%.

When two creatures of the same type are bred, a new creature is created with the combined $\frac{2}{3}$ the characteristics of both original creatures. Only original creatures can be bred.

In any case, a property can have a maximum value of 100.

Dr. Knaubichler needs a program that calculates the new creature before he can...

He's making intersections. Help him!

1. Perform a data analysis and create appropriate data and record definitions. Provide all data definitions. Natures of the record functions!
2. For each of the intersections of two basic creatures listed above, write a function, which performs the crossbreeding and produces a new creature.

Task 4.7

Extend the solution to problem 4.6:

1. Write a function that accepts two basic creatures in any order, the cross-tongue makes a change and returns a new creature.
2. Write a function that accepts three basic creatures in any order, that cross-tongue makes a change and returns a new creature.

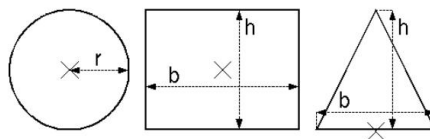
Task 4.8

The game development team at *Exciting Games, Inc.* is experiencing staff shortages: For programming The groundbreaking 2D game *The Adventures of Dr. Racket* urgently needs a representation of the The graphic shapes circle, rectangle, and triangle are needed. Unfortunately, all hackers are incredibly busy moving colorful pixels around and therefore can't dedicate time to this.

You are to dedicate yourself to a fundamental task. That's why this task falls to you! You only receive a piece of paper smeared with pizza topping, apparently intended as a work instruction. It reads:

There are three classes of shapes: circles, rectangles, and isosceles triangles. All shapes have an origin coordinate and contain information about their size (radius, height, width, etc.), as you can see in the diagram. Use the origin coordinate as shown.

the given Cartesian coordinates!



- There should be functions that work on all forms and do the following:
 - return the Cartesian coordinate of a shape
 - Return the x-coordinate of a shape
 - Return the y-coordinate of a shape
 - Return the distance of the origin of a shape to the origin of the coordinate system
 - Return the area of a shape
 - Shift the shape by $\ddot{y}x$ in the x-direction and by $\ddot{y}y$ in the y-direction, i.e., move the shape back-give, whose original coordinate is shifted accordingly
 - Comparing the area of two shapes: For shapes of equal size, the function provides the Return the string "equal" , otherwise "smaller" if the first form is smaller, or "bigger" if the first form is larger.

Show the hackers what real software engineering is and use the appropriate construction manuals to write representations for the shapes and their associated functions!

Task 4.9

There are various violations of the road traffic regulations:

- Illegal parking violations, including the location and time of the offense
- Running a red light, including the location and time of the violation, as well as the duration in seconds of how long the light had been red.
- Excessive speed, including the location and time of the violation as well as the amount of the speeding violation

In addition to the listed components, it must later be noted for each violation whether there is also a "danger to road traffic".

1. Write a data definition for traffic violations. Write data- and record definitions for the various violations.

You can represent the times using strings like "11.4.1971 22:00".

Include all signatures!

2. Write a function that accepts any violation and returns the location of the violation. returns the β es.

Similarly, write a function that accepts any violation and returns the time of the violation.

3. Write a function that accepts any violation and marks it as a threat. of road traffic considered and returns (that is, every violation is a violation) (with risks).

The violations result in different criminal offenses:

- Simple offense with a fine
 - Administrative offense with fine, points on the traffic offenders register and driving ban in Mo-data
 - Criminal offense resulting in points on the traffic offenders record and a prison sentence of months
1. Write a data definition for offenses related to violations of the road traffic regulations. Write data and record definitions for the different offenses. Include all Signatures on!
 2. Write a function that accepts any given set of facts and returns the number of points awarded.

Now write functions that accept violations and calculate the offenses:

1. Write a function that accepts illegal parking and returns a simple offense with a €20 fine. If the illegal parking poses a danger to road traffic (to
For example, in the case of obstructing emergency vehicles, the function should return a minor offense with a €40 fine, one point and no driving ban.
2. Write a function that accepts running a red light and returns a traffic violation. The following applies:
 - If the traffic light was red for less than one second and there was no danger: €50, 3 points, no driving ban
 - If the traffic light was red for at least one second or there was a hazard: €125, 4 points, 1-month driving ban
3. Write a function that accepts excessive speed and returns the result. The following applies:
 - Speeding by less than 20 km/h without endangering others: simple offense with €35
 - Speeding between 20 and 40 km/h (inclusive) without endangering others: Ord-Traffic violation with a €75 fine, 3 points and a 1-month driving ban
 - Speeding by more than 40 km/h without endangering others: Administrative offense-keit with a €200 fine, 4 points and a 3-month driving ban
 - If road traffic is endangered simultaneously: criminal offense with 3 more points than indicated without endangerment and a prison sentence twice as long as the driving ban that would have been imposed without endangerment
Danger applies (if there is no driving ban, then there is also no prison sentence)
4. Write a function that accepts a violation and returns its consequence. Use the Functions from the previous subtasks.

Task 4.10

For the `slope` function in section 4.4 on page 132 , experiment with representations other than a separate record type for "no slope":

- Return `#f` .
- Return the string `"no slope"` .

What should the signature of `slope` be in each case?

How could a program determine, after a call, whether there was no slope?

For both variants, write a replacement for the predicate `"no-slope?"` that states this situation.

What do the three variants for representing "no slope" each have in common?

What are the advantages and disadvantages?

5. Programming with self-references and combinators

So far, we have only presented data that have a fixed size and are therefore, literally, limited. However, much information has a variable size: the number of books on a shelf keeps growing, and buildings consist of a variable number of components. To represent such information as data, this chapter introduces another aspect of data analysis: *self-reference*.

Here are some simple examples: • A river

consists of a main river and a tributary – both of which are also rivers. •

A file directory on a computer can contain subdirectories. • A large stack of books consists of a slightly smaller stack of books and another one.

Book.

In all these examples, smaller things are made larger: Two rivers flowing together become one large river. Several files and directories become one large directory. A small stack of books becomes a large one. In programming, we call building large things from small things of the same kind "*combining*." The functions that do this are called *combinators*: This chapter is about them.

5.1 PICTURE RIVERS

[self-service/river.rkt](#)

code

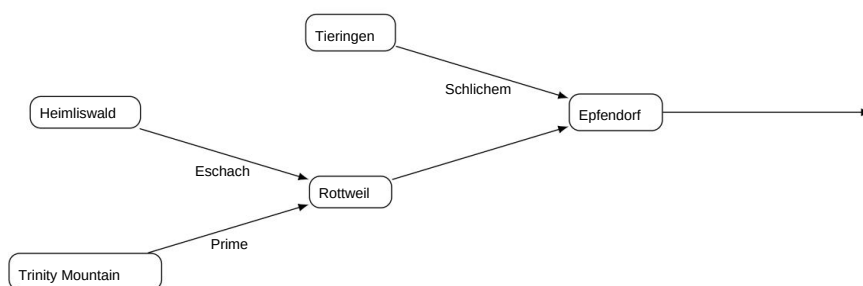


Figure 5.1: The Neckar near its sources

Figure 5.1 is a diagram describing the structure of the Neckar near its sources: It initially flows from the Eschach and Prim streams, and then more and more streams and rivers flow into it – including the Schlichem in the figure.

We will model the structure of a river using data, and then write a function that determines whether a river flows through a specific location. Our first attempt at a data definition looks like this:

A river either originates from:

; - a source

; - a main river and a tributary

One can already see that it is a case distinction, but we can the Sharpen the wording to make it clear that the data is mixed:

A river is one of the following:

; - a stream from a spring

; - a meeting of a main river and a tributary

From this we can directly create a signature definition:

```
(define river
```

```
  (signature (mixed creek confluence)))
```

The data definitions for "stream" and "meeting" are still missing. In Figure 5.1, the streams are still labeled with their names. A suitable data definition would look like this:

A stream has the following characteristics:

; - Place of origin

That sounds a bit odd, because the stream only has *one* property, but the data definition still describes composite data. Nevertheless, it's legitimate and correct – nobody would want to say that the stream *is the source*. Furthermore, you can imagine that a stream might later acquire more properties that we want to record in the data (water quality or flow rate, for example). Accordingly, it also makes sense to write a record definition for this:

```
(define-record creek
```

```
  make-creek
```

```
  Creek?
```

```
  (creek-origin string))
```

If more properties are added later, we can extend the record definition without affecting other code.

Let's move on to the confluences. Here, too, a location is relevant – the place where they meet. It's also important to know *which* rivers or streams converge there. The data definition could look like this:

A meeting consists of:

- ; - Location
- ; - main river
- ; - tributary

The wording clearly indicates that the data is composite. However, this data definition has a peculiarity that only becomes apparent when we consider it in the context of the overall data definition for flows:

A **river** is one of the following:

- ; - ...
- ; - a meeting ...
- ...

A meeting consists of:

- ; - Location
- ; - **main river**
- ; - **tributary**



The data definition for "river" itself uses the term "river". This becomes even more obvious when we create the corresponding record definition:

```
(define-record confluence
  make-confluence
  confluence?
  (confluence-location string) (confluence-
    main-stem river)
  (confluence-tributary river))
```

The definition of confluence uses "river" and vice versa. This isn't a problem – on the contrary, these two *self-referential terms* allow us to describe very different rivers, especially those of different sizes. The section of the Neckar in 5.1 can how we depict it:

```
(define eschach (make-creek "Heimliswald")) ; Source of the Neckar
(define prim (make-creek "Dreifaltigkeitsberg")) ; Source of the Neckar
; first confluence of the Neckar:
(define neckar-1 (make-confluence "Rottweil" eschach prim))
```

; Tributary of the Neckar:

```
(define schlichem (make-creek "Tieringen"))
```

```
; second confluence of the Neckar: (define neckar-2
(make-confluence "Epfendorf" neckar-1 schlichem))
```

These examples clearly show that make-confluence combines two flows into one – it is a *combinator*.

We announced that we would write a function that determines whether water flows from a specific location into a river. We're following the guidelines again, so first, the short description:

Does water flow from a place to a river?

The short description already lists the nouns "river" and "place" as inputs, which we transfer to the signature. Since the function answers a yes/no question, it returns a Boolean value and has a question mark at the end of its name.

```
(: flows-from? (string river -> boolean))
```

During the tests, we make sure to test both streams and rivers, and both cases where `#t` and those where `#f` are the result:

```
(check-expect (flows-from? "Heimliswald" eschach) #t) (check-expect (flows-
from? "Tübingen" eschach) #f) (check-expect (flows-from? "Heimliswald"
neckar-2) #t) (check-expect (flows-from? "Rottweil" neckar-2) #t) (check-expect
(flows-from? "Berlin" neckar-2) #f)
```

The framework, as always, derives directly from the signature:

```
(define flows-from?
  (lambda (location river)
    ...))
```

Since ``river`` involves mixed data, we can add a template for it. Because ``river`` has two cases (streams and confluences), the template must have two branches. The conditions are calls to the respective predicates for ``creek`` and ``confluence``:

```
(define flows-from?
  (lambda (location river)
    (cond
      ((creek? river) ...)
      ((confluence? river) ...))))
```

Now we can add code for the two branches. Let's start with the streams. Since ``creek`` is composite data (with only one component), the selector call should appear in the body of the code:

```
(define flows-from?
  (lambda (location river) (cond

    ((creek? river)
      ...
      (creek-origin river) ...)

    ((confluence? river)
      ...))))
```

The selector ``creek-origin`` returns the source. If this matches the location being searched for, the stream flows through that location; otherwise, it does not. In the first case, the result should be `#t`, in the second, `#f`. This is what it looks like:

```
(define flows-from?
  (lambda (location river)
    (cond
      ((creek? river)
        (if (string=? (creek-origin river) location)
            #t
            #f))
      ((confluence? river)
        ...))))
```

This code can be simplified further: The if expression, to put it simply, says:

If `(string=? (creek-origin river) location)` returns `#t` as a result, then `#t`, and if it returns `#f`, then `#f`.

It is sufficient to write `(string=? (creek-origin river) location)`.

Task 5.1

Can you generalize this simplification to if-expressions and write it as an equation? ỳ

Next, we can address the other case, [Confluence](#). This also involves composite data, so we can already write out the selector calls:

```
(define flows-from? (lambda
  (location river)
  (cond
    ((creek? river) (string=?
      (creek-origin river) location))
    ((confluence? river)
     ...
     (confluence-location river)
     (confluence-main-stem river)
     (confluence-tributary river) ...))))
```

So much for the template; we can now start thinking about the actual task.

The first entry is ([confluence-location river](#)), the point of confluence. If this is the location we're looking for, we can already answer the question of whether the water flows from this point into the river with "yes" or #t :

```
(define flows-from?
  (lambda (location river) (cond
    ((creek? river)
     (string=? (creek-origin river) location))
    ((confluence? river)
     (if (string=? (confluence-location river) location)
         #t
         ...
         (confluence-main-stem river)
         (confluence-tributary river) ...))))))
```

But what if the confluence point isn't the one we're looking for? Then we need to consider the other two selector calls, which provide us with the main and tributaries of the confluence. If either of these contains water from the desired location, then we could also answer our function's question with "yes." To do this, we would need to determine:

1. Does water flow from the point of origin into the main river?
2. Does water flow from the point of origin into the tributary?

Well, we're currently writing a function that determines whether water flows from a specific location into a specific river. Can we use that? Let's write it down:

```
(define flows-from?
  (lambda (location river) (cond

    ((creek? river)
     (string=? (creek-origin river) location))
    ((confluence? river)
     (if (string=? (confluence-location river) location)
         #t
         ...
         (flows-from? location (confluence-main-stem river))
         (flows-from? location (confluence-tributary river)) ...))))))
```

The two calls to `flows-from?` correspond precisely to the two questions above. We can combine their results to calculate our answer: If water from `location` flows through the main river, or if water from `location` flows into a tributary, then water from `location` also flows into the "total river". Here's how it looks:

```
(define flows-from?
  (lambda (location river) (cond

    ((creek? river)
     (string=? (creek-origin river) location))
    ((confluence? river)
     (if (string=? (confluence-location river) location)
         #t
         (or
          (flows-from? location (confluence-main-stem river))
          (flows-from? location (confluence-tributary river)))))))
```

Done! This function does something special that wasn't present in the previous functions: it contains a call to itself, a so-called *recursive call*. These recursive calls occur precisely where the data definition of `river` contains self-references.

This is no coincidence: Almost all functions that accept data with self-references contain recursive calls at the points of self-reference. From this we create a template:

Construction instructions 17 (Self-references as input: Template)

If you are writing a function that accepts data containing self-references, then write a recursive call at each of the self-references.

One more thing: The if expression in the `flows-from?` function *almost* matches the pattern from exercise 5.1. The only difference is that the alternative of the branch does not contain `#f`. In general, such an if-expression is evaluated as follows:

```
(if b #t) y #t ;
if b = #t y a ; if b = #f
```

We can compare this to the behavior of `or` :

```
(or ba)
y #t ; if b y #t y a ; if b y #f
```

We can therefore replace the if expression in `flows-from?` with an `or` :

```
(or (string=? (confluence-location river) location)
    (or (flows-from? location (confluence-main-stem river))
        (flows-from? location (confluence-tributary river))))
```

We can simplify this even further, because `or` also works with three operands:

```
(or (string=? (confluence-location river) location)
    (flows-from? location (confluence-main-stem river)) (flows-from? location
    (confluence-tributary river)))
```

If we interpret the result for its meaning, it states the following: Water flows from `location` into the confluence `river` when... • the confluence occurs

precisely at `location`.

or

- Water from `location` flows into the main river **or**
- Water from the `location` flows into the tributary.

Is the functionality of `flows-from` explained in this way? (hopefully) immediately understandable.

From this example and [exercise 5.1](#), we can create two equations according to Mantra 6 on page 46 :

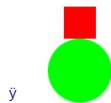
```
(if b #t #f) = b
```

```
(if b #ta) = (or ba)
```

5.2 MODELING IMAGES

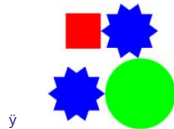
Do you remember section 1.4.2 on page 23? There, we used the `beside` and `above` functions of the `image.rkt` teachpack to combine images:

```
(define s1 (square 40 "solid" "red")) (define c1 (circle
40 "solid" "green")) (define p1 (star-polygon 20 10 3
"solid" "blue")) (beside s1 p1)
```



With both functions, it was possible to reuse the results as images:

```
(above (beside s1 p1) (beside p1 c1))
```



The `image` signature is responsible for images ; with its help we can formulate signatures for `above` and `beside` :

```
(: beside (image image -> image)) (: above
(image image -> image))
```

The signatures clearly indicate that these are combinators. We can indirectly conclude from this that there are also self-references in the data definition of images. It could look like this:

; One image is one of the following:

; ...

; - a juxtaposition of one image and another image; - a superimposition of one image and another image

; ...

This means that the same construction principle is at work with the images as with the rivers: "small" images become "larger" ones, and from these, possibly even larger ones, and so on.

The signatures of ``beside`` and ``above`` clearly show that these are combinators: ``image`` goes in and ``image`` comes out again. To clearly indicate this for the flows from the last section, it would be useful to also specify an explicit signature for ``make-confluence``:

```
(: make-confluence (string river river -> river))
```

Combinators are hidden in many problems; you just have to look for them: then you'll surprisingly often find some. That's what Mantra 8 is about:

Mantra 8

Look for combinators for your data that make bigger things out of small things – and then make even bigger things out of those.

5.3 Displaying Financial Contracts

[self-payment/contract.rkt](#)

code

It's time for a slightly larger example. We'll model financial contracts and program a highly simplified version of a classic computer science research project that has led to several publications and the founding of a successful company.¹ Self-references play an important role.

Important to note about this example: Renowned computer science researchers spent several months developing the representation of financial contracts from this section. We can recreate them using the programming tools presented so far, but you shouldn't expect to have been able to develop the example yourself in just a few hours.

¹ Anyone who wants to know more should read the paper by Peyton Jones and Eber [PE03], which is also available for free download on the internet.

In the financial sector, contracts are often concluded for the structured payment of Managing sums of money. If you ask a banker what the simplest such contract is, you often get... The answer is a *zero-bond*. Here is an example of a zero-bond:

I will receive €1000 on June 31, 2030.

Such a contract is always concluded between two contracting parties, and one is always "I." Hence the phrase "I receive." The term "zero bond" is composed of "no interest" (zero) and "right to later payment" (bond).

We can make this into a simple data definition:

A zero-coupon bond has the following properties:

; - Date

; - Amount

; - Currency

The phrase "has the following properties" immediately indicates that we are dealing with composite data. This directly leads to the following record definition:

```
(define-record zero-bond
  make-zero-bond
  (zero-bond-date date)
  (zero-bond-amount rational)
  (zero-bond-currency currency))
```

We still need to contribute definitions for date and currency, but we'll postpone that until later. Long time in the bank. Instead, we want to first fundamentally rethink the correct data definition for such contracts.

There are many more such contracts: futures, forwards, swaps, swaptions... Every contract It has several properties, so this would mean that we would have to write a lot of data definitions for composite data and the associated record definitions – and finally a data definition for "contract" that refers to a large signature for mixed data That amounts to a lot of work, and it would never be finished because the bankers are constantly devise new contracts.

However, anyone who studies enough of these contracts will notice that they consist of recurring building blocks. This suggests a different approach: identifying these building blocks. and to perform the data analysis. Instead, we are doing the work of the authors of the Papers [PE03] follow. They have (over several months) compiled a set of strikingly simple building blocks by breaking down well-known contracts into the smallest and simplest possible components.

The components are broken down. Each of these components is, in itself, a "small contract," but there are also combinators that create larger ones from small contracts, especially for previously unknown classes of contracts. Using these components and combinators, a large number of financial contracts (including unknown ones) can be modeled.

Since there are several classes of building blocks and combinators, we will refer to contracts as *ge-*represent mixed data and gradually build a data definition for it:

A contract is one of the following:

```
;- ...
```

This naturally includes a suitable signature definition, which we will also build up step by step:

```
(define contract
  (signature (mixed ...)))
```

To identify the building blocks and combinations, let's first take a closer look at the zero-coupon bond.

Although it appears simple, there are several aspects that make sense independently: the payment of a specific *amount* in a specific currency and the option to initiate a payment *at a later date*. These are three separate ideas, and it makes sense to represent them separately as data.

Payment in a specific currency . Let's start with a payment in a specific currency. For simplicity, we'll assume all payments are in euros. To find the correct data definition, we'll try to find *the simplest* euro payment, which consists of *one* euro. The data definition for this is surprisingly simple:

A euro has no inherent properties.

We could simply represent the euro as the string "EUR". This would make sense if the euro were part of an enumeration. However, this is not the case here; other contract elements do indeed have properties. (See also exercise 5.13 on page 172.) Instead, we take the data definition from the word "has... properties" and represent the euro as composite data. This results in the following record definition:

```
(define-record one-euro
  make-one-euro
  one-euro?)
```

Next, we will add *one-euro* to the data and signature definition of *contract* :

A contract is one of the following:

; - one euro

; - ...

(define contract

(signature (mixed one-euro ...)))

Arbitrary Amounts Next, we will represent the possibility of choosing an amount. An obvious first idea would be to represent the idea of "n euros", something like this:

Euros have the following property: - how many (define-record euros)

make-euros

euros?

(euros-amount rational))

If we had that, we wouldn't need "one-euro" anymore. Furthermore, the result wouldn't be as simple as possible: It would be better if we could separate the idea of "how many" from "euro." Then we could say "a multitude of one euro." That sounds strange at first:

A variety has the following properties:

; - how many

; - of which

But it already has the correct form for composite data. It's sufficient for a first sketch of a record definition:

(define-record multiple make-

multiple multiple?

(multiple-

number ...) (multiple-of ...))

However, we still need to clarify the signature of `multiple-number` and `multiple-of`. With `multiple-number`, we don't have to restrict ourselves to whole numbers – half a euro works too. That makes *rational* sense.

With `multiple-of`, it makes sense to write "one-euro": then we could already write "zero bond". However, "multiple" would be quite limited and not particularly...

Future-proof: What if other currencies are added, like [one GBP](#) ? We might be tempted to define a signature [currency](#) for currencies and use it for [multiple transactions](#) . But that would still be too restrictive: You can also transfer multiple shares (for example) within a single contract – or multiple zero-coupon bonds or other contracts.

And there it is, quite suddenly, the central idea: that a contract can mandate the conclusion of *other* contracts. Therefore, we simply write "contract" as the signature of "multiple-of" and, in doing so, refine the data definition:

A multiple consists of:

; - Number

; - Contract

(define-record multiple make-

multiple multiple?

(multiple-

number rational) (multiple-of contract))

There are now two classes of contracts – [one-euro](#) and [multiple](#). We are expanding the definition of [contract](#) accordingly:

(define contract

(signature (mixed one-euro multiple)))

Using [one-euro](#) and [multiple](#) , we can define a contract that stipulates we will receive 100 euros immediately:

(define euro100 (make-multiple 100 (make-one-euro))) ; 100 euros

Task 5.2

Define a function ``make-euros`` with a short description, signature, and test as follows:

Pay out the Euro amount (: make-

euros (rational -> contract)) (check-expect (make-euros

100) euro100)

Ÿ

Using the function from the problem, we can easily define e 200 as well:

(define euro200 (make-euros 200)) ; 200 euros

Delays: For zero-coupon bonds, we still need the option to delay a payment. We use the same trick as with [multiple bonds](#) and define another [missing information].

Combinator that delays an entire contract: So we conclude a contract, later a

To conclude a contract, which can then trigger the payment. To describe such a contract, we need the date on which the "later" contract becomes active and the subsequent contract. himself:

A delay consists of:

; - Date

; - Contract that becomes valid on that date

This data definition describes composite data. The corresponding record definition looks like this:

```
(define-record later
  make-later
  later?
  (later-date      date)
  (later-contract contract))
```

With [later](#) , we can extend the signature definition of [contract](#) :

```
(define contract
  (signature (mixed one-euro multiple later)))
```

But something is still missing: We used the signature [date](#) for [later-date](#) , which still is not defined at all. A date consists of year, month, and day, which leads directly to a record definition:

```
(define-record date
  make-date
  date?
  (date-year natural)
  (date-month natural)
  (date-day natural))
```

Task 5.3

We used the signature "natural" for all fields of the [date function](#) . This signature is very imprecise, at least for [date-month](#) and [date-day](#).

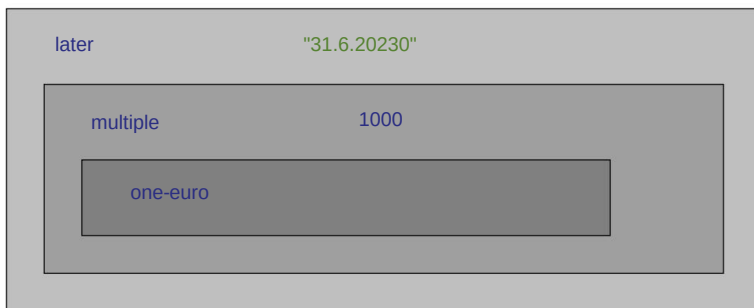


Figure 5.2: Zero-Bond represented as layers

Improve the signatures of `date-month` and `date-day` using `integer-from-to`.
from section 2.2 on page 55!

Ÿ

Before we continue, let's define two examples of the date:

```
(define date1 (make-date 2019 07 26)) ; July 26, 2019 (define date2 (make-date
2019 12 24)) ; Christmas 2019
```

Using these definitions, we can finally give two examples for `later` :

```
(define later1
  (make-later date1 euro100)) ; I will receive 100 euros on July 26, 2019 .
(define later2
  (make-later date2 euro200)) ; I will receive 200 euros for Christmas 2019 .
```

Furthermore, we can define the zero bond from the beginning of this section:

```
I will receive 1000e on June 31, 2030. ( define zero1 (make-
later (make-date 2030 06 31)
      (make-euros 1000)))
```

Figure 5.2 shows the structure of the zero-bond `zero1` as nested layers: A multiple contract is nested within the later contract , and the `one-euro` contract is nested within that. From this perspective, the multiple contract is the "inner" contract of the later contract.

Combining Contracts Now that we've covered zero-bonds, we can consider whether there are other ways to create contracts. Whenever you're building representations with combinators, you should look for ways to build something bigger from two things.

We have already seen several examples of this principle in this chapter:

- A main river and a tributary are combined to form one river.
- Two images are combined above, next to, and on top of each other to create one image.

In terms of contracts, this means that we combine two contracts into one that combines the rights and obligations of both. This could work as follows:

A combination contract consists of: - Contract No. 1 -
Contract No. 2

These are again composite data with two self-references:

```
(define-record both
  make both
  Both?
  (both-contract-1 contract)
  (both-contract-2 contract))
```

Here are two examples of `both`:

```
; Today 100s , later another 100s (define both1
(make-both euro100 later1))
; Today 200s , later another 200s (define both2
(make-both both1 later2))
```

We still need to expand the definition of `contract` :

```
(define contract
  (signature (mixed one-euro multiple later both)))
```

The Smallest Contract! There is another principle when representing with combinators: always look for the *smallest* possible building block. We have already defined a rather small building block: `one euro`. But often the small building block is a variation of "nothing," and that is also the case here: the smallest contract is one without rights or obligations. We should definitely define it as well—initially only for the sake of completeness, but we will actually need it later.

Such a "no-contract," like `one-euro` , has no properties. However, just as with `one-euro`, we represent it with a record so that we can incorporate it into the definition of a `contract` using its signature .

A nothing contract has no properties (define-record nothing
make-nothing nothing?)

Accordingly, we are expanding the contract:

```
(define contract
  (signature
    (mixed one-euro multiple later both nothing)))
```

Exercise 5.11 on page 172 deals with yet another combinator.

Contract evaluation: We've put a lot of energy into representing contracts, but haven't done anything with them yet. We're still missing some juicy features!

If we have a contract, we might want to know how much money we'll actually receive in total. We'll tackle that as our first task. We'll start with the following brief description:

Calculate how much money we will receive for a contract

The short description contains the terms "contract" and "money", which we translate into the following signature:

```
(: contract-payment (contract -> rational))
```

The function summarizes all payments into a single euro amount, ignoring interest. As always, we illustrate how it works with test cases. For fixed euro amounts, the expected output is quite clear:

```
(check-expect (contract-payment euro100) 100) (check-expect
(contract-payment euro200) 200)
```

Examples later1 and later2 are delayed contracts for e 100 and e 200 respectively.

The delay is irrelevant for contract-payment :

```
(check-expect (contract-payment later1) 100) (check-expect
(contract-payment later2) 200)
```

In both "both" examples, two payments of e 100 and e 200 respectively are combined:

```
(check-expect (contract-payment both1) 200) (check-expect
(contract-payment both2) 400)
```

The template looks like this:

```
(define contract-payment
  (lambda (contract)
    ...))
```

Since the definition of `contract` involves mixed data, the first template for `contract-payment` looks like this:

```
(define contract-payment (lambda
  (contract) (cond

    ((nothing? contract) ...) ((one-euro?
  contract) ...) ((multiple? contract) ...)

    ((later? contract) ...)
    ((both? contract) ...))))
```

The individual cases of `contract` each involve composite data, so we can add the applications of the selectors to the template, insofar as there are any:

```
(define contract-payment (lambda
  (contract)
    (cond
      ((nothing? contract) ...) ((one-euro?
  contract) ...)
      (multiple contract)
      ...
      (multiple-number contract) (multiple-
  of contract) ...) ((later? contract)

      ...
      (later-date contract) (later-
  contract contract)
      ...)
      ((both? contract)
      ...
      (both-contract-1 contract)
```

```
(both-contract-2 contract)
...)))))
```

It goes even further:

(multiple-of contract), (later-contract contract), (both-contract-1 contract) and (both-contract-2 contract) are all self-references, so we can write recursive calls there:

```
(define contract-payment (lambda
  (contract)
    (cond
      ((nothing? contract) ...) ((one-euro?
contract) ...)
      (multiple contract)
      ...
      (multiple-number contract) (contract-
payment (multiple-of contract)) ...)

      ((later? contract)
       ...
       (later-date contract)
       (contract-payment (later-contract contract)) ...)

      ((both? contract)
       ...
       (contract-payment (both-contract-1 contract)) (contract-payment
(both-contract-2 contract)) ...)))))
```

The template is now complete, and we can try to write something meaningful for each case. The first two aren't too difficult: a "nothing" contract means no payment is made, while a "one-euro" contract means one euro is paid out.

```
(define contract-payment (lambda
  (contract) (cond

    ((nothing? contract) 0)
```

```
((one-euro contract) 1)
...)))
```

Next, we'll look at the **multiple case** : The template already contains: • **(multiple-number contract)**, i.e., the *number of contracts* • **(contract-payment (multiple-of contract))**, i.e., what is *paid out in one contract*.

We need to multiply these two amounts together to calculate the total sum that will be paid out:

```
(define contract-payment (lambda
  (contract) (cond
    ...
    ((multiple? contract) (*
      (multiple-number contract) (contract-
        payment (multiple-of contract)))) ...)))
```

Now we come to the later case. For **contract-payment**, the **(later-date contract)**, i.e., the point in time at which the contract becomes active, is irrelevant: It's only about the amount of the "inner" contract, and that's already in the template.

```
(define contract-payment
  (lambda (contract)
    (cond
      ...
      ((later? contract)
        (contract-payment (later-contract contract))) ...)))
```

Finally, we come to **"both"**. The payments resulting from the two sub-contracts are already in the template. We simply need to add them together. The complete function then looks like this.

out of:

```
(define contract-payment
  (lambda (contract)
    (cond
      ((nothing? contract) 0) ((one-
        euro? contract) 1)
```

```

((multiple? contract) (*
  (multiple-number contract) (contract-
    payment (multiple-of contract))))
((later? contract)
  (contract-payment (later-contract contract))) ((both? contract)

  (+ (contract-payment (both-contract-1 contract))
    (contract-payment (both-contract-2 contract)))))

```

Managing Contracts: Another useful function of contracts concerns their evolution over time. The longer a contract is in place, the more payments are made, and consequently, fewer payments remain. To represent this, we need a function that calculates what remains of a contract after a specific point in time. A brief description could be:

What remains after payment has been made on a given date?

To write this function, we first need to consider how this "remainder" of a contract can be represented. The correct idea is so simple that it's not so easy to see: We can represent the "remainder of a contract" as a contract again. Therefore, we don't need to perform any new data analysis and can proceed with the signature. The function requires a contract as input; the output is the new contract. We also need the date mentioned in the short description.

```
(: contract-rest (contract date -> contract))
```

Let's look at some test cases: The `euro100` and `euro200` contracts are paid out immediately, so there's nothing left over:

```
(check-expect (contract-rest euro100 date1) (make-nothing)) (check-expect (contract-
rest euro200 date1) (make-nothing))
```

The example `later1` will be paid out on `date1` (July 26, 2019), so from this date onwards there is nothing left:

```
(check-expect (contract-rest later1 date1) (make-nothing))
```

The example contract `later2`, however, will only be paid out on `date2`, which is Christmas 2019.² Therefore, nothing happens at `date1`; the contract is returned unchanged.

² When this chapter was written, that was still in the future.

```
(check-expect (contract-rest later2 date1) later2)
```

Things get interesting with both contracts. With `both1`, one part happens immediately, the other is delayed until `date1`. However, by `date1`, nothing remains of either half.

```
(check-expect (contract-rest both1 date1) (make-nothing))
```

However, in `both2`, the "right" half remains at `date1` `later2`:

```
(check-expect (contract-rest both2 date1) later2)
```

But at `date2`, that too is lost:

```
(check-expect (contract-rest both2 date2) (make-nothing))
```

We now move on to the function itself. Here is the framework:

```
(define contract-rest
  (lambda (contract date)
    ...))
```

The input signature contains a `contract`, so we can again use the mixed template. Use data:

```
(define contract-rest
  (lambda (contract date) (cond

    ((nothing? contract) ...) ((one-euro?
    contract) ...) ((multiple? contract) ...)

    ((later? contract) ...)
    ((both? contract) ...))))
```

Next, we will add the templates for the individual cases, which each involve composite data:

```
(define contract-rest
  (lambda (contract date) (cond

    ((nothing? contract) ...) ((one-euro?
    contract) ...) ((multiple? contract)

    ...
```

```

(multiple-number contract) (multiple-
of contract) ...)

((later? contract)
...
(later-date contract)
(later-contract contract)
...)
((both? contract)
...
(both-contract-1 contract) (both-
contract-2 contract)
...)))

```

We almost forgot – in the cases for `multiple`, `later` and `both` there are self-references and thus recursive calls:

```

(define contract-rest
  (lambda (contract date)
    (cond
      ((nothing? contract) ...)
      ((one-euro contract) ...)
      (multiple contract)
      ...
      (multiple-number contract) (contract-
rest (multiple-of contract) date) ...)

    ((later? contract)
      ...
      (later-date contract)
      (contract-rest (later-contract contract) date)
      ...)
    ((both? contract)
      ...
      (contract-rest (both-contract-1 contract) date) (contract-rest (both-
contract-2 contract) date)
      ...))

```

```
...
(date-year date) (date-
month date)
(date-day date) ...))
```

We can also use the template for ``date``, which deals with compound dates. The date is only relevant in the ``later`` case – we fill in the template. when we get to this case.

We'll start again with the simple cases. With `"nothing"` and `"one-euro"`, there's nothing left over in either case.

```
(define contract-rest
  (lambda (contract date) (cond

    ((nothing? contract) (make-nothing)) ((one-euro?
contract) (make-nothing) ...)))
```

Task 5.4

Are both cases already covered by test cases? If not, write one!

Ÿ

Next up is `multiple`. The recursive call from the template answers the following question: "How much of a contract remains after a given date?" If we denote the result of ``multiple-number contract`` by ``n``, then we need to answer the question of how many of `n` contracts remain. We need to multiply by `n` again:

```
(define contract-rest
  (lambda (contract date)
    (cond
      ...
      (multiple contract)
      (make-multiple (multiple-number contract)
                     (contract-rest (multiple-of contract) date)))
    ...)))
```

Next, we'll look at the later case. The recursive call answers the question of what remains of the contract if the date `has` already passed or if it is today.

If the `date` is still in the future, nothing happens and the function should return the entire `contract` .

This means the `date` splits into two classes: (a) before or on the `date` (`later-date contract`) or (b) after. We therefore need a branch after the date, and to do this, we must compare the two dates. This task is complicated enough that we will postpone it and plan to write a function later with the following short description and signature:

Is one date earlier than another?

```
(: date<=? (date date -> boolean))
```

Using this function, we can complete the later case as follows:

```
(define contract-rest
  (lambda (contract date)
    (cond
      ...
      ((later? contract)
       (if (date<=? (later-date contract) date)
           (contract-rest (later-contract contract) date)
           contract)
       ...)))
```

We cheated a little and avoided filling out the template for "`date`" .

We need to make up for that with `date<=?`.

The case remains for `both`. The recursive calls from the template deliver what is left of the "left" or "right" contract. We just need to process the results again.
to assemble:

```
(define contract-rest
  (lambda (contract date) (cond
    ...
    ((both? contract)
     (make-both (contract-rest (both-contract-1 contract) date)
                (contract-rest (both-contract-2 contract) date))))))
```

Complete!

Almost there: We still need to write the function `date<=?` . For part of that, we'll span...
we invite you:

Task 5.5

Write sufficient test cases for `date<=?!`

ÿ

Here is the framework:

```
(define date<=?
  (lambda (date1 date2)
    ...))
```

That both `date1` and `date2` are composite data.

```
(define date<=?
  (lambda (date1 date2)
    ...
    (date-year date1) (date-year date2) (date-month date1)
    (date-month date2)
    (date-day date1) (date-day date2) ...))
```

To get the hull right, we need to carefully work through all possible scenarios.

First, we need to compare the two years – if they differ between `date1` and `date2`, the result is clear. If not, the function must compare the months and finally the days. The result looks like this:

```
(define date<=?
  (lambda (date1 date2)
    (cond
      ((< (date-year date1) (date-year date2)) #t) ((> (date-year date1)
      (date-year date2)) #f)
      ((< (date-month date1) (date-month date2)) #t)
      ((> (date-month date1) (date-month date2)) #f) ((< (date-day date1)
      (date-day date2)) #t) ((> (date-day date1) (date-day date2)) #f)
      (else #t))))
```

Simplifying contracts. Unfortunately, while the code for the ``contract-rest`` function is complete, it doesn't work as expected: The tests fail, almost all of them! How can that be? Here's the first one:

```
(check-expect (contract-rest euro100 date1) (make-nothing))
```

Here is the failure report:

The actual value `#<record:multiple 100 #<record:nothing>>` is not the expected value `#<record:nothing>`.

What happened? As a reminder, the `euro100` contract is defined as follows:

```
(define euro100 (make-multiple 100 (make-one-euro)))
```

The ``contract-rest`` function first calculated what was left of the one euro and then returned 100 units of the remainder: 100 units of nothing. The result is formally correct, but not as expected by the test case. It's certainly impractical.

Task 5.6

Find three more complicated ways to write "contracts about nothing".

Ÿ

To avoid this problem, it would be good if the program didn't construct contracts for "n times nothing" in the first place. For this, we need to address the ``make-multiple`` constructor. If it's applied to a ``nothing`` contract, the result must also be a ``nothing`` contract. To achieve this, we need to redefine ``make-multiple``. Since the definition of ``make-multiple`` is handled by ``define-record``, we use a trick and first rename the constructor by changing the record definition:

```
(define-record multiple really-
  make-multiple multiple?
  (multiple-
    number rational) (multiple-of
                        contract))
```

The name ``really-make-multiple`` is purely a matter of convention—as long as the name is different from ``make-multiple``! Next, we define a new function with the old name and the same contract. Here is a short description, signature, and a test case that just matches ``euro100``:

```
; construct a multiple contract, simplifying it in the process (: make-
multiple (rational contract -> contract))
```

```
(check-expect (make-multiple 100 (make-nothing))
  (make-nothing))
```

Here is the framework for the function:

```
(define make-multiple (lambda
  (factor contract)
  ...))
```

The input ``contract`` splits into two cases: ``nothing`` and all others. In the first case, we return ``nothing`` again ; in the second, we call the correct constructor.

```
(define make-multiple (lambda
  (factor contract) (if (nothing? contract)
    (make-nothing) (really-make-
      multiple factor
      contract))))
```

So the first two test cases, which originally failed, are now working. However, there are still more. The first one is this one:

The actual value

```
#<record:both #<record:nothing> #<record:nothing>>
```

is not the expected value

```
#<record:nothing>.
```

This suggests a problem analogous to that of ``multiple`` : Nothing and nothing again is indeed nothing, but the constructor of ``both`` doesn't know this. We apply the same trick and first rename the constructor:

```
(define-record both
  really-make-both
  Both?
  (both-contract-1 contract)
  (both-contract-2 contract))
```

Next, we will define a new constructor ourselves:

```
; Construct a combination contract, simplifying it in the process (: make-both
(contract contract -> contract))
```

```
(check-expect (make-both (make-nothing) (make-nothing)) (make-nothing))
```

This test case covers "nothing and nothing." However, we can generalize this further. The following test cases best illustrate this, where only the left or right side of the both contract is "nothing":

```
(check-expect (make-both (make-one-euro) (make-nothing)) (make-one-euro))
```

```
(check-expect (make-both (make-nothing) (make-one-euro)) (make-one-euro))
```

Here is the framework for its function:

```
(define make-both
  (lambda (contract-1 contract-2)
    ...))
```

The inputs fall into three classes: if `contract-1` is nothing, if `contract-2` is nothing, and all others. Accordingly, we need a branching statement with three branches:

```
(define make-both
  (lambda (contract-1 contract-2)
    (cond
      ((nothing? contract-1) contract-2) ((nothing?
        contract-2) contract-1) (else (really-make-both
          contract-1 contract-2))))))
```

And now, finally, all the tests are successful.

The functions `make-multiple` and `make-both`, which replace the original constructors of the same name and simplify the resulting contracts during construction, are also called *smart constructors*.

Finding Combinator Representations: The authors of the paper on financial contracts spent months finding the elegant representation described. They proceeded according to two principles that are also applicable to many other problems. • They decomposed financial contracts into their smallest components. • They searched for and found combinators for financial contracts, as described in Mantra 8 on page 46.

TASKS

Task 5.7

Write down functions for handling shower products!

1. Start with two different shower products: soap and shampoo. For soap, color and pH value are relevant; for shampoo, color and hair type are important. Write a data definition for shower products and the corresponding code!
2. Add shower gel as another shower product, made up of half any soap and half any shampoo. (We'll just pretend that's the case.)
3. Add a "mixture" as another shower product, which is made by mixing any two shower products together, with selectable proportions for the first and second shower products.
4. Write a function that calculates the soap content of any shower product.

Task 5.8

A geometric figure in the two-dimensional plane is either • a *square*

parallel to the coordinate axes,

• a *circle*

• or a *superimposition* of two geometric figures. (The figures are placed on top of each other; the areas of the two figures are thus combined.)

Perform a data analysis for geometric figures! Write a function that checks whether a given point lies within the area of a given geometric figure.

If you need a function that takes a square root for this task (it doesn't have to be) (which is mandatory), so it has the name `sqrt` (for «square root») and the following signature:

`(: sqrt (number -> number))`

Task 5.9

Write a sensible *smart constructor* for later contracts.

Task 5.10

Can the following contract be simplified? Extend the corresponding *smart constructor*:

`(make-multiple 100 (make-multiple 100 (make-one-euro)))`

Task 5.11

Add another class of contracts, give contracts: A give contract reverses all payments resulting from a contract. That is, a give contract of e 1 results in...

The contract holder must *give* the contracting party e1 but does not receive it. Extend the two Features so they can also handle give contracts. Use to transfer payments "into the To represent a different direction, a negative sign.

Task 5.12

The functions `contract-payment` and `contract-rest` don't really fit together:

One function calculates *all* payments, the other the remainder of a contract from a specific point in time. Write a function ``contract-payment-until`` that calculates all payments up to a specific point in time!

Task 5.13

Instead of zero-digit composite data like `nothing` or `one-euro`, one could theoretically

Also, as with lists, use fixed strings such as `"nothing"` and `"one-euro"` .

Try to implement this!

6. Programming with Lists

In the previous chapter, we already encountered self-referential data that can represent information of variable size: flows, images, and financial contracts. The representations for these were quite specific to their respective areas of application. In this chapter, we will learn about a especially practical data definitions with self-reference, which are used in almost all areas of application *Lists* are useful .

Here are some lists from everyday life:

Bread	Herbert	1	Axl	Perm	pumps
Butter	Mike	2	Slash	Dreadlocks	
Cheese		3	Duff	Mohawk	
		4	Dizzy	mullet	
		5	Izzy		
		6	Melissa		
			Brain		
			Bumblefoot		

None of these lists are fixed in length: For example, "cucumbers" could be added to the list containing "bread", and "Axl", "Slash", and so on could be extended to include, for example... Steven will join us.

6.1 LISTS REPRESENT

`listen/list0.rkt`

code

In the introduction, we saw lists with different types of elements. For To begin with, we will focus on lists of numbers to simplify things. to hold. Then we write a function to add the numbers from such a list. It There are various ways to represent lists. The representation we will discuss in this... Introducing chapters has proven to be particularly simple and universally applicable.

This representation was created according to the same principle as that for financial contracts in section 5.3 on page 150: We first look for the simplest list we can imagine. One might think that this is a list with only one element, but even more A simpler option is one with no elements at all – the *empty list*. Of course, we will also include non-empty lists . Lists are needed, therefore the first draft of a data definition is as follows:

A list of numbers is one of the following:

- ; - an empty list
- ; - a non-empty list

The empty list has no components, but we will need to distinguish it from non-empty lists later. Therefore, we use an empty record definition:

```
; empty list (define-
record empty-list make-empty-list
  empty?)
```

We've cheated a little with the name for the predicate. Officially, it should be called "empty-list?" . However, since it will appear quite frequently, we're shortening it to "empty?" .

We also define an abbreviation for the empty list. This will also come up again. before, and we don't have to reconstruct them every time:

```
(define empty (make-empty-list))
```

Now we turn to non-empty lists. Following the combinator principle from the previous chapter, we should construct a non-empty list by extending an existing list:

A non-empty list consists of:

- ; - the first element
- ; - a list of numbers with the remaining elements

This specific definition for non-empty lists is called a *Cons list*. The second field contains a "list of numbers"—that's where the self-reference is.

We are changing the data definition for lists accordingly:

A list of numbers is one of the following:

- ; - an empty list
- ; - a cons list

A cons list consists of:

- ; - the first element
- ; - a list of numbers with the remaining elements

We convert the data definition for Cons into a record definition. For this, we assume that the signature for lists of numbers is called `list-of-numbers` – we will define this later:

```
(define-record cons-list
  cons
  cons?
  (first number)
  (rest list-of-numbers))
```

Similar to `empty-list`, we've given the functions of the record definition shorter names than usual because they will appear frequently: otherwise: The constructor is called `cons` here instead of `make-cons-list`, the predicate is just `cons?` instead of `cons-list?` and the selectors are missing the `cons-` front.

The definition of the signature "list-of-numbers" for "list of numbers" is still missing.

Empty lists and cons lists are mixed:

```
(define list-of-numbers (signature
  (mixed empty-
    list cons-list)))
```

Here are some examples of lists:

```
; empty list (define
list0 empty) ; single-element
list containing the number 42 (define list1 (cons 42
empty))
List with the numbers 1 2 3
(define list3 (cons 1 (cons 2 (cons 3 empty))))
; List containing the numbers e and pi (define
list2 (cons 2.7183 (cons 3.14159 empty)))
; List with numbers 2 3 5 7 (define list4 (cons
2 (cons 3 (cons 5 (cons 7 empty)))))
```

The examples illustrate a difference between the intuitive structure of lists and their representation. Intuitively, a list consists of a sequence of its elements. However, the representation is nested, as shown in Figure 6.1. This allows us, in particular, to build larger lists from smaller ones and still reuse the smaller lists.

```
List with the numbers 1 2 3 5 7
(define list5 (cons 1 list4))
```

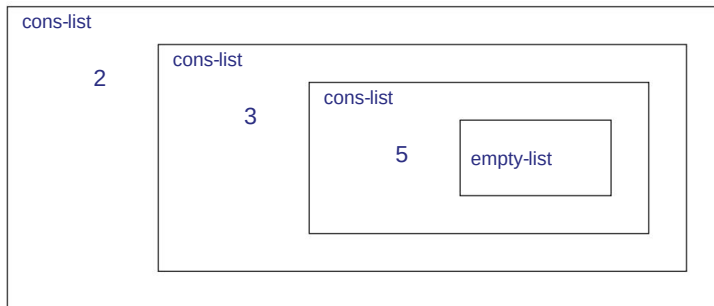


Figure 6.1: Structure of the list with elements 2 3 5

This example also shows that the constructor `cons` appends an element to *the front* of an existing list .

Now we are ready to write a function on lists that adds up the numbers in a list. The short description and signature look like this:

Calculate the sum of the elements in a list of numbers

(: list-sum (list-of-numbers -> number))

Next, we need test cases, which we will construct using the example lists from above (and possibly a calculator):

```
(check-expect (list-sum list2) 5.85989) (check-expect (list-sum
list3) 6) (check-expect (list-sum list4) 17)
```

Task 6.1

Write test cases for ``list-sum`` based on the example lists ``list0`` and ``list1``! If you're unsure, guess! ÿ

Next, as always, comes the scaffolding:

```
(define list-sum
  (lambda (list)
    ...))
```

The signature states that ``list-sum`` accepts a list as input – mixed data. The template for this looks like this:

```
(define list-sum
  (lambda (list) (cond

    ((empty? list) ...) ((cons?
    list) ...))))
```

The ``cons-list`` case involves composite data, so we can write selector calls according to the template. The ``rest`` case also contains a self-reference, so we also write a recursive call in the template:

```
(define list-sum (lambda
  (list)
    (cond
      ((empty? list) ...) ((cons? list)
      ...
      (first list) (list-sum
      (rest list)
      ...))))
```

Now all possible construction instructions have been applied. We complete the template, starting with the case for cons lists. The template contains `(first list)` and `(list-sum (rest list))`.

To understand this, it helps to translate these two expressions into English:

- `(first list)` is the *first element of the list* and
- `(list-sum (rest list))` is the *sum of the remaining elements*.

But what we need is the sum *of all elements*; to find that, we have to add the two:

```
(+ (first list) (list-sum (rest list)))
```

The case of the empty list remains. Intuitively, it has no sum at all, but we have to write something there. For now, we'll write 0 into the program, but we'll discuss this choice further.

```
(define list-sum
  (lambda (list) (cond

    ((empty? list) 0) ((cons?
    list)
      (+ (first list) (list-sum (rest list))))))
```

The test cases now work and the function is complete!

Task 6.2

Follow the evaluation of the test cases in the Stepper!

Ÿ

Task 6.3

Change the branch for the empty list and enter something other than 0. Do the test cases still work then?

Ÿ

In fact, 0 is the only correct number that can appear in the empty case. This might seem intuitive: after all, the empty list is "nothing," and 0 is also somehow "nothing." However, another example will soon show that this argument is too simplistic.

Next, we will write a function that *multiplies all the numbers in a list*. Here are Brief description, signature, tests and framework:

Calculate the product of the elements of a list of numbers (: list-product (list-of-numbers -> number))

```
(check-expect (list-product list1) 42) (check-expect (list-product list3) 6) (check-expect (list-product list4) 210)
```

```
(define list-product (lambda
  (list) ...))
```

The template is created exactly as with [list-sum](#), from the construction instructions for mixed Data, composite data and self-references:

```
(define list-product (lambda
  (list)
    (cond
      ((empty? list) ...) ((cons?
        list)
          ...
          (first list)
          ...
          (list-product (rest list))))))
```

We'll start with the cons case again. The template contains `(first list)`, the first element, and `(list-product (rest list))`, the product of the remaining elements. We simply need to multiply these two together to calculate the product of all elements:

```
(* (first list) (list-product (rest list)))
```

The empty case remains – the list is empty again. The temptation is strong to write "nothing" again, i.e., 0. However, then *all* test cases will fail.

The actual value 0 is not the expected value 42.

The actual value 0 is not the expected value 6.

The actual value 0 is not the expected value 210.

Why does the result always return 0? We can see this by evaluating the output of `(list-product list3)`. It looks like this – we've omitted some intermediate steps:

```
(list-product list3) ÿ (list-
product (cons 1 (cons 2 (cons 3 empty)))) ÿ (cond ((empty? (cons
1 ...) ...) ((cons? (cons 1 ...) ...) ÿ (cond ((empty? (cons 1 ...) ...) ((cons? (cons 1 ...) ...)
ÿ (* (first (cons 1 ...) (list-product (rest (cons 1 ...))) ÿ (* 1 (list-product (rest (cons 1 ...))) ÿ
(* 1 (list-product (cons 2 (cons 3 empty)))) ÿ (* 1 (* 2 (list-product (cons 3 empty)))) ÿ (* 1
(* 2 (* 3 (list-product empty)))) ÿ (* 1 (* 2 (* 3 (cond ((empty?
empty) 0) ...))) ÿ (* 1 (* 2 (* 3 (cond (#t 0) ...))) ÿ (* 1 (* 2 (* 3 0)))
```

The evaluation is going so well – but at the end, ``list-product`` breaks everything and multiplies by 0. And this happens with *every* input. So instead of multiplying by 0, we need to use a number that doesn't break the result but leaves it exactly as it is. In the case of multiplication, that's 1, not 0. The correct function would then look like this:

```
(define list-product (lambda
(list)
(cond
((empty? list) 1) ((cons?
list)
(* (first list) (list-product (rest list))))))
```

In retrospect, this also explains why `list-sum` must contain 0 in the empty case: The 0 doesn't invalidate the result when adding. That's why 0 is said to be the additive *neutral element*, and 1 the multiplicative neutral element.

6.2 LISTS WITH OTHER ELEMENTS

```
listen/list-of.rkt
```

```
code
```

The "list" principle is not limited to lists of numbers: Lists of character strings, the animals from Chapter 4.1, or images are equally conceivable. To make this possible, we can adapt the data definition. We remove all mentions of numbers and speak exclusively of "elements":

A list is one of the following: - an empty list; - a cons list

A cons list consists of:

; - the first element

; - a list containing the remaining elements

In fact, we can already build lists whose elements are not numbers:

```
(define string-list (cons "Mike" (cons "Sperber" empty)))
```

Task 6.4

Try the definition of `string-list` !

ÿ

DrRacket can execute this definition, but there are numerous signature violations.

The violated signatures are all `number`, most prominently in the record definition of `cons-list`. If we want to use strings there without prohibiting lists of numbers, we have to abstract away from `number` and `string`.

In the record definition, this is done by changing `cons-list` to `(cons-list-of element)`. This acts like a

`lambda` with `element` as a parameter. We can then use `element` within the record definition.

use:

```
(define-record(cons-list-of element)
  cons
  cons?
  (first element)
  (rest ...))
```

This record definition defines ``cons-list-of`` as a function that accepts a signature (the signature of the elements) and returns a signature, namely that of the cons lists with the given signature for the elements. We can also write the signature as follows:

```
(: cons-list-of (signature -> signature))
```

Since `cons-list-of` constructs a signature, we also call this function *the signature constructor*.

The `element` is a *signature parameter*. Therefore, where we previously wrote `cons-list`, we must now write `(cons-list-of ...)` and specify a signature for the elements, for example `(cons-list-of number)` or `(cons-list-of string)`.

The signature of ``restwar`` was previously ``list-of-numbers``, so we need to change that as well. Exactly how will only become clear in connection with the abstracted definition of ``list-of-numbers``. Since this is a completely normal definition using ``define``, we can use a **lambda** to abstract from the signature of the elements:

```
(define list-of (lambda
  (element)
    (signature
      (mixed empty-list
        (cons-list-of element))))))
```

With this definition, we can now write `(list-of number)` instead of `list-of-numbers`. This also allows us to complete the record definition of `cons - list -of` by writing `(list-of element)` as the signature for the `rest` field, where previously `list-of-numbers` was used.

```
(define-record(cons-list-of element)
  cons
  cons?
  (first element) (rest
    (list-of element)))
```

That resolves the signature violations.

A `define-record` form with signature parameters has the following general form:

```
(define-record (t sp1 . . . spk)
```

```
  c
```

```
  p
```

```
  (sel 1 sig1)
```

```
  . . .
```

```
  (sel n sign))
```

- The signature parameters `sp1 . . . spk` can be used in the signatures `sig1 . . . sign` the.
- `t` is the name of the signature constructor with the signature:

```
(: t (signature . . . (k times) -> signature))
```

- `c`, `p` and `sel 1, . . . , sel n` are the names of constructor, predicate, and selectors.

Figure 6.2: `define-record` with signature parameters

Figure 6.2 summarizes the new form of `define-record` with signature parameters.

With this new form of `define-record`, it's no longer obvious what the signatures of the constructor and the two selectors should look like when we write them out. We need a new construct to express that potentially a different element signature is used with each call to `cons`, `first`, and `rest`. It looks like this:

```
(: cons (%element (list-of %element) -> (cons-list-of %element)))
```

```
(: first ((cons-list-of %element) -> %element)) (: rest ((cons-list-of  
%element) -> (list-of %element)))
```

The `%` symbol denotes *signature variables*. The signature variable `%element` represents any signature. The signature declaration of `cons` also means that the signature of the new element (the first `%element`) and the signature of the list elements (the `%element` within `(list %element)`) should match. (However, DrRacket does not check this.)

You might have thought that instead of `%element`, the signatures above could also contain `"any"`. Technically, that's correct. However, the resulting signatures would be less precise. For example, imagine they contained something like this:

```
(: cons (any (list-of any) -> (cons-list-of any)))
```

This signature says "`cons` accepts any value and any list and produces any list again". For comparison, the signature says

```
(: cons (%element (list-of %element) -> (cons-list-of %element)))
```

This means that the new element that accepts `cons` must have the same signature as the existing elements of the list and the elements of the result list: All three occurrences of

`%elements` must have the same signature in a specific application of `cons`.

Before the program works again, we need to fix one small problem: ``list-of-numbers`` is no longer defined, but it's still in the signature declarations of ``list-sum`` and ``list-product``. We can either replace ``list-of-numbers`` with ``(list-of-number)`` or add a signature definition:

```
(define list-of-numbers (signature (list-of number)))
```

This definition must precede `list-sum` and `list-product` so that it is recognized there.

6.3 CONSTRUCTION INSTRUCTIONS

When constructing functions that accept lists, we combined three templates: mixed and composite data, as well as self-references. We will be writing many more functions with lists as input. Therefore, it is worthwhile to dedicate a separate construction guide to such functions.

Construction instructions 18 (Lists as input: Template)

The template for a function that accepts a list looks like this:

```
(: f (... (list-of elem) ... -> ...))
```

```
(define f
  (lambda (... list ...)
    (cond
      ((empty? list) ...) ((cons?
        list)
        ...
        (first list)
        ...
        (f ... (rest list) ...)
        ...
      )))
```

The built-in ``list`` function allows you to create lists from their elements without using ``cons``. It accepts any number of arguments, creates a list from them, and returns it.

```
(list 1 2 3) ⇒ #<list 1
2 3> (list "Axl" "Slash"
"Izzy") ⇒ #<list "Axl" "Slash" "Izzy">
```

Figure 6.3: list

6.4 BUILT-IN LISTS

Since lists will appear frequently in this book and it is cumbersome to put the definitions for `cons` and `list-of` at the beginning of programs every time, the next language level makes programming with lists easier.

Therefore, switch to the next language level: Start a new

file. Then select the Language menu again, then the Select Language menu item, and then Write Your Program! (*without* Beginner). Then press Start again.

From now on, ``cons``, ``cons?``, ``first``, and ``rest`` are predefined, just like the signature constructor ``list-of``. There is also a handy new function ``list``, described in Figure 6.3. is.

In the new language level, non-empty lists are printed more clearly than before, namely as `#<list ...>`, with the list elements arranged between the angle brackets:

```
(list "Axl" "Slash" "Izzy") ⇒ #<list "Axl"
"Slash" "Izzy">
```

6.5 PARAMETRIC POLYMORPH

[listen/polymorphism.rkt](#)

code

Don't be intimidated by the highfalutin title: In this section, we'll write a fairly simple function that determines the length of a list. Here's a brief description:

Calculate the length of a list

The beginning of a signature declaration looks like this:

```
(: list-length ((list-of ...) -> natural))
```

What can we write instead of the ellipsis? The function shouldn't care which signature it uses.

The elements of the list match. We can express this by using a signature variable, just like with the signatures of `cons`, `first`, and `rest`. It looks like this:

```
(: list-length ((list-of %element) -> natural))
```

The use of the signature variable `%element` means, just as above, that the signature of the elements of the list can be different each time `list-length` is called.

Functions that accept arguments whose signatures contain signature variables are called *polymorphic* or *parametrically polymorphic* (because the signature variable provides a kind of parameter), and the associated concept is called *parametric polymorphism*: a big word,

This is just a small example. More and more interesting examples of parametric polymorphism will be given in Chapter 9.

Continuing with `list-length` – here is the framework:

```
(define list-length
  (lambda (lis)
    ...))
```

The template is the same as before:

```
(define list-length
  (lambda (lis)
    (cond
      ((empty? lis) ...)
      ((cons? lis)
       ... (first lis) ...
       ... (list-length (rest lis)) ...))))
```

The `'list-length'` property doesn't care what the value of `'(first lis)'` is. The length of the list is independent. It doesn't matter what kind of value it contains – the only crucial factor is how many there are. (This fact is currently responsible for the parametric polymorphism.) Therefore, we can remove the selector call `(first lis)` from the template and then add it to the complete body.

Zen:

```
(define list-length
  (lambda (lis)
```

```
(cond
  ((empty? lis) 0)
  ((cons? lis)
   (+1
    (list-length (rest lis))))))
```

The `list-length` function is built-in under the name `length` .

6.6 FUNCTIONS THAT PRODUCE LISTS

`listen/dillo-list.rkt`

code

In the previous sections, we programmed only functions that *accept lists*. In this section, we will write functions that *produce lists*. This is done with

Techniques we've already presented. We'll make things more interesting by looking at lists of composite data in a first example and then, in a second example, Process two lists.

6.6.1 ARMADILLES RUN OVER

On page 105 we wrote the function `run-over-dillo`, which is for running over...

This section describes the function that does this en masse, for example, for all armadillos on a highway.

For this, we adopt the data and record definitions of Armadillo from section 3.4 , as well as the function definition of `run-over-dillo`. You can also simply use the Armadillo code.

To expand – however, you then need to switch the language level to "Write your program!" so that the lists are available.

We can put armadillos in lists, just like numbers, strings, or Boolean operators.

Values. Here is an example:

```
; Armadillos on Highway 75
(define highway75 (list dillo1 dillo2 dillo3 dillo4))
```

(`Dillo1`, `dillo2`, `dillo3` and `dillo4` are the example armadillos from section 3.4.)

This list has the signature `(list-of dillo)`. If we want to write a function,

To run over all armadillos on a list, the list would therefore need to have the following short description and signature:

```
; Armadillos run over (: run-over-
dillos ((list-of dillo) -> (list-of dillo)))
```

The above example list can serve as a test case:

```
(check-expect (run-over-dillos highway75) (list (make-dillo
55000 #f)
dillo2
(make-dillo 60000 #f) dillo4))
```

As a reminder: `dillo2` and `dillo4` are already dead, so they've been run over as before. Here's the scaffolding:

```
(define run-over-dillos
  (lambda (dillos) ...))
```

The function accepts a list as input, so we can, as so often before, use the appropriate Use a stencil:

```
(define run-over-dillos
  (lambda (dillos) (cond

    ((empty? dillos) ...) ((cons?
dillos)
... (first dillos) ...
... (run-over-dillos (rest dillos)) ...))))
```

In the first branch, things are clear: if an empty list goes in, an empty list comes out. In the second branch, we can first focus on the first armadillo. We already have a function that processes a single armadillo; we can apply this to the first element of the Apply list:

```
(define run-over-dillos
  (lambda (dillos) (cond

    ((empty? dillos) empty) ((cons?
dillos) ... (run-over-
dillo (first dillos)) ...
... (run-over-dillos (rest dillos)) ...))))
```

Let's read again the two expressions that are in the second branch:

- `(run-over-dillo (first dillos))` is the first armadillo on the list, run over.
 - `(run-over-dillos (rest dillos))` is a list of the remaining armadillos, also run over.
- drive.

We are asked for a list of *all* armadillos that have been run over: So we only need the results of the two
Combine printouts with `cons` :

```
(define run-over-dillos (lambda
  (dillos)
    (cond
      ((empty? dillos) empty) ((cons?
        dillos)
          (cons (run-over-dillo (first dillos))
            (run-over-dillos (rest dillos)))))))
```

Complete!

This example shows that we do not need a new technique for functions that produce lists: If a function is to produce an empty list, we use `empty` at the appropriate place, and for non-empty lists we use `cons`, thus using the template for functions that produce composite data.

6.6.2 EXTRACTING ARMADILLAS

We will now tackle another exercise involving the production of lists: From a list of armadillos, we want to select all those that are (still) alive, perhaps to bring them to safety.

This is what a short description, signature, test case, and framework look like:

```
; Collect live armadillos (: live-dillos ((list-of dillo) ->
(list-of dillo)))
```

```
(check-expect (live-dillos highway75)
  (list dillo1 dillo3))
```

```
(define live-dillos (lambda
  (dillos)
    ...))
```

Next, we'll insert the standard template for functions that accept lists:

```
(define live-dillos
  (lambda (dillos)
    (cond
      ((empty? dillos) ...)
      ((cons? dillos)
       ... (first dillos) ... ... (live dillos
       (rest dillos)) ...))))
```

The first case concerns the empty list – we cannot live from an empty list of armadillos-

To get rid of the armadillos, that's where "empty" goes . In the second case, it's a bit more complicated.

Let's clarify again what the two components in the template mean:

- (first dillo) is the first armadillo in the list. • (live-dillos

(rest dillos)) are the living ones among the remaining armadillos.

Since the overall task is to extract the live armadillos, we should first determine whether the first armadillo is (still) alive. The interim result is as follows:

```
(define live-dillos
  (lambda (dillos) (cond
    ((empty? dillos) empty) ((cons?
    dillos)
    ... (dillo-alive? (first dillos)) ...
    ... (first dillos) ... ... (live dillos
    (rest dillos)) ...))))
```

The call (dillo-alive? (first dillos)) lends itself to a binary branch:

```
(define live-dillos
  (lambda (dillos) (cond
    ((empty? dillos) empty) ((cons?
    dillos)
    (if (dillo-alive? (first dillos))
      ...
      ...) ...
    (first dills) ...
    ... (live dillos (rest dillos)) ...))))
```

We now need to consider what to write in the two branches of the **if statement**. In the first branch – the consistent one – the armadillo (*first dillos*) is (still) alive, in the second – the alternative one – native – it is dead.

In the alternative (armadillo dead), (*live-dillos (rest dillos)*) is already the correct answer. (*first dillos*) is ignored.

```
(define live-dillos
  (lambda (dillos)
    (cond
      ((empty? dillos) empty) ((cons?
                                dillos)
                                (if (dillo-alive? (first dillos))
                                    ...
                                    (live-dillos (rest dillos))))
      ... (first dills) ...))))
```

Even though we seem to have already used up the recursive call (*live-dillos (rest dillos)*), we need the remaining live armadillos again in the other branch (armadillo lives (still)). This time, however, we must make sure that we include the first armadillo in the output and *append it to the beginning of the remaining armadillos*:

```
(define live-dillos
  (lambda (dillos)
    (cond
      ((empty? dillos) empty)
      ((cons? dillos)
       (if (dillo-alive? (first dillos))
           (cons (first dillos)
                 (live-dillos (rest dillos)))
           (live dillos (rest dillos))))))
```

Complete!

6.6.3 JOINING TWO LISTS

Our next example requires a function called ``concatenate`` that concatenates two lists. The short description, signature, and structure are as follows:

```
; concatenate two lists
```

```
(: concatenate ((list-of %element) (list-of %element)
  -> (list-of %element)))
```

This function is polymorphic like `list-length` – the signature variable `%element` appears in the signature. Note that `%element` appears *three times*: The signature indicates that the signatures of the elements in the two input lists and the signature of the output list correspond to each other. Here is a test case and framework for `concatenate`:

```
(check-expect (concatenate (list 1 2 3) (list 4 5 6))
  (list 1 2 3 4 5 6))
```

```
(define concatenate
  (lambda (list1 list2) ...))
```

Construction guide 18 on page 183 is actually only intended for functions that accept a single list. Which of the two is the `list` from the guide? If in doubt, we can try both alternatives. We'll try `list2` first:

```
(define concatenate
  (lambda (list1 list2) (cond

    ((empty? list2) ...) ((cons?
    list2)
    ... (first list2) ...
    ... (concatenate list1 (rest list2)) ...))))
```

The first branch of the **cond function** is still simple: If `list2` is empty, `list1` must be the result. However, in the example above, the value of `(concatenate list1 (rest list2))` would be the list `#<list 1 2 3 5 6>`. This list is missing element 4 in the middle, and it's not clear how our function could add it. This approach therefore leads to a dead end. We will therefore try applying the template to `list1` instead of `list2`:

```
(define concatenate
  (lambda (list1 list2) (cond

    ((empty? list1) ...) ((cons?
    list1)
    ... (first list1) ...
    ... (concatenate (rest list1) list2) ...))))
```

The first ellipsis is easy to replace: If the first list is empty, the result is the second list, `list2`. For the second case, we should recall what the value of `(concatenate (rest list1) list2)` returns: the result of this call is a list composed of `(rest list1)` and `list2`. Applying this to the example above, if `list1` is the list `#<list 1 2 3>` and `list2` is the list `#<list 4 5 6>`, then `(rest list1)` has the value `#<list 2 3>`. The value of `(concatenate (rest list1) list2)` would therefore be:

```
#<list 2 3 4 5 6>
```

The first element of `list1` (`first list1`) is missing; it needs to be appended to the beginning of the result. This can be done using `cons`:

```
(define concatenate
  (lambda (list1 list2)
    (cond
      ((empty? list1) list2) ((cons? list1)
                              (cons (first list1)
                                    (concatenate (rest list1) list2))))))
```

This example shows another template element that will appear more often: As with other composite data, functions that are intended to construct lists must contain a `cons` somewhere.

Since many programs require the `list-length` and `concatenate` functions, they are. Incidentally, the terms "length" and "append" are already incorporated into the teaching languages.

6.7 MINIMUM OF A LIST AND LOCAL VARIABLES

```
listen/list-min.rkt
```

```
code
```

We'll tackle another exercise: calculating the minimum of a list of numbers. It seems simple, but it has some pitfalls. Here's a first attempt at a brief description, tests, and framework:

```
; Calculate the minimum of a list of numbers
```

```
(: list-min ((list-of real) -> real))
```

```
(check-expect (list-min (list 5 3 1 4)) 1) (check-expect (list-min
(list 5 3 1 4 -4 3)) -4)
```

Perhaps you can already see the first problem. The tests omit a simple but important case, namely the empty list – and that has no minimum.

One possibility would be to invoke [violation](#) in that case, as in section [2.10](#) on page [82](#). (See exercise [6.16](#) on page [204](#).) Another approach, as described in section [4.4](#) on page [132](#), is to use an extra value for this case. As in that case, this has the advantage that we can see in the signature that the function can also fail. Therefore, we introduce the following empty record type for this purpose:

```
; No result (define-
record no-result
  make-no-result
  no result?)
```

We will expand the signature accordingly:

```
(: list-min ((list-of real) -> (mixed real no-result)))
```

We will also write a test case for this:

```
(check-expect (no-result? (list-min empty)) #t)
```

Now we can apply the template for lists as usual:

```
(define list-min (lambda
  (list)
    (cond
      ((empty? list) ...) ((cons?
        list)
          ... (first list) ...
          ... (list-min (rest list)) ...))))
```

The empty case is precisely the one in which there is no result:

```
(define list-min (lambda
  (list) (cond
    ((empty? list) (make-no-result)) ((cons? list)
    ... (first list) ...
    ... (list-min (rest list)) ...))))
```

In the cons case, we need to be careful that the recursive call to `list-min` returns mixed data. Therefore, we need to implement a branch that distinguishes between `no-result` and a number. We do this with a binary branch:

```
(define list-min (lambda
  (list)
    (cond
      ((empty? list) (make-no-result)) ((cons? list)

      (if (no-result? (list-min (rest list)))
          ...
          ...))
      ... (first list) ...
      ... (list-min (rest list)) ...))))
```

If the recursive call returns `no result`, we only have the first element of the list (`first list`) – the rest of the list must then be empty. In that case, (`first list`) must also be the minimum:

```
(define list-min (lambda
  (list)
    (cond
      ((empty? list) (make-no-result)) ((cons? list)

      (if (no-result? (list-min (rest list))) (first list)

          ...))
      ... (first list) ... ... (list-min
      (rest list)) ...))))
```

If the recursive call to `list-min`` returns a result, that result is the minimum of the remaining list. However, to calculate the minimum of the entire list, we must compare it to `(first list)``, which could also be the minimum. Depending on the result of the comparison, one or the other is the minimum.

```
(define list-min
  (lambda (list) (cond
    ((empty? list)(make-no-result))
```

The body of a **lambda function** and the branch of a **cond function** can each contain a definition at the beginning. Such a *local definition* binds a *local variable*. This definition is evaluated each time the evaluation arrives at the body of the **lambda** or at the branch of the **cond** and is only visible within it ("locally").

Figure 6.4: local definition, local variable

```
((cons? list)
  (if (no-result? (list-min (rest list))) (first list)

      (if (< (first list)
            (list-min (rest list))) (first list)

          (list-min (rest list))))))
```

This function is correct.

However, it still has one flaw: `(list-min (rest list))` appears a total of three times, and it's actually possible that all three are evaluated: three times the same work. (You can verify this with the stepper.) We should ensure that the recursive call is only evaluated once. To do this, we introduce a new programming construct, *the local variable* (see Figure 6.4), for which we first create a definition within the body of `list-min`:

```
(define list-min
  (lambda (list)
    (cond
      ((empty? list) (make-no-result)) ((cons? list)

      (define rest-min (list-min (rest list))) ...))))
```

This definition is evaluated every time the evaluation reaches the `cond` branch for `cons` and is only active within this branch. We now replace `(list-min (rest list))` with the new variable `rest-min`:

```
(define list-min
  (lambda (list) (cond

    ((empty? list)(make-no-result))
```

```

((cons? list)
 (define rest-min (list-min (rest list))) (if (no-result? rest-min)

      (first list)
      (if (< (first list) rest-min)

            (first list)
            rest-min))))))

```

Complete!

That means... we can still improve it a little bit: The if-expression at the end.

This calculates the minimum of [the first list](#) and [the remainder](#). There is already a built-in function called ``min`` for calculating the minimum of two numbers, which we can use:

```

(define list-min (lambda
  (list)
    (cond
      ((empty? list) (make-no-result))
      ((cons? list)
       (define rest-min (list-min (rest list))) (if (no-result? rest-min)

            (first list)
            (min (first list) rest-min))))))

```

That's it!

6.8 FUNCTIONS ON NON-EMPTY LISTS

The ``list-min`` example was useful for demonstrating how we can represent "there is no result". However, there's another way to deal with the empty list problem in ``list-min``: we simply forbid it! Then we don't need ``no-result`` at all.

We therefore define an alternative version of [list-min](#) with a short description and signature as follows:

```

; Calculate the minimum of a non-empty list of numbers (: list-min-nonempty ((cons-
list-of real) -> real))

```

The signature constructor `cons-list-of` corresponds to the signature constructor of the same name from section 6.2 and describes cons lists – which are precisely the non-empty ones.

We are using the same test cases as before:

```
(check-expect (list-min-nonempty (list 5 3 1 4)) 1) (check-expect (list-min-nonempty
(list 5 3 1 4 -4 3)) -4)
```

We need to adjust the template slightly compared to the one for `list-of`. After all, for a non-empty list, the condition `(empty? list)` *always* returns `#f` and `(empty? list)` *always* returns `#t`, so branching with it would be pointless. Instead, we use `(rest list)`, which can be either empty or non-empty. It looks like this:

```
(define list-min-nonempty (lambda (list)

  (cond

    ((empty? (rest list)) ...) ((cons? (rest list))

      ... (first list) ... ... (list-min-
nonempty (rest list)) ...)))
```

In the first branch, the list has exactly one element; in the second branch, it has at least two elements. The minimum of a list with one element is simply that single element. In the second branch...

In this case, just as with `list-min`, we can calculate the minimum of `(first list)` and the minimum of the rest of the list:

```
(define list-min-nonempty (lambda (list)

  (cond

    ((empty? (rest list)) (first list))

    ((cons? (rest list)) (min (first list)

      (list-min-nonempty (rest list))))))
```

Complete!

Task 6.5

Can you predict exactly what will happen if you apply `list-min-nonempty` to an empty list? Try it out! ÿ

Task 6.6

Write a specific construction guide for functions that accept non-empty lists!

ÿ

NOTES

Lists are a kind of all-purpose glue: they are also suitable for representing tables, quantities, and many other composite data sets. For lists, there is a huge number of practical functions, of which the functions in this chapter are only the tip of the iceberg. From here on...

Since lists are firmly embedded in the teaching languages, they can serve as a universal means of communication between programs because functions on lists from one program can also be accessed in another. This distinguishes functional languages from many other programming languages, in which lists must be defined by the programmer themselves or can only be used by another. play a subordinate role.

TASKS

Task 6.7

Write expressions for lists that match the example lists from the beginning of Chapter 6 on page 173. represent.

Task 6.8

Write the following functions into lists:

1. A function ``count-zeroes`` that calculates the number of zeros in a list of numbers
net.
2. A function ``contains>10`` that determines whether a list of numbers contains a number that
greater than 10.

Task 6.9

Write versions of `list-sum` and `list-product` that only work on non-empty lists and where you don't need to specify a neutral element.

ÿ

Task 6.10

In a field, to put it simply, there are potatoes, clods of earth, and stones. A component of the field is therefore a potato, a clod of earth, or a stone. Each of these components has a weight in grams; potatoes also possess the additional property of whether they are edible. are or are not (and therefore need to be sorted out). Write a program for a potato harvester that must filter the edible potatoes from the field debris.

1. Conduct a data analysis for arable land components and write up corresponding data and Record definitions. Give a few examples of field components that you will encounter in the next You can use subtasks in test cases.
2. Write a function `called edible-potato?` that checks whether a given field component is an edible potato.
3. Write a function ``weight`` that returns the weight of any field component.
gives.
4. Write a data and record definition for lists of arable land components. Include all signatures! Also, provide at least five different examples of lists of arable land components.
5. Write a function `total-weight` that accepts a list of field components and calculates the total weight of all arable components in the list.
6. Write a function `total-edible-potatoes-weight` that accepts a list of arable components and calculates the total weight of all edible potatoes in the list.
7. Write a function ``count`` that accepts a list of field components and returns the following values:
The number of arable components in the list is determined.
8. Write a function ``count-edible-potatoes`` that returns a list of field components.
accepted and the number of edible potatoes in the list determined.
9. A machine is supposed to automatically harvest the potatoes from the field. Unfortunately, the machine is very old and therefore cannot cope with large stones and clods of earth.
The harvesting process is aborted as soon as a stone or clod of earth heavier than 1kg is encountered.
is, into the machine. Write the function `harvester` that simulates this process.
This function receives a list of field components as input, which it processes gradually.
The task is to produce a list of field components that includes the harvested edible potatoes. Write enough test cases, also considering the case where the
The machine is running at idle. Use the functions from the previous task sections.
10. Write a function ``heaviest-potato`` that takes a list of field components and calculates the
picks out the heaviest edible potato and returns it. If there is no edible potato on the list-

If ``toffel`` is included, the function (`make-no-result`) should return ``null``. (See section 4.4) (on page 132.)

11. Write a function ``filter-edible-potatoes`` that returns a list of field components. accepted and returns a list of edible potatoes.
12. Write a function ``drop-stones`` that accepts a list of field constituents and returns a list in which there are no more stones.
13. Write a function `average-weight-edible-potatoes` that accepts a list of arable components and calculates the average weight of edible potatoes.

Task 6.11

We represent a lithium-ion battery as a list of lithium-ion cells. A cell consists of a maximum charge (in milliampere hours, mAh) and an instantaneous charge (also in mAh).

If a cell has a charge of 1000mAh, that means the cell has a current strength of It can supply 1000mA (milliamperes) for one hour. The instantaneous charge must never exceed 1000mA. The maximum charge should increase and never fall below 10% of the maximum charge, which in our example cell is 100mAh, otherwise the cell will break.

1. Perform a data analysis for Li-ion batteries and Li-ion cells, write down the data- Define the data definitions and implement them.
2. Write the following functions:
 - `cell-full?`, which checks whether a cell is fully charged (that is, whether the current- (aunt charge equals the maximum charge)
 - `cell-empty?`, which checks whether a cell is discharged (that is, whether the current la- (where the charge is equal to the minimum charge, i.e., 10% of the maximum charge)
 - `cell-defect?`, which checks whether a cell is defective (that is, whether the current charge (exceeds the maximum charge or falls below the minimum charge)
 - `cell-ok?`, which checks whether a cell is working (that is, whether the current charge is sufficient). (within the minimum and maximum charge range)
3. Write the following functions:
 - `battery-full?`, which checks whether a battery is fully charged (that is, whether all cells are fully charged). (are full); a battery without a cell is considered charged.

- **Battery empty?**, which checks whether a battery is discharged (that is, whether all cells are empty). are); a battery without a cell is also considered empty.
- **Battery broken?**, which checks whether a battery is defective (i.e., whether at least one cell is defective); a battery without any cells is considered not defective.
- **Battery OK?**, which checks whether a battery is working (i.e., whether all cells are functioning); a battery without any cells is considered functional.

In the following subtasks, assume that only *functioning* cells and batteries will be handed over:

4. The charger can charge one cell by 500mAh per hour. Write a function.
time-to-fully-charge-cell, which calculates how many hours it takes for the charger to fully charge the cell.
5. Write a function **charge-cell** that accepts a cell and a time in hours and returns the cell that was charged with the charger mentioned in the previous subtask for the given time. The charge must not exceed the maximum charge!
The remaining time is lost if the cell is already full.
6. Write a function called **charge-battery** that charges a battery. The function returns a battery that has been charged by the charger for the specified time. If a cell is fully charged, the next cell is charged for the remaining time. If all cells are charged but the time has not yet expired, the remaining time elapses.
7. The discharge time of a cell depends on the load. A device has a current draw value, specified in mA.
Write a function **time-to-fully-discharge-cell** that accepts a cell and a current draw per hour and calculates how long it takes to fully discharge the cell.
8. Write a function **time-to-fully-discharge-battery** that accepts a battery and a consumption per hour and calculates how long it takes until the battery is empty.

9. Write a function ``discharge-cell`` that accepts a cell, a consumption rate per hour, and the duration (in hours) for which the load draws power from the cell. The function should return a cell that has supplied power to the load for the specified duration. The charge must not fall below the minimum charge! Any remaining time is wasted if the cell is already empty. (In reality, the load simply runs out of power.)

10. Write a function ``discharge-battery`` that accepts a battery, a consumption rate per hour, and the duration (in hours). The function should return a battery that has supplied power to the device for the specified duration. When the charge of one cell is depleted, the charge of the next cell is used for the remaining time. If all cells are discharged but the time has not yet expired, the time expires, and the device runs out of power.

Task 6.12

Write a function that sorts a list of numbers in ascending order in the simplest possible way! Proceed as follows:

1. Write a function `delete-once` that retrieves the first occurrence of a number from a list of numbers. a given number away.
 Example: `(delete-once 5 (list 1 2 5 3 4 5))` results in `#<list 1 2 3 4 5>`.
 (If the list does not contain the number, it should be returned unchanged.)
2. Write a function `called sort` that sorts a list of numbers in ascending order.
 Use `list-min` in section 6.7 on page 192 and `delete-once`.
 Example: `(sort (list 1 5 3 2 4))` results in `#<list 1 2 3 4 5>`.

Task 6.13

In a company, employees clock in and out on time cards to record their working hours: Each time card shows the employee's number and a list of hours worked. For simplicity, assume that only whole hours are recorded.

Furthermore, each employee has a personnel file containing their name, number, and hourly wage (also only in whole euros). At the end of each month, each employee's wages must be transferred. This is done by generating payment transfers; each transfer consists of the employee number and a payment amount.

Write data and record definitions for time cards, personnel files, and transfer forms.
sung.

Write a function that retrieves a list of personnel files and a list of time cards for
One month accepted, hours worked for each employee totaled, and a list compiled.
which returns the transfers.

Task 6.14

You own a parking lot with designated areas for different vehicles: cars , trucks , motorhomes ,
buses , and bicycles. Due to financial difficulties, the city of Tübingen has decided that you
will be charged a fee for each vehicle parked in your parking lot.

You must pay taxes, each per day:

- car: 3 cents
- truck: 5 cents + 3 cents per axle
- caravan: 4 cents
- bus: 10 cents + 1 cent per seat
- bike: 1 cent.

The vehicle owner can only pay the fees for the respective day, so you don't need to pay attention to the
parking duration.

1. Conduct a data analysis and write data and record definitions for the vehicle types **truck**,
bus , and **simple-vehicle**. **Simple-vehicle** refers to vehicles that do not incur variable costs.
2. Write functions **tax-truck**, **tax-bus** , and **tax-simple-vehicle** to determine the
Taxes for these vehicle types
3. Define a data type **`vehicle`**. Write a function **`tax-vehicle`** that calculates the taxes .
Calculated for any vehicle.
4. Now write a function **`calculate-tax`** that calculates the daily taxes you pay to
You have to pay the tax office for all vehicles in your parking lot. The function is accepted.
It compiles a list of vehicles and calculates the total taxes for a day.

Use the construction guidelines for every function and every record you write:

First, create a short description, signature, some test cases, and the skeleton of the function.

Then, complete the skeleton and make sure that the test cases run correctly!

Task 6.15

Since you can now program with lists, Dr. Knaubichler asks for your help again (see exercise 4.7 on page 136): Write a program that selects from a list of basic creatures the two basic creatures that, when crossed, produce the most powerful creature.

Proceed as follows: 1.

Write a predicate for creatures.

2. Write a function `creature-power` that accepts any creature and calculates its power. Power is the sum of all its attributes.

3. Write a function `find-optimal-breeding-partner` that accepts a base creature `c1` and a list of base creatures. The function should select and return the base creature `c2` from the list that is the best breeding partner of `c1`. In other words, the cross of `c1` and `c2` should produce the highest-performing creature among all possible crosses. If the list of base creatures is empty, the function should return `make-no-result`. (See section 4.4 on page 132.)

4. Write a function `breed-optimal-partner` that accepts a base creature `c1` and a list of base creatures. The function should cross `c1` with the base creature from the list that results in the most powerful creature. If the list of base creatures is empty, the function should return `(make-no-result)`. Use `find-optimal-breeding-partner`.

5. Write a function `optimal-breed` that accepts a list of basing creatures. The function should breed the two basing creatures from the list that, among all possible crosses, produce the creature with the highest performance. Use the functions from the previous subtasks.

Task 6.16

Program a variant of the `list-min` function in section 6.7 on page 192, in which you call the `violation` function instead of `make-no-result`. What else do you need to change?

7 NATURAL NUMBERS

This chapter deals with functions that count something and therefore interact with numbers. We've already seen a number of programs that calculate with numbers, but Counting is something special and therefore deserves its own chapter with its own construction instructions.

7.1 Numbers That Matter

`natural-numbers/natural.rkt`

code

You probably know from math class the *power* of a number (of *base*) b to a *Exponents* e :

$$b^e$$

We are interested here in exponents that are integers starting from 0. For such exponents, the power is... defined as follows:

$$b^e \stackrel{\text{def}}{=} \underbrace{b \times \dots \times b}_{e\text{-mail}}$$

The exponent e must therefore be a natural number.¹

In everyday language, natural numbers are simply the numbers that count objects. Intuitively, the natural numbers are simply arranged on a number line and form A series. But that's not very helpful for programming. We can do much more with the following data definition:

A natural number is one of the following:

; - 0

; - the successor of a natural number

This definition contains familiar elements: it involves a case distinction, and the second case contains a self-reference.

The term "successor" may seem somewhat strange to you: The successor of a number is simply the number "plus 1". Because the successor in the context of natural numbers Since it has a special significance, we will first give it its own auxiliary function:

¹ Astute readers might point out that in some textbooks the natural numbers begin with 1, but We adhere to DIN 5473, and the 0 is included in that.

```
; successor of a number (:
successor (natural -> natural))
```

```
(define successor
  (lambda (n) (+
    n 1)))
```

Where there is a successor, there must also be a predecessor, and the predecessor subtracts one:

```
; predecessor of a number (:
predecessor (natural -> natural))
```

```
(define predecessor (lambda
  (n)
  (- n 1)))
```

However, we've cheated a bit with this definition, because the predecessor only works for natural numbers that are the successors of other natural numbers – that is, for so-called *positive* natural numbers. The signature can't express this, as there's no built-in signature for it. But we can implement a branch according to the data definition to protect the function from misuse:

```
(define predecessor (lambda
  (n) (cond
    ((zero? n)
      (violation "0 does not have a predecessor")) ((positive? n)
      (- n 1))))
```

In the function definition, you can see the two built-in functions `zero?` and `positive?`, which distinguish between the two cases of the data definition of natural numbers.

Back to the actual task, the potency. Here's what a short description, signature, and two tests could look like:

```
; Calculate the power of a number (: power
(number natural -> number))
```

```
(check-expect (power 5 0) 1) (check-
expect (power 5 3) 125)
```

Next up is the scaffolding:

```
(define power
  (lambda (base exponent) ...))
```

For the body of the expression, we can derive a template from the data definition for `exponent`, which is the natural number. The data definition for natural numbers is a case distinction with two cases; accordingly, we need a branching statement with two branches.

```
(define power
  (lambda (base exponent) (cond
    ((zero? exponent) ...) ((positive?
    exponent) ...))))
```

The second case of data definition also contains a self-reference, so we also write... a recursive call to the template:

```
(define power
  (lambda (base exponent)
    (cond
      ((zero? exponent) ...) ((positive?
      exponent)
      ... (power base (predecessor exponent)) ...))))
```

In the first case – exponent 0 – the result must be a power of 1, the neutral element with respect to multiplication: This is nothing more than multiplying the numbers in the empty list. (See also page 180 in section 6.1.) In the other case, the result of the recursive call is the power of `base` with the predecessor of `exponent` as the exponent. To obtain `base` multiplied by itself *exponentially* as the result, we still need to perform a multiplication with `base`:

```
(define power
  (lambda (base exponent) (cond
    ((zero? exponent) 1) ((positive?
    base) (* base (power
    base (predecessor exponent))))))
```

Complete!

The term "predecessor" is somewhat of a hindrance here: While it fits well with the use of the word "successor" in the data definition, (`predecessor exponent`) is the same as (`- exponent 1`), and accordingly, we will use (`- ... 1`) directly in the template instead of "predecessor" .

Task 7.1

The function ``predecessor`` has two branches: Why is the call in ``power`` always equivalent to (`- exponent 1`) – what about the other branch?

Construction instructions 19 (Natural numbers as input: template)

A template for a function that accepts a natural number looks like this.

out of:

```
(define f
  (lambda (... n ...)
    (cond
      ((zero? n) ...)
      ((positive? n)
       ...
       (f ... (- n 1) ...)
       ...
      )))
```

7.2 Natural Numbers and Lists

`natural-numbers/list.rkt`

code

There is a close relationship between natural numbers and lists: The data definition for natural numbers corresponds in structure to that of lists. The ``list-length`` function in section 6.5 on page 185 converts a list into a natural number. The reverse is also true:

```
; Create a list of copies of a value (: copies (natural
%element -> (list-of %element)))
```

```
(check-expect (copies 4 23) (list 23 23 23 23))
```

```
(check-expect (copies 5 "Mike")
  (list "Mike" "Mike" "Mike" "Mike" "Mike"))
```

The function's signature contains the signature variable `%element`, meaning it is polymorphic and makes it clear that the elements of the list can only belong to the signature `%element`.

```
(define copies
  (lambda (count element) ...))
```

The construction instructions suggest the following template:

```
(define copies
  (lambda (count element)
    (cond
      ((zero? count) ...) ((positive?
        count)
        ... (copies (- count 1) element) ...))))
```

For the first case, we need a list with 0 elements. In the second case, the recursive...

We call a list with one fewer element than we need. We complete the definition as follows:

```
(define copies
  (lambda (count element)
    (cond
      ((zero? count)
        empty)
      ((positive? count) (cons
        element (copies (-
          count 1) element))))))
```

Another useful function accepts the number of a list item – a so-called *index* – and returns the item:

```
; Retrieve a numbered element from a list
(: nth ((list-of %element) natural -> %element))
```

```
(check-expect (nth (list 1 2 3 4 5) 0) 1) (check-expect (nth (list
  1 2 3 4 5) 2) 3)
```

We should consider what should happen if the index doesn't exist in the list. The function can't return a meaningful result that matches the signature – it has to return an element of the list. Therefore, we can only display an error. This leads to the following test case:

```
(check-error (nth (list 1 2 3 4 5) 5))
```

The framework of the function looks like this:

```
(define nth
  (lambda (list index) ...))
```

We can use the template for lists as input, or the one for natural numbers. Here, the index is the determining factor – the length is irrelevant for the second element of a thousand-element list. Therefore, we follow the construction instructions for natural numbers:

```
(define nth
  (lambda (list index)
    (cond
      ((zero? index) ...)
      ((positive? index)
       ... (nth (rest list) (- index 1)) ...))))
```

In the first case, index 0, i.e., the first element, is requested. In the second case, the recursive call returns the element at position `(- index 1)` in the `rest of the list`, i.e., the `index-th` element in the list. The function looks like this:

```
(define nth
  (lambda (list index)
    (cond
      ((zero? index) (first list))
      ((positive? index) (nth (rest
                               list) (- index 1))))))
```

All test cases are already running – even though we haven't programmed anything for the error case.

Task 7.2

Values with this definition the following expression:

```
(nth (list 1 2 3 4 5) 5)
```

The error message is technically correct, but it doesn't reveal the cause of the error. We can correct this by adding the template for lists as input and using ``violation``. (See section 2.10 on page 82.)

```
(define nth
  (lambda (list index)
    (cond
      ((empty? list)
       (violation "nth: The list is not long enough"))
      ((cons? list)
       (cond
         ((zero? index) (first list))
         ((positive?
          index)
          (nth (rest list) (- index 1))))))))
```

Complete!

7.3 COUNTING THE OTHER WAY

`natural-numbers/count-from.rkt`

code

We'll program another example: Two numbers define the lower and upper limits of a range of integers, and we want to generate the list of these numbers. For example, the numbers between 3 and 7 are 3, 4, 5, 6, 7.

Here is a brief description, signature, and a test:

Calculate a list of consecutive integers (`: between (integer integer) -> (list-of integer)`)

```
(check-expect (between 3 7) (list 3 4 5 6 7))
```

Here is the framework for the function:

```
(define between
  (lambda (from to)
    ...))
```

It's not entirely clear what kind of template we should use to complete the function: The signature states `"integer"`, so the inputs can also be negative numbers,

Therefore, these are not natural numbers. The construction instructions for natural numbers are thus not directly applicable. However, the function clearly involves counting, so natural numbers are somehow involved after all.

We can approach the solution by counting how many elements the output list has. In the test case, there are five elements; generally, this is one more than the difference between the two input numbers – and that is always a natural number.

We can therefore easily rephrase the task: Not "the numbers between 3 and 7"

We don't generate a number, but rather "the 5 numbers starting from 3". We can write a separate function for this:

```
; Generate consecutive numbers starting from a given number (: count-from
(integer natural -> (list-of integer)))
```

```
(check-expect (count-from 3 5) (list 3 4 5 6 7))
```

The framework is as follows:

```
(define count-from
  (lambda (from count) ...))
```

Since `count` is a natural number, we can apply the corresponding template:

```
(define count-from
  (lambda (from count) (cond
    ((zero? count) ...)
    ((positive? count)
     ... (count-from ... (- count 1)) ...))))
```

In the first case, zero elements are required, so the result must be `empty`. Since a list is returned, we will use ``cons`` in the second case to construct it. Furthermore, the desired first element is precisely ``from``. We can therefore further develop the template as follows:

```
(define count-from
  (lambda (from count) (cond
    ((zero? count) empty)
    ((positive? count) (cons
      from
      ... (count-from ... (- count 1)) ...))))
```

For the remainder of the output list, we need a list that starts with the next number, i.e., with `(+ from 1)`. If we use this as the first argument of the recursive call, it is already exactly the required remainder list:

```
(define count-from
  (lambda (from count)
    (cond
      ((zero? count) empty) ((positive?
      count) (cons from
                    (count-from (+ from 1) (- count 1)))))))
```

Now ``count-from`` is finished, but our actual function ``between`` isn't. However, we can use ``count-from`` to implement ``between`` :

```
(define between
  (lambda (from to)
    (count-from from (+ (- to from) 1))))
```

Complete!

Perhaps it seems cumbersome to you to first remodel the task into a task about natural numbers and then to process them. Isn't there a "direct" way? Yes, there is.

The key to the direct solution lies in the realization that the natural number in the problem is precisely the difference between "from" and "to" . We can formulate a corresponding branch as follows:

```
(define between
  (lambda (from to)
    (cond
      ((= from to) ...)
      (< from to)
      ... (between ... ...) ...))))
```

For the first branch, we need a single-element list. In the second, it again makes sense to write ``cons`` first , because we know that the first element is ``from`` :

```
(define between
  (lambda (from to) (cond
    ((= from to) ...)
```

```
((< from to)
  (cons from (between ... ...))))))
```

Now we can add the arguments to the recursive call. We need a list containing starts with the next number and ends exactly as before:

```
(define between
  (lambda (from to)
    (cond
      ((= from to) (list from))
      ((< from to)
       (cons from (between (+ from 1) to))))))
```

Done! You might notice during this task that it's easy for all the numbers to "shift by one". For example, one could easily get the idea to...

Instead of simply writing `(list from)`, it's advisable to write `empty`. Therefore, it's useful to do this when programming. to carefully examine whether functions involving natural and integers are correct.

7.4 EXCURSUS: OPTIMIZING POTENCY

natural-numbers/power.rkt

code

We will return to the concept of exponents once more in this section. With a trick, we can...

We can significantly reduce the computational effort: For example, we can 3^{10} Calculate as follows:

$$3^{10} = 3^5 \times 3^5$$

So we don't need to laboriously multiply 3 by itself ten times; it's enough to simply multiply 3^5 to be-
to calculate and multiply by itself. The trick also works with odd exponents, we
But we have to multiply by the base once more:

$$3^9 = 3^4 \times 3^4 \times 3$$

We can also turn this trick into a program. For simplicity, we'll call the function power2, but otherwise it should work exactly like power. Here's the framework:

```
(define power2
  (lambda (base exponent)
    ...))
```

In `power2`, the number is no longer counted directly, even though the exponent is still a natural number. Because the number is not counted, we cannot (again) directly use the template for natural numbers. However, we can assume an alternative data definition for natural numbers that halves the numbers instead of referring to the predecessor (as in the first definition).

A natural number is:

; - 0

; - the double of a natural number ; - the successor of the double of a natural number

Since the data definition has three cases, the body must consist of a branching with three branches:

```
(define power2
  (lambda (base exponent)
    (cond
      ((zero? exponent) ...) ((even?
        exponent) ...) ((odd?
        exponent) ...))))
```

The first case corresponds to the original power function – a 1 must be entered there. The second and third cases each contain a self-reference, which is why recursive calls are necessary there.

In the second case, we have "double," so we need to halve. In the third case, we need to subtract one before halving. We use `quotients` in each case because we are halving whole numbers.

```
(define power2
  (lambda (base exponent) (cond
    ((zero? exponent) 1) ((even?
    exponent) ... (power2
    base (quotient exponent 2)) ...) ((odd? exponent)
    ... (power2 base (quotient (- exponent 1) 2)) ...))))
```

We will now address the second case: If b is the base and e is the exponent, then the recursive call yields the following result:

$$b^{\frac{e}{2}}$$

Wanted

$$b^e = b^{\frac{e}{2} \times 2} = b^{\frac{e}{2}} \times b^{\frac{e}{2}}.$$

For this we need $b^{\frac{e}{2}}$. Multiplying 2 by itself. To prevent this number from being calculated twice, we create a local definition for it:

```
(define power2
  (lambda (base exponent) (cond
    ((zero? exponent) 1) ((even?
    exponent) (define half
    (power2 base (quotient exponent 2)))
    (* half half)
    ((odd? exponent)
    ... (power2 base (quotient (- exponent 1) 2)) ...))))
```

In the third case, the following applies:

$$b^e = b^{\frac{e-1}{2}} \times b^{\frac{e+1}{2}}$$

Accordingly, we can fill in the last branch:

```
(define power2
  (lambda (base exponent)
    (cond
      ((zero? exponent) 1) ((even?
      exponent)
      (define half (power2 base (quotient exponent 2))) (* half half) ((odd?
      exponent)
      (define half (power2 base (quotient (- exponent 1) 2))) (* half half base))))))
```

Task 7.3

See for yourself with the stepper that `power2` requires fewer multiplications than `power!` ÿ

TASKS

Task 7.4

Write a function that calculates the *factorial* of a number $n!$ It is defined as:

$$n! \stackrel{\text{def}}{=} 1 \times 2 \times \dots \times n$$

Task 7.5

Program the following functions on lists: 1. Program a

function `take` that accepts a list `l` and a number `n` as arguments and returns the first `n` elements of the list `l`. Example:

```
(take (list 1 2 3 4 5 6 7 8) 5)
⇒ #<list 1 2 3 4 5>
```

Assume that `l` is at least `n` elements long.

2. Program a function `drop` that accepts a list `l` and a number `n` as arguments, and returns the list `l` without the first `n` elements. Example:

```
(drop (list 1 2 3 4 5 6 7 8) 3)
⇒ #<list 4 5 6 7 8>
```

Assume that `l` is at least `n` elements long.

Task 7.6

- The built-in function `even?` accepts an integer and returns `#t` if it is even and `#f` if it is not. Using `even?`, write a function called `evens` that returns a list of the even numbers between `a` and `b` for two numbers `a` and `b`:

```
(evens 1 10)
⇒ #<list 2 4 6 8 10>
```

- The built-in function `odd?` accepts an integer and returns `#t` if it is odd and `#f` if it is not. Using `odd?`, write a function `odds` that, for two numbers `a` and `b`, returns a list of the odd numbers between `a` and `b`:

```
(odds 1 10)
⇒ #<list 1 3 5 7 9>
```


8 EXCURSUS: INDUCTIVE EVIDENCE AND DEFINITIONS

In previous chapters, we frequently discussed self-references in data definitions and the recursive functions that accept them. This chapter deals with the mathematical aspects of such functions. It is recommended reading if you are particularly interested in this topic.

If you are interested in mathematics, or need to be for your studies. For understanding

However, you can also omit it from the following chapters.

Mathematics represents self-referential data as so-called *inductively defined* sets.

This allows us to verify the correctness of a function using a so-called inductive proof. This chapter explains what that is and how it works.

8.1 QUANTITIES

Before we discuss inductively defined sets, let's first talk about sets in general. We'll use the concept of sets from so-called "naive set theory," which avoids a rigorous mathematical formalization but is sufficient for our purposes. Our main focus is on the notation for working with sets, which we'll use in later sections. You may already be familiar with all of this. If so, feel free to skip this section or just skim it.

In naive set theory, the basic assumption is that the world consists of "objects": these can be anything, real things as well as imagined ones. If you now pick out some of these objects, i.e., a kind of collection of objects, then you have a set.

Important: For every set and every object, you must have a clear criterion that states whether the object is in the set or not. A set is also an object. Therefore, you can define a set whose elements are themselves sets.¹ Another important point regarding the concept of a set: The elements of a set are always distinct; an element can only be contained once in a set.

The objects of a set M are called *elements* of M . The notation $x \in M$ means that x to be an element of M , $x \notin M$, that x is not an element of M .

We often deal with sets of numbers: N denotes the set of *natural numbers*, corresponding to the signature [natural](#) (see page 34). Z denotes the set of

¹ If you pursue this line of thought further, you quickly arrive at contradictions. The set of all sets, for example, contains itself. But what about the set of all sets that *do not* contain themselves—does that set also contain itself? The classic book *Gödel, Escher, Bach* [Hof79] makes great entertainment out of such questions.

Integers are whole numbers, corresponding to the signature *integer*. $\mathbb{R}$ is the set of *real numbers*; its signature is *real*.

We write finite sets, that is, sets with a finite number of elements, as a list of their elements enclosed in curly brackets:

$$M = \{11, 13, 17, 19\}.$$

However, sets are often defined by a specific property that their elements possess. This can also be done with curly brackets and a vertical bar. Here is an example:

$$M = \{x \mid x \text{ is prime, } 10 \leq x \leq 20\}.$$

The *empty set* is the set that has no elements and is denoted by $\emptyset$.

A is called a *subset* of B , written $A \subseteq B$, if every element of A is also an element of B .

Two sets are equal if they have the same elements. This is the case if one set is a subset of the other and vice versa.

$$A \subseteq B \text{ and } B \subseteq A$$

This results in, for example, the following equation.

$$\{11, 13, 17, 19\} = \{17, 13, 19, 11\}$$

The order in which the elements are written within the curly brackets is therefore irrelevant. Furthermore, it doesn't matter if an element appears multiple times within the curly brackets. As stated above, each element is contained in a set only once or not at all.

$$\{11, 13, 17, 19\} = \{11, 13, 11, 17, 17, 11, 13, 19\}$$

The notation $A \not\subseteq B$ means that $A \subseteq B$ is not true, and $A \not= B$ means that $A = B$ is not true. A is called a *proper subset* of B if $A \subseteq B$, but $A \not= B$. The notation for this is $A \subset B$. It means $B \not\subseteq A$ that $A \not\subseteq B$, and likewise for $B \subset A$.

The *union* $A \cup B$ of two sets A and B is defined as follows:

$$A \cup B = \{a \mid a \in A \text{ or } a \in B\}$$

The *intersection* $A \cap B$ of two sets A and B is as follows:

$$A \cap B = \{a \mid a \in A \text{ and } a \in B\}$$

The *difference* $A \setminus B$ between two sets A and B is defined as follows:

$$A \setminus B = \{a \mid a \in A \text{ and } a \notin B\}$$

The *Cartesian product* $A \times B$ of two sets A and B is defined by

$$A \times B = \{(a, b) \mid a \in A, b \in B\}$$

The elements of the Cartesian product – pairs consisting of one element each from A and B – are written with parentheses around them and a comma in between, and are also called *tuples*.

For example, if $A = \{1, 2, 3\}$ and $B = \{4, 5\}$, then:

$$A \times B = \{(1, 4), (2, 4), (3, 4), (1, 5), (2, 5), (3, 5)\}$$

Tuples are therefore the mathematical counterpart to composite data.

You might be wondering why it's called a "product," and why the symbol $\times$ is the same as for the product of two numbers. Well, if A and B are finite sets, meaning A consists of m elements and B of n elements, then $A \times B$ consists of $m \times n$ elements. Therefore, in the example above, the product has $3 \times 2 = 6$ elements.

Tuples are not only binary, but can also have any number of digits: 3-tuples, 4-tuples, and so on.

8.2 STATEMENTS ABOUT NATURAL NUMBERS

In mathematics, there are many curious statements about natural numbers. The Gaussian summation formula, for example, has legendary status. It states that for a natural number n , the sum of all numbers from 1 to n can be expressed by the formula...

$$\frac{n \times (n + 1)}{2}$$

It can be calculated: There's no need to laboriously add the numbers individually. Mathematicians love special notations and could write Gauss's claim as follows:

$$\forall n \in \mathbb{N} : \sum_{i=0}^n \frac{n \times (n + 1)}{2} = 2$$

The $\forall$ means "for all", the $\in$ means "element of", and $\mathbb{N}$ is the abbreviation for the set of natural numbers. So, at the beginning, we have "for all n that are elements of $\mathbb{N}$ " or, more simply, "for all natural numbers n ".

This is a Greek sigma and stands for "sum". The $i = 0$ below and the n above mean that a new variable is introduced, which successively takes the values $0, 1, 2, \dots, n$, and for each value is substituted into the formula to the right of the $\sum$.

In Since i is only on the right side of $\sum$, this means $0 + 1 + 2 + \dots + n$. The Gaussian summation formula therefore states that this sum always yields the same result as the formula $\frac{n \times (n+1)}{2}$.

How can we prove Gauss's summation formula? Gauss's argument was that if he writing out the sum (we omit the 0):

$$1 + 2 + \dots + (n - 1) + n$$

...the two "outer" summands 1 and n together yield $n+1$, the two "next inner" summands 2 and $n-1$ also yield $n+1$, and so on until the "middle"—effectively, $n + 1$ is added to itself half as many times as the series itself is long. It is therefore easy to understand how Gauss arrived at the formula and why it is correct.

But what about the following series?

$$\sum_{i=0}^n i^2$$

Looking at the written form doesn't directly help:

$$1 + 4 + 9 + 16 + \dots + (n - 1)^2 + n^2$$

However, it is worthwhile to take a look at the first few terms of the series and to place them in tabular form above the Gaussian sums:

n		0	1	2	3	4	5	6	...
$\sum_{i=0}^n i$	$2i$	0	1	5	14	30	55	91	...
	0	1	3	6	10	15	21	...	
	$i=0$								

If you stare at the pairs of sums in the two series long enough, you might see that they can all be reduced to fractions with a denominator of 3:

n	0	1	2	3	4	5	6	...
$\frac{\sum_{i=0}^n i^2}{\sum_{i=0}^n i}$	?	$\frac{3}{3}$	$\frac{5}{3}$	$\frac{7}{3}$	$\frac{9}{3}$	$\frac{11}{3}$	$\frac{13}{3}$	...

The only exception is the fraction for 0: there, it is divided by 0, so it is unclear which fraction should be in that position in the table. Otherwise, the table suggests the following formula:

$$\frac{\sum_{i=0}^n i^2}{\sum_{i=0}^n i} = \frac{2n+1}{3}$$

We can solve the equation with $\sum_{i=0}^n i = n(n+1)/2$:

$$\sum_{i=0}^n i^2 = \frac{n(n+1)(2n+1)}{6} \quad (8.1)$$

Nice formula – but does it hold true for all $n \in \mathbb{N}$? (It holds true for the unclear case 0.) You can try it for others, for example 7, 8, ..., and it always gives the correct result. But that's not enough to prove the claim for all $n \in \mathbb{N}$.

If the statement is true for all $n \in \mathbb{N}$, and therefore also for a specific $n \in \mathbb{N}$, then it should also apply to $n+1$, which would then be the following equation, where $n+1$ was substituted for n in each case compared to the equation above:

$$\sum_{i=0}^{n+1} i^2 = \frac{(n+1)(n+1+1)(2(n+1)+1)}{6}$$

We can simplify this somewhat:

$$\sum_{i=0}^{n+1} i^2 = \frac{(n+1)(n+2)(2n+3)}{6} \quad (8.2)$$

In the series $\sum_{i=0}^{n+1} i^2$ Can we isolate the last addend and thus simplify the equation?

Write it as follows:

$$\left(\sum_{i=0}^n i^2 \right) + (n+1)^2 = \frac{(n+1)(n+2)(2n+3)}{6} + (n+1)^2$$

This provides the opportunity to subtract the respective sides of equation 8.1 from the sides of equation 8.2 :

$$\sum_{i=0}^n (i^2 + (n+1)2) - \sum_{i=0}^n i^2 = \frac{(n+1)(n+2)(2n+3)}{6} - \frac{n(n+1)(2n+1)}{6}$$

It remains:

$$= 6 \quad \frac{(n+1)(n+2)(2n+3)}{6} - \frac{(n+1)2}{6} - \frac{n(n+1)(2n+1)}{6}$$

If this equation is true, then equation 8.2 is also true. This can be calculated:

$$\begin{aligned} \frac{(n+1)(n+2)(2n+3)}{6} - \frac{n(n+1)(2n+1)}{6} &= \frac{(n+1)(n+2)(2n+3) - n(n+1)(2n+1)}{6} \\ &= \frac{(n+1)((n+2)(2n+3) - n(2n+1))}{6} \\ &= \frac{(n+1)(2n^2 + 3n + 4n + 6 - 2n^2 - n)}{6} \\ &= \frac{(n+1)(6n+6)}{6} \\ &= \frac{6(n+1)2}{6} \\ &= (n+1)2 \end{aligned}$$

Indeed! But what exactly was shown? It's easy to lose track with all the steps above. Here's a summary: It *seemed* as if the following equation was correct:

$$\sum_{i=0}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

The following equation is correct :

$$= 6 \quad \frac{(n+1)(n+2)(2n+3)}{6} - \frac{(n+1)2}{6} - \frac{n(n+1)(2n+1)}{6}$$

If the first equation ("seemed so") were correct, then the following equation would follow:

$$\sum_{i=0}^{n+1} i^2 = \frac{(n+1)(n+2)(2n+3)}{6}$$

This is the same claim, only for $n + 1$ instead of n – in other words, the claim for n implies the claim for $n + 1$. Since we have already shown above by calculation,

Since the statement holds for 1, ..., 6, it also holds for 7. Since it holds for 7, it also holds for 8. And so on for all natural numbers. We don't need to test this for each number individually.

The assumption made above is therefore proven.

8.3 Providing Inductive Evidence

The proof principle used in the previous section for proofs of theorems of the form «for all $n \in \mathbb{N}$. . . » is called *mathematical induction*.² This works for problems where a claim

The proof must be completed for all $n \in \mathbb{N}$. The following ingredients are needed for the proof:

1. a proof that the claim is true for $n = 0$ and
2. a proof that if the claim holds for any n , it also holds for $n + 1$.

The first ingredient (the *base case of the induction*) can usually be proven by substitution, for the second

The ingredient (the *inductive step*) usually requires algebra. Since the claim depends on ingredient number 1...

If it applies to 0, then according to ingredient no. 2 it must also apply to 1, and therefore also to 2, ... and thus to all natural numbers.

It does not necessarily have to start at 0, but can also begin at any number a – but then the claim only applies to all natural numbers from a onwards.

Once the claim is formulated, inductive proofs are often easy to construct, as they usually follow the scheme described above. The most important thing is to actually follow the scheme. to adhere to this. Therefore, it is recommended that you follow these instructions – a kind of construction guide for proofs using complete induction:

1. Formulate the claim to be proven as a claim of the form «For all $n \in \mathbb{N}$, the following holds: . . . », if that hasn't already been done.
2. Write the heading " $n = 0$:". Rewrite the statement, this time replacing the previous one.
Omit «for all $n \in \mathbb{N}$ » and substitute 0 for n .
3. Prove the copied claim. (This is often easy.)
4. Write the phrase "Induction hypothesis:" and then write the statement underneath it.
Take it apart, leaving out the "for all $n \in \mathbb{N}$ " part – leave the n where it is.
5. Write "Inductive step (to show):". Below that, write the claim again.
from, where you substitute $(n + 1)$ for n instead. (Remove the "for all $n \in \mathbb{N}$ ".)

² Not to be confused with *philosophical induction*. While complete induction is related, from a philosophical perspective it is more of a deductive technique.

6. Prove the inductive inference using the induction hypothesis.

If the claim is an equation of the form $A = B$, you can often insert the induction hypothesis directly or after some transformations into the induction step.

We will go through the instructions using the example above. The claim is:

$$\sum_{i=0}^n 2i = \frac{n(n+1)(2n+1)}{6}$$

The claim arose through guesswork – unfortunately, there's no magic formula for finding such equations:

Experimentation and patience are required. Once the claim is established, however, it's quite easy to prove:

1. When written out, the statement already has the correct form:

$$\forall n \in \mathbb{N} : \sum_{i=0}^n 2i = \frac{n(n+1)(2n+1)}{6}$$

2. $n = 0$:

$$\sum_{i=0}^0 2i = \frac{0(0+1)(2 \times 0 + 1)}{6}$$

3. Both sides of the equation are 0, so it is proven.

4. Induction hypothesis:

$$\sum_{i=0}^n 2i = \frac{n(n+1)(2n+1)}{6}$$

5. Inductive step (to be shown):

$$\sum_{i=0}^{n+1} 2i = \frac{(n+1)((n+1)+1)(2(n+1)+1)}{6}$$

6. We can divide the sum on the left side:

$$\left(\sum_{i=0}^n 2i \right) + 2(n+1) = \frac{(n+1)((n+1)+1)(2(n+1)+1)}{6} + (n+1)2$$

This allows us to use the induction hypothesis, in which $\sum_{i=0}^n 2i$ 2 on the left side i

This is true. It almost always works for inductive proofs using sums or products:

$$\frac{n(n+1)(2n+1)}{6} + (n+1)2 = \frac{(n+1)((n+1)+1)(2(n+1)+1)}{6}$$

We can simplify the left side as follows:

$$\begin{aligned}
 & \underline{(2n + 1) + (n + 1)2 = 6} & \underline{n(n + 1)(2n + 1) + 6(n + 1)2 \ n(n + 1)} \\
 & & = \underline{6 \ (n + 1)(n(2n + 1) + 6(n + 1)) \ 6 + n +} \\
 & & = \underline{(n + 1)(2n^2 + 6n + 6) \ 6 + 7n + 6)} \\
 & & = \underline{(n + 1)(2n^2 + 6n + 7)}
 \end{aligned}$$

We can simplify the right side as follows:

$$\begin{aligned}
 & \underline{(n + 1)((n + 1) + 1)(2(n + 1) + 1) \ 6} & = & \underline{(n + 1)(n + 2)(2(n + 1) + 1) \ 6 \ (n + 1)(n} \\
 & & = & \underline{+ 2)(2n + 2 + 1) \ 6} \\
 & & = & \underline{(n + 1)(n + 2)(2n + 3) \ 6 \ 2} \\
 & & = & \underline{(n + 1)(2n^2 + 4n + 3n + 6) \ 6} \\
 & & = & \underline{(n + 1)(2n^2 + 4n + 3n + 6)} \\
 & & = & \underline{(n + 1)(2n^2 + 7n + 6)}
 \end{aligned}$$

Thus, the same result was obtained on both the left and right sides, and the claim is proven.

8.4 STRUCTURE OF NATURAL NUMBERS

The mathematical induction from the previous section is only suitable for the set of natural numbers. It doesn't work for all sets: for the real numbers $\mathbb{R}$, for example, the construction "starting at 0 and then counting up by 1 each time" simply doesn't reach all elements of the set. The natural numbers are therefore something special. We already saw this in Chapter 7: they are numbers that count, and a data definition can be found on page 205.

A natural number is one of the following: - 0 - the successor of a natural number

We can interpret this data definition as a set of instructions for constructing the set of natural numbers: This data definition encompasses every natural number – every number can be written in the form $0 + 1 + \dots + 1$. This type of set-building instruction is characterized by the presence of one or more *basic elements* (in this case, 0) and then one or more rules that construct "larger" elements from arbitrary "smaller" elements using a self-reference, in this case the rule stating that for every natural number, its successor is also a natural number.

Here is the mathematical equivalent of the data definition of natural numbers:

Definition 8.1 (natural numbers)

The set of natural numbers N is defined as follows:

1. $0 \in N$
2. For $n \in N$, $n + 1 \in N$ as well.
3. The above rules define *all* $n \in N$.

Such a definition is also called an *inductive definition*, which defines an *inductive set*. The last clause is always included in inductive definitions – without it, for example, the set of real numbers could be considered N , because the preceding clauses could not claim to be complete. This clause is called *the inductive closure*.

If you look closely, you'll see that the inductive definition has exactly the same structure as the definition of induction on page 225: There's a clause for 0 and a clause for the step from n to $n + 1$. The inductive definition states that there are two different kinds of natural numbers: the base element 0 from the first rule of the definition and the (positive) numbers $n + 1$ from the second clause. This structure also corresponds to the data definition and the template from Construction Guide 19 on page 208.

Accordingly, a claim about the natural numbers must be proven for both types:

One clause applies to the base element 0, and the other to the positive numbers, which are not base elements. The clauses for the base elements are called *inductive anchors*. Since the second clause contains a *self-reference* —a natural number must already exist in order to find another—an inductive step must be proven there.

The natural numbers have only one base element, and there is only one self-referencing clause. Section 5.3 on page 5.3 dealt with financial contracts – there was more than one self-referencing clause there, and we will see another such example in this chapter.

8.5 FINAL EPISODES

The natural numbers are not the only inductively defined set. Another example is...

The *finite sequences* that correspond to the lists. In programming, we use the notation (list-of s) to write the signature for lists consisting of values of the signature ss.

exist. The mathematical equivalent is the notation M^* , the number of episodes with Elements from the set M.

Definition 8.2

Let M be an arbitrary set. The set M^* of *finite sequences over M* is defined as follows:

1. There exists an *empty sequence* $\epsilon \in M^*$.
2. If $f \in M^*$ is a sequence and $m \in M$, then $mf \in M^*$, so also a consequence.
3. The above rules define *all* $f \in M^*$.

A sequence is thus created from an existing sequence by appending an element to the beginning. Sequences over $M = \{a, b, c\}$ are therefore approximately

$\epsilon, a\epsilon, b\epsilon, c\epsilon, aa\epsilon, ab\epsilon, ac\epsilon, \dots, abc\epsilon, \dots, cbba\epsilon, \dots$ (not sorted alphabetically)

Since ϵ is always part of non-empty sequences, it is often not written down.

The definition of finite sequences is analogous to definition 8.1: The first clause is the *basic* case, the second clause contains a *self-reference*, and the third clause forms the inductive closure.

8.5.1 FUNCTIONS ON FINITE SEQUENCES

This section demonstrates how mathematical functions are formulated on finite sequences. will be. As an example, a function called s is required that calculates the sum of a sequence. For example:

$$s(1\ 2\ 3\ \epsilon) = 6$$

$$s(2\ 3\ 5\ 7\epsilon) = 17$$

$$s(17\ 23\ 42\ 5\ \epsilon) = 87$$

³ This Greek letter ϵ is a lowercase epsilon, often used in mathematics for things that are 'empty' or are "small".

The function thus corresponds to list-sum from section 6.1. Since there are two different types of finite sequences – the empty sequence and the “non-empty” sequences – it makes sense to write the corresponding function with a branch that distinguishes between the two types:

$$s(f) \stackrel{\text{def}}{=} \begin{cases} 0 & \text{if } f = \epsilon \\ f & \text{if } f = mf', m \in M, f' \in M^* \end{cases}$$

This definition has a branching with two branches. The first branch applies if f is the empty sequence. The second branch applies accordingly if f is not empty. This is expressed as $f = mf'$, which means that f has the *form* mf' , i.e., it consists of a first element m and a remainder sequence f' . So, for example, if f is the sequence $abcd$, then $m = a$ and $f' = bcd$.

The places marked with question marks are still missing. The sum of the empty sequence is 0: If it were a different number $m \neq 0$, the sum of any sequence could be changed by appending the empty sequence by m . The first gap has therefore already been filled.

$$s(f) \stackrel{\text{def}}{=} \begin{cases} 0 & \text{if } f = \epsilon \\ f & \text{if } f = mf', m \in M, f' \in M^* \end{cases}$$

In the second case, f is a composite object with the components m and f' .

Therefore, $s(f) = m + s(f')$. The following can be used for the construction of the function value:

$$s(f) \stackrel{\text{def}}{=} \begin{cases} 0 & \text{if } f = \epsilon \\ m + s(f') & \text{if } f = mf', m \in M, f' \in M^* \end{cases}$$

So far, the known techniques for constructing functions on mixed and composite data have been applied, merely transferring program functions to mathematical functions.

For the next step, a recursive call to self-reference is appropriate – just as in programming. The function definition can be summarized as follows:

$$s(f) \stackrel{\text{def}}{=} \begin{cases} 0 & \text{if } f = \epsilon \\ m + s(f') & \text{if } f = mf', m \in M, f' \in M^* \end{cases}$$

⁴ Perhaps the apostrophe in “f’” is confusing you. ⁵ Because in school calculus it stands for “derivative”. In this context, it’s just another variable, pronounced “f prime”.

Here, $s(f)$ of $\bar{y}$ is the sum of all sequence elements in f of $\bar{y}$. The question is: what is the sum of all sequence elements? $f = mf\bar{y}$. Therefore, only m itself is missing for the sum:

$$s(f) \stackrel{\text{def}}{=} \begin{cases} \bar{y} \cdot 0 & \text{if } f = \bar{y} \\ \bar{y} \cdot m + s(f \bar{y} \bar{y}) & \text{if } f = mf\bar{y}, m \bar{y} M, f\bar{y} \bar{y} M\bar{y} \end{cases}$$

Using paper and pencil, we can quickly verify that the definition works correctly:

$$\begin{aligned} s(17 \ 23 \ 42 \ 5 \ \bar{y}) &= 17 + s(23 \ 42 \ 5 \ \bar{y}) = \\ &= 17 + 23 + s(42 \ 5 \ \bar{y}) = 17 \\ &+ 23 + 42 + s(5 \ \bar{y}) = 17 + \\ &23 + 42 + 5 + s(\bar{y}) \\ &= 17 + 23 + 42 + 5 + 0 \\ &= 87 \end{aligned}$$

The definition of s thus calls itself into question at the point where the inductive definition of finite sequences defines the self-reference $\bar{y} \cdot M\bar{y}$ contains a sequence».

You might be skeptical of the definition of s , since s appears on both the left and right sides. It looks as if s is "self-defined." However, this isn't actually a problem because:

- $s(f)$ always ends up on a *shorter* sequence f • $\bar{y}$ calls up, and eventually at the empty sequence where the branching occurs and no further re-
no further action is taken with the italicized call.

As long as a recursive function follows the scheme of s and thus the structure of the sequences themselves, these two conditions are automatically fulfilled.

8.5.2 Follow-up Induction

The counterpart to mathematical induction, when applied to sequences, is called *sequential induction*. In sequential induction, the "inference from n to $n + 1$ " becomes an inference from f to mf for all sequences f and all $m \bar{y} M$. Often, the sequence element m is irrelevant.

To prove that a claim holds for all $f \bar{y} M\bar{y}$, the following steps suffice:

1. The claim holds for $f = \bar{y}$ (*base case of induction*)
2. If the statement holds for a sequence f , then it also holds for all sequences mf where $m \bar{y} M$. (*Inductive circuit*).

Sequence induction works because it follows the structure of the definition of finite sequences (8.2) in the same way that mathematical induction follows the structure of the natural numbers.

The lemma can also be proven using mathematical induction – see exercise 8.8.

Corresponding to complete induction, there is also a guideline for sequential induction.

Task:

1. Formulate the claim to be proven as a claim of the form «For all $f \in M^*$, the following holds: . . . », if that hasn't already been done.
2. Write the heading « $f = \tilde{y}$ ». Rewrite the statement, this time replacing the previous one. omits «for all $f \in M^*$ » and substitutes $\tilde{y}$ for f .
3. Prove the copied claim. (This is often easy.)
4. Write the phrase "Induction hypothesis:" and then write the statement underneath it. Let's remove the part "for all $f \in M^*$ " – leave the f where it is.
5. Write "Inductive step (to show):". Below that, rewrite the statement, substituting mf for f . (Omit "for all $f \in M^*$ ".)
6. Prove the inductive step using the induction hypothesis. Remember to prove the claim for *all* $m \in M$ – this is usually, but not always, easy.

If the claim is an equation of the form $A = B$, you can often substitute the induction hypothesis into the inductive step.

A useful example is the function `cat`, which concatenates sequences:

$$\text{cat}(f1, f2) \stackrel{\text{def}}{=} \begin{cases} f2 & \text{if } f1 = \tilde{y} \\ m \text{ cat}(f \tilde{y}1, f2) & \text{if } f1 = m\tilde{y} \end{cases} \quad 1, m \in M, \tilde{y} \in M^*$$

It should be proven that cat is associative, that is, for all $u, v, w \in M^*$ the following holds:

$$\text{cat}(u, \text{cat}(v, w)) = \text{cat}(\text{cat}(u, v), w).$$

1. The form of the claim is still problematic, as it mentions three sequences – and none of them is called f . In the worst case, you have to guess which sequence the induction is based on and possibly try all possibilities. We choose the first sequence u and rename it f so that the proof fits the instructions. Furthermore, when applying the definition of cat for $f1$, we have to substitute this f .

$$\forall f \in M^* \forall v, w \in M^* : \text{cat}(f, \text{cat}(v, w)) = \text{cat}(\text{cat}(f, v), w)$$

2. $f = y$: Here the following applies

$$\text{cat}(\tilde{y}, \text{cat}(v, w)) = \text{cat}(v, w)$$

$$\text{cat}(\text{cat}(\tilde{y}, v), w) = \text{cat}(v, w)$$

according to the defining equation.

3. Induction hypothesis:

$$\text{cat}(f, \text{cat}(v, w)) = \text{cat}(\text{cat}(f, v), w)$$

4. Inductive step (to be shown):

$$\text{cat}(mf, \text{cat}(v, w)) = \text{cat}(\text{cat}(mf, v), w)$$

5. We use the definition of cat to further calculate the left side:

$$\text{cat}(mf, \text{cat}(v, w)) = \text{mcat}(f, \text{cat}(v, w))$$

But here we have $\text{cat}(f, \text{cat}(v, w))$ – and that's also on the left-hand side of the induction hypothesis. So we can substitute:

$$= \text{mcat}(\text{cat}(f, v), w)$$

We can also calculate the right side:

$$\begin{aligned} \text{cat}(\text{cat}(mf, v), w) &= \text{cat}(\text{mcat}(f, v), w) = \\ &\quad \text{mcat}(\text{cat}(f, v), w) \end{aligned}$$

But that's the same result that came out on the left side – the proof is complete.

8.6 NOTATION FOR INDUCTIVE DEFINITIONS

Inductive sets are extremely common in computer science—so common, in fact, that a shorthand notation for their inductive definitions has become established: the so-called *context-free grammar*. In most computer science books, the term is accompanied by a lengthy mathematical definition, which we don't need in its entirety for the purposes of this book. We use the context-free grammar (hereafter simply "grammar") informally as an abbreviation for a

A more detailed inductive definition. Here is the grammar for the natural numbers:

$$n \in \mathbb{N} \iff 0 \mid n \in \mathbb{N} + 1$$

In grammar, $n \in \mathbb{N}$ stands for "a natural number". You can read the grammar like this: A natural number is either 0 or has the form $n + 1$, where n is again a natural number.

The obligatory inductive conclusion ("The above rules define *all* $n \in \mathbb{N}$.") is implicitly assumed and therefore omitted.

The symbol $\in$ can therefore be read as "is" or "has the form", and the symbol $\mid$ as "or". The notation $\in X$ defines a corresponding set X . The grammar is thus a kind of mathematical notation for composite data, mixed data ($()$), and self-references, but specifically for mathematical objects.

For the set of sequences over a set M , the following notation suffices:

$$s \in M^* \iff s \in M \mid s \in M^* M$$

In both definitions, the self-reference is clearly visible: In the case of natural numbers, $n \in \mathbb{N}$ appears in one clause, and in the case of sequences, $s \in M^*$. All other clauses – i.e., those without self-reference – describe basic cases.

8.7 Structural Recursion

Section 8.5 shows that the definition of the two example functions on sequences (s and cat) each "follows" the inductive definition of sequences – the definition of both functions has the form:

$$F(f, \dots) \stackrel{\text{def}}{=} \begin{cases} \dots & \text{if } f = 0 \\ \dots F(f \cdot y) \dots & \text{if } f = m \cdot y, m \in M, f \cdot y \in M^* \end{cases}$$

This is no coincidence: The recursive function definition belongs to the inductive set definition like pitch belongs to sulfur. Two basic rules determine this form:

1. For each clause, there is a branch with a branch of the inductive definition.
2. For self-references, there is a self-call in the corresponding branch.

Many operations with natural numbers can also be written recursively. For example, exponentiation:

$$b^n \stackrel{\text{def}}{=} \begin{cases} 1 & \text{if } n = 0 \\ b \cdot b^{n-1} & \text{if } n = n' + 1, n' \in \mathbb{N} \end{cases}$$

The notation $n = n + 1$. Although $+1$ follows the inductive definition, here it is equivalent to $n + 1$.
 $n > 0$ and n . Therefore, inductive definitions on natural numbers are usually expressed as follows:
 written:

$$b^n \stackrel{\text{def}}{=} \begin{cases} 1 & \text{if } n = 0 \\ b^{n-1} & \text{if } n > 0 \end{cases}$$

The correspondence between inductive definitions and recursive functions can also be formulated more generally. Suppose the set X is defined by a grammar that has n clauses:

$$X \stackrel{\text{def}}{=} C_1 \mid \dots \mid C_n$$

A function on this set then needs – just like with mixed data – a branching with n branches, one for each clause:

$$F(x) \stackrel{\text{def}}{=} \begin{cases} R_1 & \text{if } x = C_1 \\ \vdots & \\ R_n & \text{if } x = C_n \end{cases}$$

The condition $x = C_i$ is derived from the corresponding clause. If there are references to other sets or self-references, these are replaced by variables and corresponding set memberships. For example, suppose the clause C_i has this form:

$$[\text{AWAY}]$$

Then F_i would have to look like this:

$$x = [a \ \& \ b], \ a \in A, \ b \in B$$

A self-referential clause C_i would look like this, for example:

$$\{ X \}$$

The corresponding F_i would look like this:

$$x = \{ x \}, \ x \in X$$

Accordingly, the right-hand side R_i most likely contains a recursive call $F(x)$.

Functions of this form, which directly follow the structure of an inductive set, are called *structurally recursive*.

The following grammar describes simple propositional logic expressions, i.e., statements with "and", "or" and "not":

$$\begin{aligned} & \bar{y}E\bar{y} \bar{y} \bar{y} \mid \bar{y} \\ & \mid \bar{y}E\bar{y} \bar{y} \bar{y}E\bar{y} \\ & \mid \bar{y}E\bar{y} \bar{y} \bar{y}E\bar{y} \mid \\ & \neg \bar{y}E\bar{y} \end{aligned}$$

The idea is as follows: $\bar{y}$ stands for "true", $\bar{y}$ for "false", $\bar{y}$ for "and", $\bar{y}$ for "or", and $\neg$ for "not". Examples of expressions are:

$$\begin{aligned} & \bar{y} \\ & \bar{y} \bar{y} \bar{y} \\ & \bar{y} \bar{y} \bar{y} \\ & \neg \bar{y} \end{aligned}$$

The expression $\bar{y} \bar{y} \bar{y} \bar{y} \bar{y}$ is unclear because it lacks parentheses. In such cases, we write the expression with parentheses, just as in arithmetic:

$$\begin{aligned} & (\bar{y} \bar{y} \bar{y}) \bar{y} \bar{y} \bar{y} \bar{y} \\ & (\bar{y} \bar{y} \bar{y}) \end{aligned}$$

Each such expression has a truth value, that is, "true" or "false," and a structurally recursive function can calculate this truth value. This function is "called" $\bar{5}$ – for a

The expression e returns the value 1 if the expression is "true" and 0 if "false". The double square brackets are called "semantic brackets" in technical language and are often used to write functions on sets defined by a grammar.

As described above, the function definition definitely consists of a branching structure that has a branch for each clause of the grammar:

$$\begin{aligned} & \bar{y} \text{ ? if } e = \bar{y} \\ & \text{ ? if } e = \bar{y} \\ e \stackrel{\text{def}}{=} & \text{ ? if } e = e1 \bar{y} e2 \\ & \text{ ? if } e = e1 \bar{y} e2 \\ & \text{ ? if } e = \neg e \end{aligned}$$

⁵ Unfortunately, there is no elegant oral pronunciation for this.

In the branches for $\dot{\vee}$, $\dot{\wedge}$ and $\neg$, recursive calls are indicated because the grammar contains self-references there:

$$\begin{array}{ll}
 \dot{\vee} ? & \text{if } e = \dot{\vee} \\
 ? & \text{if } e = \dot{\vee} \\
 e \stackrel{\text{def}}{=} \begin{array}{l} \dots e1 \dots e2 \\ \dots e1 \dots e2 \\ \dots e \dot{\vee} \dots \end{array} & \begin{array}{l} \text{if } e = e1 \dot{\vee} e2 \\ \text{if } e = e1 \dot{\wedge} e2 \\ \text{if } e = \neg e \dot{\vee} \end{array}
 \end{array}$$

The first two cases return 1 and 0 respectively; for the subsequent cases, the following auxiliary functions are defined:

$$\begin{array}{ll}
 a(t1, t2) \stackrel{\text{def}}{=} \begin{array}{l} \dot{\vee} 1 \text{ if } t1 = 1 \text{ and } t2 = 1 \\ \dot{\vee} 0 \text{ otherwise} \end{array} & o(t1, t2) \stackrel{\text{def}}{=} \begin{array}{l} \dot{\vee} 0 \text{ if } t1 = 0 \text{ and } t2 = 0 \\ \dot{\vee} 1 \text{ otherwise} \end{array} \\
 n(t) \stackrel{\text{def}}{=} \begin{array}{l} \dot{\vee} 1 \text{ if } t = 0 \\ \dot{\vee} 0 \text{ if } t = 1 \end{array} &
 \end{array}$$

Function a returns 1 only if $t1$ and $t2$ are 1 (otherwise 0). Function o then returns 1.

The function n returns 1 if $t1$ or $t2$ is 1 (otherwise 0). The function n then returns 1 if t is *not* 1 (otherwise 0). This

Functions now complete the definition of:

$$\begin{array}{ll}
 \dot{\vee} 1 & \text{if } e = \dot{\vee} \\
 0 & \text{if } e = \dot{\vee} \\
 e \stackrel{\text{def}}{=} \begin{array}{l} a(e1, e2) \text{ if } e = e1 \dot{\vee} e2 \\ o(e1, e2) \text{ if } e = e1 \dot{\wedge} e2 \\ n(e \dot{\vee}) \text{ if } e = \neg e \dot{\vee} \end{array}
 \end{array}$$

8.8 Structural Induction

Proofs of properties of structurally recursive functions work – according to the natural numbers and their sequences – using *structural induction*. Structural induction follows the Grammar structure – complete induction and sequential induction are special cases. The following example builds on the propositional logic expressions from the previous section:

Theorem 8.3 From a propositional logic expression e , the expression $\bar{e}$ is obtained by replacing every $\neg$ with $\neg$, every $\wedge$ with $\vee$, every $\vee$ with $\wedge$, and every $\rightarrow$ with $\rightarrow$. This holds true for every expression e :

$$\bar{\bar{e}} = e$$

The claim already has the form suitable for an inductive proof, "for all e ," where e comes from an inductively defined set. The claim must now be proven for all possible cases of e —since the grammar for expressions has five cases, five cases must be proven: $e = \neg$, $e = \neg$, $e = e_1 \vee e_2$, $e = e_1 \wedge e_2$, and $e = \neg e$, where e_1 , e_2 and e in turn

Expressions are.

Here are the proofs for the cases $e = \neg$, $e = \neg$:

$\neg = 1$	$\neg = 0$
$\bar{\neg} = \neg$	$\bar{\neg} = \neg$
$= 0$	$= 1$
$= n(1)$	$= n(0)$

Next, we will consider the case $e = e_1 \vee e_2$. Since the clause

$$\neg E \vee \neg \dots \quad | \quad \neg E \vee \neg \neg E \vee$$

Since there are two self-references, there is also a two-part inductive hypothesis – the claim can be assumed for e_1 and e_2 : $e_1 = n(e_1)$, $e_2 = n(e_2)$. The inductive step for this looks like this:

$$\begin{aligned}
 e_1 \vee e_2 &= a(e_1, e_2) \\
 \overline{e_1 \vee e_2} &= \overline{e_1 \vee e_2} \quad \text{Definition of } \overline{} \\
 &= o(\overline{e_1}, \overline{e_2}) = \\
 &= o(n(e_1), n(e_2)) \text{ Induction assumption} = n(a(e_1, e_2))
 \end{aligned}$$

The final step results from a close examination of the definitions of o and a respectively by substituting all possible values for t_1 and t_2 .

The case $e = e_1 \wedge e_2$ follows analogously. Again, there are two self-references, so the two-part induction hypothesis $\bar{e}_1 = n(e_1)$, $\bar{e}_2 = n(e_2)$ holds again. Inductive step (to be shown):

$$e1 \dot{\vee} e2 = o(e1, e2)$$

$$\frac{}{e1 \dot{\vee} e2 = e1 \dot{\vee} e2} \quad \text{---} \quad \text{---}$$

Definition of ---

$$= a(e1, \overline{e2}) = \text{---}$$

$$a(n(e1), n(e2))$$

Induction prerequisite

$$= n(o(e1, e2))$$

Finally, the case $e = \neg e$ is still missing. $\dot{\vee}$. Here there is only a self-reference, accordingly the formula is =

Induction hypothesis: $e \quad \overline{\dot{\vee}} \quad n(e \quad \dot{\vee})$. Inductive step (to be shown):

$$\dot{\vee} \neg e = n(e \quad \dot{\vee})$$

$$\frac{}{\dot{\vee} \neg e} \quad \dot{\vee} \neg e \quad \text{---}$$

Definition of ---

$$= n(e \quad \overline{\dot{\vee}}) =$$

$$n(n(e \quad \dot{\vee}))$$

Induction prerequisite

This example shows that structurally inductive proofs can indeed have more than one "base case" – here the cases $\dot{\vee}$ and $\dot{\vee}$. Likewise, there is more than one inductive step – one for each clause with self-reference; here, that's three cases.

For proofs using structural induction, the following instructions apply: 1.

Formulate the claim to be proven as a claim of the form «For all $x \dot{\vee} X, \dots$ » (where X is an inductively defined set), if this has not already been done.

2. Now provide a proof for each individual clause C_i of the inductive definition: 3. Write the heading « $x = F_i$ », where F_i is a condition corresponding to the clause.

Rewrite the statement, omitting "for all $x \dot{\vee} X$ " and substituting F_i for x .

4. If the clause does not contain a self-reference, prove the claim directly.

5. If the clause contains one or more self-references, the heading should contain "Varia" or similar, which For each self- $\dot{\vee}$ are themselves elements of X . Then write the heading "Induk-reference", rewrite the statement, omitting "for all $x \dot{\vee} X$ " and substituting x_j or x for x .

y

Write the word "inductive reasoning" below and prove the claim. Remember to use the inductive hypothesis.

NOTES

The first constructive definition of the natural numbers was already given by Peano in 1889. suggested:

Definition 8.4 (Peano axioms)

The set $\mathbb{N}$ of natural numbers is given by the following properties, the *Peano axioms* :

1. There exists a natural number $0 \in \mathbb{N}$.
2. For every number $n \in \mathbb{N}$ there exists a number $n' \in \mathbb{N}$, which is called the *successor of n* .
3. For all $n \in \mathbb{N}$, $n' \neq 0$.
4. From $n' = m'$ implies $n = m$.
5. A set M of natural numbers, which contains 0 and for every number $m \in M$ also whose successor m' , is identical to $\mathbb{N}$.

This definition is equivalent to definition 8.1 on page 228. As in definition 8.1, $\mathbb{N}$ can be defined as Construct step by step from the Peano axioms. From the first and second axioms, it follows that there are natural numbers.

$$0, 0', 0'', 0''', 0'''', \dots$$

There is. Without the 0 at the beginning, the representation of natural numbers is created using lines.

Axioms 3 and 4 state that for every number there is only one single representation: every addition a line « $'$ » generates a completely new number. Without clause 4, one could theoretically access the

The idea arises that there are two different numbers that both have the same successor.

Although the Peano formulation is similar to Definition 8.1, it also has some interesting limitations. differences include:

- In everyday usage, the term « n' » is of course « $n + 1$ ».
- The third and fourth conditions together state that the successor function is injective,
This means that through continued application of the successor function, new elements are constantly being created.
mente.
- Finally, the fifth condition describes the *inductive conclusion*, which stipulates that, apart from
No further elements exist beyond those generated in this way. Axiom 5 is also called the *induction axiom*. In Definition 8.1, the induction axiom has the more common form: "The above
Rules define all $n \in \mathbb{N}$. A formulation like "... the *smallest* " is also common.
Set with the properties ... ».

TASKS

Task 8.1

Prove the Gaussian summation formula using complete induction:

$$\forall n \in \mathbb{N} : \sum_{i=0}^n \frac{n \times (n+1)}{2} = \frac{n \times (n+1)}{2}$$

Task 8.2

Consider the following table:

$$\begin{aligned} 1 &= 1 \\ 1 + 4 &= 1 + 2 \\ 1 + 4 + 9 &= 1 + 2 + 3 \\ 1 + 4 + 9 + 16 &= 1 + 2 + 3 + 4 \end{aligned}$$

Guess the equation underlying this table and write it down in mathematical notation.

Prove the equation!

Task 8.3

Prove that the following equation holds for all $n \in \mathbb{N}$:

$$\sum_{i=0}^n 3i = \sum_{i=0}^n i^2$$

Task 8.4

What is wrong with the following inductive proof for the claim "All horses are the same color"?

Base case: For an empty set of horses, the claim holds trivially.

Inductive reasoning : Given a set of $n + 1$ horses, remove one horse from the set. By induction, the remaining horses are the same color. Remove another horse from the set. Again, by induction, the remaining horses are the same color. Since the remaining horses could not have suddenly changed color in the meantime, they were the same color in both cases, and all $n + 1$ horses are this color.

Task 8.5

For Boolean variables A and B, the so-called *De Morgan's laws* apply :

$$\begin{aligned}\neg(A \vee B) &= \neg A \wedge \neg B \\ \neg(A \wedge B) &= \neg A \vee \neg B\end{aligned}$$

Show that the laws are valid by trying all possible values for A and B.

Task 8.6

Prove by induction that the following holds for sequences for $u, v \in M^*$:

$$\text{len}(\text{cat}(u, v)) = \text{len}(u) + \text{len}(v)$$

Here, len is the length of sequences, defined as:

$$\text{len}(f) \stackrel{\text{def}}{=} \begin{cases} 0 & \text{if } f = \epsilon \\ \text{len}(f^{\text{y}}) + 1 & \text{if } f = mf^{\text{y}} \end{cases}$$

Task 8.7

Proofs by sequential induction:

$$\begin{aligned}\text{cat}(v, w) &= \text{cat}(z, w) \vee v = z \text{ cat}(v, \\ w) &= \text{cat}(v, z) \vee w = z\end{aligned}$$

Task 8.8

Prove the correctness of the sequence induction from section 8.5.2 (page 231) using mathematical induction. To do this, describe how a statement about all sequences can be transformed into a statement about the lengths of the sequences. Then relate the clauses of mathematical induction to the corresponding clauses of sequence induction.

Task 8.9

Give an inductive definition of the colloquially defined function from section 8.8 (page 238) .

Task 8.10

Provide a data definition for the propositional logic expressions from Section 8.8. Program the function and the function.

Task 8.11

The Fibonacci function on the natural numbers is defined as follows:

$$\text{fib}(n) \stackrel{\text{def}}{=} \begin{cases} 0 & \text{if } x = 0 \\ 1 & \text{if } x = 1 \\ n - \text{fib}(n - 2) + \text{fib}(n - 1) & \text{else} \end{cases}$$

Prove that $\text{fib}(n)$ is the integer closest to $n / \sqrt{5}$, where $\sqrt{5} = (1 + \sqrt{5})/2$.

Instructions: Show that $\text{fib}(n) = (n - \sqrt{5}) / \sqrt{5}$, where $\sqrt{5} = (1 + \sqrt{5})/2$.

Task 8.12

A partial order $\preceq$ on a set M is called *well-founded* or *Noetherian* if there are no infinite sequences $(x_i)_{i \in \mathbb{N}}$ such that for all $i \in \mathbb{N}$, the following holds:

$$x_{i+1} \preceq x_i \text{ and } x_{i+1} \neq x_i$$

also written as:

$$x_{i+1} \prec x_i.$$

The proof principle of *well-founded* or *Noetherian induction* is responsible for proving the validity of a property P on M if there exists a well-founded order $\preceq$ on M . It states that it suffices to prove $P(z)$ under the assumption that $P(y)$ holds for all predecessors y of z . In other words:

$$\forall z \in M : (\forall y \in M : y \prec z \rightarrow P(y)) \rightarrow P(z) \rightarrow \forall x \in M : P(x)$$

Derive complete and structural induction as special cases of well-founded induction.

9 HIGHER-ORDER PROGRAMMING

The renowned computer scientist Paul Hudak was once asked what the three most important things in programming were. He replied: "Abstraction, abstraction, abstraction." Indeed, Mantra 4 on page 45 is one of the most important:

Mantra 4:

If you see two program parts that differ only in a few places and are related in content, abstract!

In this chapter, we will consistently apply this mantra. This often results in a special type of function: functions that accept other functions as input or return them as output. Such functions are also called *higher-order functions*. This chapter focuses on the resulting type of programming— *higher-order programming*.

9.1 FINDING FOOTBALL FACTS

[higher-order/soccer.rkt](#)

code

Football results provide a virtually inexhaustible source of discussion. Important questions arise, such as:

- How did Bayern Munich perform away against 1. FC Kaiserslautern this season?
- How many goals have been scored this season?
- Which team is in danger of relegation?

All this information is derived from the results of a season's games. We can characterize a game as follows:

A game has the following properties:

- ; - Matchday
- ; - Home team
- ; - Home team goals
- ; - Visiting team
- ; - Away team goals

¹ We know that was a while ago.

This is clearly composite data. We need to consider which signatures are appropriate for the individual components. Matchday and goals are all natural numbers. We can represent a team as a string of characters using its name. This results in the following record definition:

```
(define-record game make-
  game game? (game-
    matchday natural) (game-home-
    team string) (game-home-goals
    natural) (game-guest-team string)
    (game-guest-goals natural))
```

The signature " *string* " in a signature is rather meaningless. (This is often the case with *strings* in particular .) If, for example, we wanted to include player names later, these might also be strings. To differentiate this in signatures, we use the following signature definition:

A team is identified by its name.

```
(define team (signature string))
```

Furthermore, we replace " *string* " with " *game* " in the definition of " *game* " .

The following are examples of the results from the first matchday of the 2009/2010 Bundesliga season:

```
(define game1 (make-game 1 "Wolfsburg" 2 "Stuttgart" 0)) (define game2 (make-game
1 "Mainz" 2 "Bayer 04" 2)) (define game3 (make-game 1 "Hertha" 1 "Hannover"
0)) (define game4 (make-game 1 "Bremen" 2 "Frankfurt" 3)) (define game5
(make-game 1 "Nuremberg" 1 "Schalke" 2)) (define game6 (make-game 1
"Dortmund" 1 "1. FC Köln" 0)) (define game7 (make-game 1 "Hoffenheim" 1
"Bayern" 1)) (define game8 (make-game 1 "Bochum" 3 "Gladbach" 3)) (define game9
(make-game 1 "Freiburg" 1 "Hamburg" 1))
```

```
(define day1 (list
  game1 game2 game3 game4 game5 game6 game7 game8 game9))
```

A fairly simple question is determining the score achieved by the home team in a particular game. While the score is a natural number, there are only three possibilities: 0, 1, and 3. Therefore, we can define a more precise signature than " *points* " :

```
; Score in game (define
points (signature
  (enum 0 1 3)))
```

Our scoring function includes a short description, signature, tests, and

Scaffolding as follows:

```
Calculate score for home team
(: home points (game -> points))
```

```
(check-expect (home-points game1) 3) (check-
expect (home-points game2) 1) (check-expect
(home-points game3) 3) (check-expect (home-
points game4) 0)
```

```
(define home points
  (lambda (game) ...))
```

According to the signature of the edition, the games fall into three different classes, meaning we need a branching with three branches corresponding to the three possible scores:

```
(define home points
  (lambda (game)
    (cond
      (... 3) (...
        0)
      (... 1))))
```

The conditions of the three branches come from the football rules – we have to compare the goals scored by the home and away teams:

```
(define home-points
  (lambda (game)
    (cond
      ((> (game-home-goals game) (game-guest-goals game)) 3) ((< (game-home-
goals game) (game-guest-goals game)) 0) ((= (game-home-goals game)
(game-guest-goals game)) 1))))
```

This already works correctly, but it's still inelegant. The `(game-home-goals game)` is repeated three times – we should introduce a local definition so that it only appears once.

The same applies to calling `game-guest-goals`. It looks like this:

```
(define home points (lambda
  (game)
    (define goals1 (game-home-goals game)) (define goals2
      (game-guest-goals game)) (cond
      ((> goals1 goals2) 3) ((<
        goals1 goals2) 0) ((= goals1
          goals2) 1))))
```

You could rightfully ask why the local variables are called `goals1` and `goals2`, and not `home-goals` and `guest-goals`. This will (hopefully) become clear with the next function, which calculates the points awarded to the visiting team:

Calculate the points for the away team (: guest-points
(game -> points))

```
(define guest points (lambda
  (game)
    (define goals1 (game-guest-goals game)) (define goals2
      (game-home-goals game)) (cond ((> goals1 goals2) 3)
      ((<
        goals1 goals2) 0) ((= goals1
          goals2) 1))))
```

These two functions are largely identical; the only difference is the definition of the local variables `goals1` and `goals2`. Such code duplication is always bad, especially when things change. Theoretically, the football association could change the rules for awarding points, and then we would have to adjust both functions accordingly. The solution to this problem is to create a new function from the shared code of the two functions, following the abstraction mantra:

Mantra 4:

If you see two program parts that differ only in a few places and are related in content, abstract!

To abstract, we copy one of the two function definitions and give it a new name, in this case `compute-points`:

```
(define compute-points (lambda
  (game)
    (define goals1 (game-guest-goals game)) (define goals2
      (game-home-goals game)) (cond ((> goals1 goals2) 3)
      ((<
        goals1 goals2) 0) ((= goals1
        goals2) 1))))
```

Now, in the copied function, we identify the parts that differ between the two original functions and replace these parts with new names. In this case, those are ``game-guest-goals`` and ``game-home-goals``; we replace them with the names ``get-goals-1`` and ``get-goals-2``.

```
(define compute-points (lambda
  (game) (define
    goals1 (get-goals-1 game)) (define goals2 (get-
      goals-2 game)) (cond ((> goals1 goals2) 3) ((<
      goals1
        goals2) 0) ((= goals1 goals2)
        1))))
```

These new variables are still unbound, so we still need to place them in the **lambda**.
gene:

```
(define compute-points (lambda
  (get-goals-1 get-goals-2 game) (define goals1 (get-
    goals-1 game)) (define goals2 (get-goals-2
    game))
    (cond
      ((> goals1 goals2) 3) ((<
        goals1 goals2) 0) ((= goals1
        goals2) 1))))
```

This allows us to replace the previous definitions of `home points` and `guest points` with the following:

```
(define home points
  (lambda (game)
    (compute-points game-home-goals game-guest-goals game)))
```

```
(define guest points (lambda
  (game)
    (compute-points game-guest-goals game-home-goals game)))
```

These new definitions clearly highlight the similarities and differences between the two functions.

Recognize and avoid the double definition of the football scoring rules.

Important: These two definitions must come *after* the definition of `compute-points`, because `compute-points` has not been defined before them.

The new function is still missing a signature. The function accepts three arguments. The last one has the signature ``game``. The first two arguments are taken from the functions ``game-home-goals`` and ``game-guest-goals``, which each have the signature ``(game -> natural)``. From this, we can construct the signature for ``compute-points``:

```
(: compute-points ((game -> natural) (game -> natural) game ->
  points))
```

The signature contains multiple arrows because the ``compute-points`` function itself accepts two functions as arguments. Such functions with multiple arrows in their signature are called *higher - order functions*.

We could have handled the abstraction in the development of ``compute-points`` differently: We added the two parameters ``get-goals-1`` and ``get-goals-2`` to the existing `lambda` in ``compute-points``. We could also insert a new `lambda` instead. (We would have to do this anyway for definitions that don't already contain a `lambda`.) It would then look like this:

```
(define make-compute-points (lambda
  (get-goals-1 get-goals-2) (lambda (game)
    (define goals1 (get-
      goals-1 game)) (define goals2 (get-goals-2
      game)) (cond

      ((> goals1 goals2) 3) ((<
      goals1 goals2) 0) ((= goals1
      goals2) 1))))))
```

The signature of this function differs slightly from the signature of `compute-points`.

It no longer accepts three arguments but only two, but provides a function that calculates the points for a game:

```
(: make-compute-points ((game -> natural) (game -> natural) -> (game -> points)))
```

The `make-compute-points` function also allows us to write alternative definitions of `home-points` and `guest-points`. At first glance, this seems cumbersome if we do it the same way as with `compute-points`, since we need even more parentheses:

```
(define home points
  (lambda (game)
    ((make-compute-points game-home-goals game-guest-goals) game)))
```

```
(define guest points (lambda
  (game)
    ((make-compute-points game-guest-goals game-home-goals) game)))
```

Perhaps you will notice the pattern that these two definitions have in common:

```
(define f
  (lambda (x)
    (gx)))
```

That means when the program calls `f`, this function directly calls `g`, with the same input: `f` and `g` are therefore equivalent, and we could just as well write:

```
(define fg)
```

We can apply the same principle to `home points` and `guest points` :

```
(define home-points
  (make-compute-points game-home-goals game-guest-goals))
```

```
(define guest points)
  (make-compute-points game-guest-goals game-home-goals))
```

You can also clearly see from the signature of `make-compute-points` that it returns a function with the signature `(game -> points)` as a result, which is exactly the desired signature of `home-points` and `guest-points`.

You might have wondered about the name ``make-compute-points``: While the ``compute-points`` function directly calculates a score, ``make-compute-points`` creates a function that calculates the score: Hence the prefix ``make-``. It's essentially a kind of "function factory".

Task 9.1

We could have inserted the new **lambda** below the old **lambda** instead of above it.

What signature would the ``make-compute-points`` function have then? Why is that not such a good idea? ÿ

The abstraction technique we used to construct the functions ``compute-points`` and ``make-compute-points`` will be used several more times in this book. It therefore deserves its own construction guide:

Construction guide 20 (abstraction)

If you have written two definitions that are related in content and have many similarities, abstract them as follows: 1. Copy

one of the two definitions and give it a new name.

2. Replace the points where the two definitions differ with a new variable.

3. Add the new variables as parameters to the **lambda** in the definition, or add a new **lambda** with these parameters. You may need to adjust recursive calls to the function.

4. Write a signature for the new feature.

5. Replace the two old definitions with calls to the new definition.

9.2 HIGHER-ORDER FUNCTIONS ON LISTS

higher-order/list.rkt

code

Next, we want to find out which games in a season ended in a draw.

First, a function is needed that determines whether a particular game is a draw. was:

Is the game a draw? (: game-draw? (game -> boolean))

```
(define game-draw?
  (lambda (g) (=
    1 (home-points g))))
```

The season's games are in a list, so we'll write a function that extracts the draws from this list. The short description, signature, and structure follow the construction guidelines for functions on lists. Here's the template:

```
Filter out drawn games (: drawn-games ((list-of
game) -> (list-of game)))
```

```
(check-expect (drawn-games day1) (list game2 game7 game8 game9))
```

```
(define drawn-games
  (lambda (list)
    ...))
```

The function takes a list as input, so the corresponding template is applied:

```
(define drawn-games
  (lambda (list)
    (cond
      ((empty? list) ...) ((cons?
list)
... (first list) ...
... (drawn-games (rest list)) ...))))
```

The recursive call returns the undecided games of the rest, so we only need to decide whether the first game ended in a draw:

```
(define drawn-games
  (lambda (list) (cond
    ((empty? list) ...) ((cons?
list) ... (game-
draw? (first list)) ... ... (drawn-games (rest
list)) ...))))
```

The function `game-draw?` returns a boolean value, which we can only really put into a binary branch:

```
(define drawn-games
  (lambda (list)
    (cond
      ((empty? list) ...)
      ((cons? list)
       (if (game-draw? (first list))
           ...
           ...))
      ... (drawn games (rest list)) ...))))
```

Depending on whether the game is a draw or not, we will include it in the result:

```
(define drawn-games
  (lambda (list)
    (cond
      ((empty? list) empty) ((cons?
                                list)
                             (if (game-draw? (first list)) (cons (first list)
                                                                    (drawn-games (rest list)))
                                (drawn-games (rest list))))))
```

Similar to `drawn-games`, we will now write a function that extracts all games from a list that the home team has won. For this, we need a function analogous to `game-draw`.

```
Did the home team win? (: home-won? (game ->
boolean))
```

```
(check-expect (home-won? game1) #t) (check-
expect (home-won? game2) #f) (check-expect
(home-won? game4) #f)
```

```
(define home-won?
  (lambda (game)
    (= 3 (home-points game))))
```

The `home-won-games` function is created analogously to `drawn-games`.

```
; Filter out games where the home team won (: home-won-games ((list-of game) ->
(list-of game)))
```

```
(check-expect (home-won-games day1) (list game1 game3 game6))
```

```
(define home-won-games
  (lambda (list) (cond
    ((empty? list) empty) ((cons?
    list)
    (if (home-won? (first list))
        (cons (first list)
              (home-won-games (rest list)))
        (home-won-games (rest list)))))))
```

The two functions `drawn-games` and `home-won-games` are almost identical except for their names. The only difference is that `home-won` replaces `game-draw?` in the second function. We can abstract from this, following construction instructions 20 on page 252. To do this, we copy the last function and come up with a new name – since both involve “extracting,” we’ll use `extract-games`.

```
(define extract-games
  (lambda (list)
    (cond
      ((empty? list) empty)
      ((cons? list)
       (if (home-won? (first list))
           (cons (first list)
                 (extract-games (rest list)))
           (extract-games (rest list)))))))
```

Next, we need to denote the point that differs between the two definitions (`game-draw?` or `home-won?`) by a new variable. We’ll call this `f?`:

```
(define extract-games (lambda
  (list) (cond

    ((empty? list) empty)
```

```

((cons? list)
 (if (f? (first list))
     (cons (first list)
           (extract-games (rest list)))
     (extract-games (rest list)))))

```

Next, we need to add the new variable as a parameter to the **lambda** . Note: Both recursive calls must also be extended with the parameter.

```

(define extract-games (lambda
  (f? list)
    (cond
      ((empty? list) empty) ((cons?
                                list)
                              (if (f? (first list))
                                  (cons (first list)
                                        (extract-games f? (rest list)))
                                  (extract-games f?
                                                  (rest list)))))

```

(It's a matter of taste whether the new parameter comes before or after the old ones.)

The function definition is now complete, but we still need to write a signature. We'll start with the signature of ``home-won-games`` and extend it with the additional parameter:

```

(: extract-games (... (list-of game) -> (list-of game)))

```

The additional parameter is `f?`, which we used for `game-draw?` and `home-won?`. These two functions have the signature `(game -> boolean)`, so we can use that for the ellipsis:

```

(: extract-games ((game -> boolean) (list-of game) -> (list-of game)))

```

Now we can replace calls to the old functions with calls to the new ones. We do this by adjusting the test cases for ``drawn-games`` and ``home-won-games`` :

```

(check-expect (extract-games game-draw? day1) (list game2 game7
                                                    game8 game9))
(check-expect (extract-games
  home-won? day1) (list game1 game3 game6))

```

Complete!

You might feel like you've seen the pattern of the ``extract-games`` function before. Indeed, the ``live-dillos`` function in section 6.6.2 on page 6.6.2 also looks like this.

very similar to the `drawn-games` and `home-won-games` functions . Here it is again for Memory:

```
(: live-dillos ((list-of dillo) -> (list-of dillo)))

(define live-dillos
  (lambda (dillos)
    (cond
      ((empty? dillos) empty)
      ((cons? dillos)
       (if (dillo-alive? (first dillos))
           (cons (first dillos)
                 (live-dillos (rest dillos)))
           (live dillos (rest dillos)))))))
```

And indeed – if we rename `dillos` to `list` and use the same abstraction process. If the process is carried out as with `drawn games` and `home-won games` , then the identical function definition to `extract games` is created. However, there is still one difference – the Signatures differ. The ``extract-games`` function can only process lists of games, while ``live-dillos`` accepts a list of armadillos. You can also...

One time `game`, the other time `dillo` – abstracting?

We can! To do this, we should rename the function; we'll choose `extract-list`.

instead of `extract-games`. Most importantly, we need to change the signature, which is currently subscribed to `game` . At the signature level, we abstract using signature variables and, for the sake of argument, simply replace every `game` with `%a`:

```
(: extract-list ((%a -> boolean) (list-of %a) -> (list-of %a)))
```

We can now try to understand this function: It accepts a list of `%as` and a function that accepts `%as` . You can almost see what the function does from this: The only thing it can do with the function is apply it to the elements of the list – otherwise. There are no other `%as` for miles around . The fact that the function produces a boolean value, This already suggests that a distinction is made based on this value. The signature therefore provides valuable information. Information about what the function does – though not all of it.

Task 9.2

Write another useful function with the same signature as `extract-list`!

Let's consider another application of `extract-list`: We want to extract from a list of games the games in which a specific team has participated. For this, we also define a helper function that determines whether a team is the home or away team in a given game:

Does the team play in the game? (:
`plays-game? (team game -> boolean))`

```
(check-expect (plays-game? "Wolfsburg" game1) #t) (check-
expect (plays-game? "Stuttgart" game1) #t) (check-expect (plays-
game? "Hannover" game1) #f)
```

```
(define plays-game?
  (lambda (team game)
    (or (string=? team (game-home-team game))
        (string=? team (game-guest-team game)))))
```

Let's assume we want to extract all games involving Nuremberg using the `extract-list` function. For this, we need a function with a signature (`game` -> boolean``). We can define this as follows:

Is Nuremberg playing? (:
`plays-nürnberg? (game -> boolean))`

```
(check-expect (plays-nürnberg? game1) #f) (check-
expect (plays-nürnberg? game5) #t)
```

```
(define plays-nürnberg?
  (lambda (game)
    (plays-game? "Nuremberg" game)))
```

This allows us, for example, to extract all Nuremberg games from the season:

```
(extract-list plays-nürnberg? season-2009/2010)
```

So far, so good. But what if we also want to extract HSV's matches? We could define a function called `"plays-hamburg?"`, but if we make many such requests, it will become cumbersome in the long run to write a separate definition for each club.

It is simpler to instead use the function we pass to `extract-list` "ad hoc".
 to construct with a **lambda** :

```
(extract-list (lambda (game) (plays-game? "Nuremberg" game))
              season-2009/2010)

(extract-list (lambda (game) (plays-game? "Hamburg" game))
              season-2009/2010)
```

We proceeded from the signature: `extract-list` expects a function with a parameter of the signature `game`, so we write a **lambda** with a corresponding parameter.

We want to extract all games in which Nuremberg or Hamburg is involved, so we write the corresponding expression in the body.

Task 9.3

Write a function called ``games-playing`` that extracts all games from a list in which a specific team participated. `γ`

The `extract-list` function is so useful that it is built-in under the name `filter` .
is.

9.3 CONVERTING LISTS

Certain football statistics deal with how many goals were scored in a match, on a matchday, or in an entire season. This is based on the number of goals scored in a single match. Here is a brief description, signature, tests, and framework for them:

Calculate the total goals of a game (: total-goals
(game -> natural))

```
(check-expect (total-goals game1) 2) (check-expect
(total-goals game2) 4)
```

```
(define total-goals (lambda
  (game) ...))
```

As a template, we use the assumption that the input consists of composite data:

```
(define total-goals (lambda
  (game)
```

```
... (game-matchday game) ... ... (game-
home-team game) ... ... (game-home-
goals game) ... ... (game-guest-team
game) ... ... (game-guest-goals game) ...))
```

To determine the total number of goals, we only need `(game-home-goals game)` and `(game-guest-goals game)`, which are added together:

```
(define total-goals (lambda
  (game) (+ (game-
             home-goals game)
            (game-guest-goals game))))
```

Next, we want to find the total goals for all matches on a given matchday. To do this, we will create a list of the matches and their corresponding total goals:

```
; Calculate total goals in a list of games (: list-total-goals ((list-of game)
-> (list-of natural)))
```

The first element of the result should be the number of goals scored in the first game from the input list, the second element the number of goals scored in the second game, and so on. This test illustrates how it works:

```
(check-expect (list-total-goals day1)
  (list 2 4 1 5 3 1 2 6 2))
```

The following is the framework with the obligatory template for functions on lists:

```
(define list-total-goals (lambda (list)

  (cond
    ((empty? list) ...) ((cons?
list)
  ... (first list)
  ... (list-total-goals (rest list)) ...))))
```

If the input is an empty list, then the output can only be empty as well. In the other branch, the template contains the first game: the function must determine the number of goals for that game; this has already been done for the rest. We combine the two results using ``cons``:

```
(define list-total-goals (lambda (list)
```

```
(cond
  ((empty? list) empty) ((cons?
    list)
    (cons (total-goals (first list)) (list-total-goals
      (rest list))))))
```

Task 9.4

Use the `list-sum` function from section 6.1 on page 176 to write a function that calculates the average number of goals scored in a list of games. Ÿ

Fans might be less interested in total goals than in their team's goals. To get an overview of this, we first need a function that calculates the number of goals scored by a specific team in a single match:

```
; Calculate a team's goals from a game (: team-goals (team game ->
natural))
```

```
(check-expect (team-goals "Wolfsburg" game1) 2) (check-expect
(team-goals "Stuttgart" game1) 0)
```

What should happen if the desired team wasn't present at the game? A mistake:

```
(check-error (team goals "Hannover" game1))
```

The framework and template again make use of the fact that `game` data is composite:

```
(define team goals
  (lambda (team game) ...
    (game-matchday game) ... ... (game-
      home-team game) ... ... (game-home-
        goals game) ... ... (game-guest-team
          game) ... ... (game-guest-goals game) ...))
```

Furthermore, the data includes a case distinction depending on whether the team being searched for is the home or away team, resulting in a branching with two branches:

```
(define team goals
  (lambda (team game)
```

```
(cond
  ((string=? team (game-home-team game)) (game-
    home-goals game))
  ((string=? team (game-guest-team game)) (game-guest-
    goals game))))
```

Now we want to use this function to convert an entire list of games in which a specific team participates into a list of that team's goals. The signature and short description look like this:

```
; List goals of a team from a list of games (: list-team-goals (team (list-of game) -> (list-of
natural)))
```

Task 9.5

Write a test case that calculates a list of Hamburg's goals in a season. Use [filters to do this!](#) ⁵

The scaffold and template look like this:

```
(define list-team-goals (lambda
  (team list) (cond

    ((empty? list) ...) ((cons?
  list)
  ... (first list) ...
  ... (list-team-goals team (rest list)) ...))))
```

And again, an empty list as input results in an empty list as output, and in the cons case, we apply the previously written helper function to the first element and build the cons with cons .

Results list:

```
(define list-team-goals (lambda
  (team list) (cond

    ((empty? list) empty) ((cons?
  list)
  (cons (team-goals team (first list)) (list-team-goals
    team (rest list))))))
```

Done! However, the abstraction mantra beckons: The two functions ``list-total-goals`` and ``list-team-goals`` are very similar. They differ only in that the former calls ``total-goals`` on the first element of the list, and the latter calls ``team-goals``.

To abstract, we again follow construction instructions 20 on page 252. We copy one of the two functions (we'll take the first one) and give it a new name:

```
(define list-apply (lambda
  (list)
  (cond
    ((empty? list) empty) ((cons? list)
    (cons (total goals (first list))
          (list-apply (rest list))))))
```

The function is called `list-apply` because it applies a function to all elements of a list.

Next, we need to introduce a variable where the two functions differ, namely with `total goals`. We'll simply call this function `f`:

```
(define list-apply
  (lambda (list)
    (cond
      ((empty? list) empty)
      ((cons? list)
       (cons (f (first list))
             (list-apply (rest list))))))
```

Now we need to place the new variable in the `lambda`, without forgetting to include it in the recursive call:

```
(define list-apply (lambda (f
  list) (cond
    ((empty? list) empty) ((cons?
  list) (cons (f (first
  list))
    (list-apply f (rest list))))))
```

Task 9.6

Change the definition of `list-total-goals` so that it uses `list-apply`.

We still need a signature for the new feature. Let's try creating it "from scratch".

We can construct it by writing down what we know. We know that `list-apply` has two arguments, the first of which is a unary function and the second a list. Furthermore

The function also produces a list as a result. Written, it looks like this:

```
(: list-apply ((... -> ...) (list-of ...) -> (list-of ...)))
```

We did abstract away two functions that deal with games. But the resulting `list-apply` function no longer has anything football-specific about it. So we know

Nothing is mentioned about the signature of the elements in the input list. Therefore, we write a signature variable there:

```
(: list-apply ((... -> ...) (list-of %element) -> (list-of ...)))
```

Now we know that the elements of the input list are used as input for the function `f`, so we can also write `%element` there:

```
(: list-apply ((%element -> ...) (list-of %element)
              -> (list-of ...)))
```

The definition of `list-apply` does not specify a signature for the result of `f`. We write

So, add a signature variable there as well:

```
(: list-apply ((%element -> %result) (list-of %element)
              -> (list-of ...)))
```

All that's missing is the signature for the elements in the results list. For that, we need the call

Consider `cons`, whose first argument is precisely the result of the call to `f`. There, it must be

That's why `%result` is also mentioned:

```
(: list-apply ((%element -> %result) (list-of %element)
              -> (list-of %result)))
```

In such signatures, where absolutely nothing is known about the signature variables, it is

It is common to choose shorter names for the signature variables, for example like this:

```
(: list-apply ((%a -> %b) (list-of %a) -> (list-of %b)))
```

Complete!

Task 9.7

Also rewrite `list-team-goals` so that it uses `list-apply`. You'll need to use a similar trick to the one used when combining `extract-list` and `plays-game` in the previous section on page 258. ⁵

The `list-apply` function is just as useful as `filter` from the previous section and is therefore permanently built in under the name `map`.

9.4 FOLDING LISTS

In section 6.1, on page 176, we described the functions `list-sum` and `list-product`, which calculate the sum of a list of numbers:

Calculate the sum of the elements in a list of numbers

```
(: list-sum ((list-of number) -> number))
```

```
(define list-sum (lambda
  (list)
    (cond
      ((empty? list) 0) ((cons?
        list)
          (+ (first list) (list-sum (rest list)))))))
```

Calculate the product of the elements of a list of numbers (: list-product (list-of-numbers -> number))

```
(define list-product
  (lambda (list)
    (cond
      ((empty? list) 1)
      ((cons? list)
        (* (first list) (list-product (rest list)))))))
```

The definitions of `list-sum` and `list-product` differ, apart from the name, in only two places: in the `empty` branch, where the respective neutral element is located, and in the `cons` branch, where the function used to combine the first element with the result of the recursive call is located. Therefore, it's a good idea to abstract! We proceed again according to Construction Guide 20 on page 252. Because we don't yet know exactly what the function will do, we call it `xxx` in the hope that we'll come up with a better name later.

```
(define xxx
  (lambda (list)
```

```
(cond
  ((empty? list) 1) ((cons?
    list)
    (* (first list) (xxx (rest list))))))
```

Next, we need to explain the differences between the two functions by introducing new variable-

Replace `blen`, which in this case are 1 and *. We call the new variables `for-empty` and `for-cons` because they represent `empty` and `cons` respectively in the branches :

```
(define xxx
  (lambda (list)
    (cond
      ((empty? list) for-empty) ((cons?
        list)
        (for-cons (first list) (xxx (rest list))))))
```

For the next step, we add the new variables to the existing **lambda** . We must again ensure that the variables are also added to the recursive call:

```
(define xxx
  (lambda (for-empty for-cons list)
    (cond
      ((empty? list) for-empty) ((cons?
        list)
        (for-cons (first list)
          (xxx for-empty for-cons (rest list))))))
```

This function can perform the task of both **list-sum** and **list-product**.

gene:

```
(xxx 0 + (list 1 2 3 4))
↳ 10
```

and multiply it up like this:

```
(xxx 1 * (list 1 2 3 4))
↳ 24
```

Following the construction instructions, the next step is to define the signature for the new function. It makes sense to generalize the signatures of `list-sum` and `list-product` . For the new parameter `for-empty` , we use the signature of 0 and 1, i.e., `number` . For `for-cons` , we use the signature of + and *, i.e., (number number -> number):

```
(: xxx (number (number number -> number) (list-of number) -> number))
```

However, upon closer inspection, the definition of `xxx` contains nothing "number-specific", so we could try to replace `number` with a signature variable, as we did with `list-apply` and `map` in the previous section:

```
(: xxx (%a (%a %a -> %a) (list-of %a) -> %a))
```

This means we could try calling `xxx` with something that isn't a list of numbers. For this, we need a suitable second argument, i.e., a function whose signature has the form `(%a %a -> %a)`. An example is the `append` function, which appends two lists and has the following signature:

```
((list-of %element) (list-of %element) -> (list-of %element))
```

Because we are using `(list-of %element)` for `%a`, we need an additional list as the first argument for `xxx` and a list of lists as the third argument. This could look like this:

```
(xxx empty append (list (list 1 2 3) (list 4 5 6) (list 7 8 9))) y #<list 1 2 3 4 5 6 7 8 9>
```

So we can use `xxx` to "flatten" a list of lists.

But is that really the best signature for `xxx`? As with `list-map`, we can investigate whether the `%a`s actually all need to be the same. To find out, we could assume they are all different:

```
(: xxx (%a (%b %c -> %d) (list-of %e) -> %f))
```

However, if we look closely at the definition of `xxx`, we see that some signature variables must be the same in each case:

- The signature `%a` of `for-empty` must be the same as `%f`, because `for-empty` as a result is used by `xxx`.
- The signature `%b` of the first argument of `for-cons` must be the same as that of the list-elements `%a`, because it is used as the first argument (`first list`).
- The signature `%c` of the argument of `for-cons` must be the same as `%f`, because the result of `xxx` is used there.
- The signature `%d` of the result of `for-cons` must also be the same as `%f`, because the call to `for-cons` is used as the result of `xxx`.

But at least `%a` and `%b` don't have to be the same. The following remains:

```
(: xxx (%a (%b %a -> %a) (list-of %b) -> %a))
```

Task 9.8

Follow the evaluation of

```
(xxx 0 + (list 1 2 3))
```

in the stepper!

ŷ

In the stepper you can see that `xxx` works as follows:

```
(xxx ec #<list a1 . . . an >)
```

```
ŷ. . ŷ
```

```
(c a1 (c a2 (. . . (c an u) . . .)))
```

In the case of `0` and `+`, the list is then reduced to a single value or "folded", which is why we call the function `list-fold`.

We can illustrate how `list-fold` works by pointing out that the original original list also as

```
(cons a1 (cons a2 (. . . (cons an empty) . . .)))
```

can be written. That is, `c` replaces `cons` and `e` replaces `empty`.

Task 9.9

Can you deduce what the result will be without using DrRacket?

```
(list-fold empty cons (list 1 2 3))
```

ŷ

Another practical way to represent `list-folding` is to write the equation with the operator *between* the operands (and not before them). To make this clear, we write `ŷ` instead of `c` for `for-cons`:

```
(list-fold c ŷ #<list a1 . . . an>)
```

```
ŷ. . ŷ
```

```
a1 ŷ (a2 ŷ (. . . (an ŷ e). . .))
```

The ``list-fold`` function is surprisingly versatile and can perform the tasks of many other functions on lists. For example, we can revisit ``list-apply`` and try writing the definition using ``list-fold``. Here's the previous version as a reminder:

```
(define list-apply (lambda
  (f list) (cond

    ((empty? list) empty) ((cons?
    list) (cons (f (first
    list))

      (list-apply f (rest list))))))
```

We want to rewrite `list-apply` into the following form:

```
(define list-apply (lambda
  (f list) (list-fold ...

    ...

    list)))
```

We need to find suitable values for the two parameters of ``list-fold``, ``for-empty`` and ``for-cons``. In ``list-apply``, the ``empty`` branch contains ``empty``.

With `for-cons`, it's not so obvious. But we can use the signature as a guide.

We need a function with two arguments: the first is the first list element, the second is the result of the recursive call. So we can start like this:

```
(define list-apply
  (lambda (f list)
    (list-fold empt (lambda
      (first-list result)
        ...)
    list)))
```

In the body of the new lambda function, we now need to write code that corresponds to the code from `list-apply` in the `cons` branch, i.e.:

```
(cons (f (first list))
      (list-apply f (rest list)))
```

However, we need to use the parameters from the **lambda**, namely ``first-list`` and ``result``: ``first-list`` is simply the value of ``(first-lits)``, and ``result`` is the result of the recursive call. So we're essentially replacing "backwards":

```
(define list-apply (lambda
  (f list)
```

```

(list-fold empty (lambda
                  (first-list result) (cons (f first-list)
                                           result)
                  list)))

```

Complete!

Task 9.10

Rewrite `extract-list` using `list-fold` in the same way !

ŷ

The `list-fold` function is so practical that it is also built in, under the name `fold`.

9.5 SCHÖNFINDEN

You might remember the ``plays-game?`` function on page 258. We wrote it to extract games involving a specific team using ``extract-list``. However, we couldn't use ``plays-game?`` directly with ``extract-list`` because ``extract-list`` accepts a single-digit function, while ``plays-game?`` is a two-digit function.

We had to use `lambda` to construct a suitable function. As a reminder, it looked like this:

```

(extract-list (lambda (game) (plays-game? "Nuremberg" game))
              season-2009/2010)

(extract-list (lambda (game) (plays-game? "Hamburg" game)) season-2009/2010)

```

The two `lambdas` are almost identical except for the team name; this clearly calls for abstraction! For this, we need a function that generates `lambdas` like the one above. If we copy one of them and replace just the different part with a variable, it looks like this:

```

(lambda (game) (plays-game? team game))

```

Now we need to bind ``team`` somewhere. It doesn't make sense to add it to the existing `lambda function` – then we'd just have another two-digit function. Instead, we wrap it in a `lambda function` and give it a name:

```
(define plays-game?/team (lambda
  (team)
    (lambda (game) (plays-game? team game))))
```

(The slash is a normal part of the name.)

This function accepts a team and in turn provides a function that is useful for the game. It yields a boolean value. Therefore, it has the following signature:

```
(: plays-game?/team (team -> (game -> boolean)))
```

Now we can write the calls to `extract-list` using `plays-game?/team` :

```
(extract-list (plays-game?/team "Nuremberg")
  season-2009/2010)

(extract-list (plays-game?/team "Hamburg")
  season-2009/2010)
```

We can perform a similar construction for `team-goals` to, for example, make it easier to use with `list-apply` .

This results in the following definition:

```
(: team-goals/team (team -> (game -> natural))) (define team-goals/team
  (lambda (team)
```

```
    (lambda (game)
      (team goals team game))))
```

We can abstract again, using ``plays-game?/team`` and ``team-goals/team`` – these are identical except for the function that is called at the end. So we copy it again and replace it with the new variable ``f``. For now, we'll call the function ``xxx`` again:

```
(define xxx
  (lambda (team)
    (lambda (game) (f
      team game))))
```

For the new name, we'll create another new **lambda**:

```
(define xxx
  (lambda (f)
    (lambda (team)
      (lambda (game) (f
        team game))))))
```

We need a signature. Since three **lambdas** are involved, we need three arrows. Furthermore, the first argument for **f** is a function that accepts two arguments – another arrow:

```
(: xxx ((... ... -> ...) -> (... -> (... -> ...))))
```

Many arrows – don't let them unsettle you!

We're abstracting from two functions that deal with football, *but once again*, the resulting function has nothing football-specific about it. So, we'll try using type variables again, starting with the signature of **f**. We'll just use three different variables – we might find later that some of them need to be the same:

```
(: xxx ((%a %b -> %c) -> (... -> (... -> ...))))
```

Within **xxx**, **f** is called with **team** and **game**, which belong to signatures **%a** and **%b** respectively. Further analysis is therefore simplified if we rename **team** to **a** and **game** to **b**.

to name:

```
(define xxx
  (lambda (f)
    (lambda (a)
      (lambda (b)
        (fab))))))
```

We can read the rest of the signature using these names:

```
(: xxx ((%a %b -> %c) -> (%a -> (%b -> %c))))
```

What should the function be called? It's actually quite famous and goes back to the work of mathematicians Moses Schönfinkel and Haskell Curry. In English-speaking countries, the verb for it is therefore "*curry*," while in German-speaking countries it's "*schönfinkeln*" or "*curryfizieren*." So here's the final version:

```
; Function schönfinkeln (: curry
((%a %b -> %c) -> (%a -> (%b -> %c))))
```

```
(define curry
  (lambda (f)
    (lambda (a)
      (lambda (b)
        (fab))))))
```

Here is a test case that uses `curry` together with `plays-game?`:

```
(check-expect (list-apply ((curry team-goals) "Hamburg")
                          (filter ((curry plays-game?) "Hamburg")
                                  season-2009/2010)) (list 1
                                                            4 4 3 3 1 1 3 0 3 2 2 0 1 0 4 2
                                                            2 0 1 3 3 0 0 1 2 2 0 0 2 0 1 4 1))
```

Task 9.11

Write an example that uses `curry` together with `team goals` !

Ÿ

The ``curry`` function converts a binary function into a single-argument function. To call the binary function, the program must provide both arguments at once; otherwise, they come one after the other – but the result is the same.

If we can turn a two-digit function into a one-digit function (i.e., a one-digit function that yields a one-digit function), can we also do the reverse? Absolutely! To do this, we reverse the signature compared to `curry`'s by swapping the parts before and after the arrow in the middle:

; Function to de-embellish

```
(: uncurry ((%a -> (%b -> %c)) -> (%a %b -> %c)))
```

As a test, we run `uncurry` on the result of `curry` :

```
(check-expect ((uncurry (curry +)) 3 4) 7)
```

For the construction, we follow the signature as with `curry` : `curry` accepts a function and returns a binary function. Therefore, we begin as follows:

```
(define uncurry (lambda
  (f) (lambda (ab)

    ...)))
```

The inner function should produce a value with signature `%c` . To obtain this, we must first call `f` with a value of signature `%a` – which we have given as a:

```
(define uncurry (lambda
  (f) (lambda (ab)

    ... (fa) ...))
```

The call `(fa)` in turn returns a function with signature `(%b -> %c)`, so we need to put parentheses around it again and return a `%b` – in this case, that's `b`:

```
(define uncurry (lambda
  (f) (lambda (ab)

    ((fa) b))))
```

Complete!

Therefore, the transformation is an *isomorphism*; the following equation holds for functions `f` with two parameters:

$$(\text{uncurry } (\text{curry } f)) = f$$

Task 9.12

Try writing functions `curry3` and `uncurry3`. They should work like `curry` and `uncurry`, but for functions with three inputs instead of just two. ÿ

9.6 Lexical Link

In this section, we'll make an exception and not write any new functions, but instead examine a subtle aspect of function behavior: how they handle variables. You've probably already noticed that we often use the same names in different functions, for example here:

```
(define feed-dillo (lambda
  (dillo)
  ...))

(define run-over-dillo (lambda (dillo)

  ...))
```

Although the parameter in both functions is called ``dillo``, the two functions are unrelated. You can see this in the Stepper program by watching the evaluation process. You can also see it directly in the program. To get to the bottom of this, let's look at the following program:

```
(define pi 3.14159265)
```

```
; Calculate the area of a circle (: circle-area (real ->
real))
```

```
(check-expect (circle-area 5) 78.53981625)
```

```
(define circle-area
  (lambda (radius)
    (* pi radius radius)))
```

In the program, we are interested in the variables `pi` and `radius`, which appear multiple times – therefore, one speaks of different *occurrences* of these variables.

The test results in the replacement of the variable `radius` in the body of the `lambda` function of `circle-area`. This process is also called *binding*, because `radius` is a parameter within the `lambda function`. This occurrence as a parameter is also called *binding* the variable.

The two occurrences of `radius` in the body (`*pi radius radius`) that are replaced are called *bound occurrences*. The `pi` in the body of `circle-area` is also replaced and is therefore a bound occurrence. The corresponding bound occurrence is listed above in the `define`.

Depending on whether the binding is done with `define` or `lambda`, it behaves differently. You can see this if you run the program above and evaluate `radius` and `pi` in the REPL:

```
> radius
radius: Variable is undefined

> pi
3.14159265
```

This means we can use `pi` anywhere: It's also said that `pi` is *visible* everywhere in the program. `pi` is a *global variable*. Global variables are also bound by record definitions using `define-record`. In contrast, `radius` is only visible within the body of the `lambda` function of which it is a parameter – it's a so-called *local variable*.

Local variables also arise through local definitions, such as the `home-points` function in Section 9.1 on page 248. If a local definition is at the beginning of a lambda body, it is visible within that body. If it is within a branch of a `cond`, it is only visible within that branch.

You can visualize the relationship between the binding and bound occurrences in DrRacket by clicking the "Check Syntax" button and

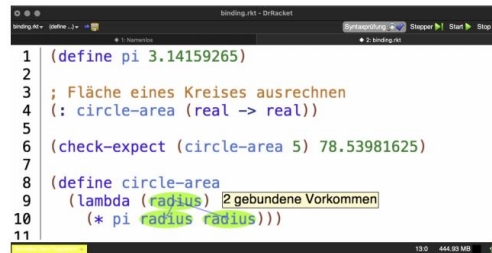


Figure 9.1: Display of binding after syntax check

Then hover the mouse pointer over the occurrences of the variable. Figure 9.1 shows an example of what this looks like.

Task 9.13

Try out the syntax check on the football code and examine, for example, the variables in the `'home-points'` function.

There is also a third category of binding, namely the "built-in" variables like `string-append` and `+`, which are visible globally in a similar way to global variables. If you hover over them after syntax checking, it will display "imported from".

One might think that this covers everything regarding binding: Global and built-in variables are visible everywhere; a local variable is visible within its lambda body or cond branch. But what about the following function?

```

(define f
  (lambda (x)
    (define y (+ x 1)) (lambda
      (x)
        (+ xy))))

```

Admittedly, it's artificially constructed. But how does it behave?

Task 9.14

What does the evaluation of `((f 1) 2)` yield?

Ÿ

This function has *two* variables called `x`: two bindings as parameters and also two bound occurrences. Which bound occurrences belong to which bindings? Here, the rule of so-called *lexical binding applies*:

Definition 9.1 (Lexical Binding)

You can find the binding associated with a bound occurrence by searching from the inside out in the function's bracket structure: The first binding as a parameter or local definition that you find in this way belongs to the bound occurrence.

If you can't find a binding this way, look for a global definition. If you still can't find a global definition, it must be a built-in or imported definition.

Lexical binding assigns bound occurrences to a binding. This means that if you rename a binding and all its bound occurrences, you will not change the program's behavior. Accordingly, variables belonging to different bindings are different variables. If you right-click on a variable after a syntax check, there is a "Rename" option that automates the renaming process.

The definition of `f` might seem esoteric to you – we have artificially differentiated-Variables with the same name have been created. However, we have often written programs where multiple variables with the same name appear, most recently, for example, in the ``home-points`` function:

```
(: home points (game -> points))
...
(define home points
  (lambda (game) ...))
```

Here, there are two variables named ``game``: In the signature declaration, this is the signature ``game`` from the record definition of the same name. Within the function, it's the parameter ``game``. This parameter fulfills the signature ``game`` – that's why the signature and the parameter have the same name, to clearly indicate this relationship.

The term "lexical binding" suggests that other forms of binding exist. In fact, binding in some other, primarily older, languages works according to different principles, often summarized under the term *dynamic binding*. Lexical binding is also often called *static binding*. In the context of programming, "static" means "before the program runs," and static binding means that you can see the relationship between bindings and bound occurrences in the program text. With dynamic binding, this relationship only emerges when the program is running.

One might think that everything has *now* been said about binding. But there is another special case, namely when a variable that is defined in a definition (whether local or global) also appears again immediately. We have already made frequent use of this, namely in recursive functions, where the variable in whose definition it is located is used for the recursive call.

Task 9.15

To which bonds do the bound occurrences of `x` in the following program belong?

```
(define g
  (lambda (x)
    (define x (+ x 1))
    (define y (+ x 1)) (lambda
      (x)
        (+ xy))))
```

Can you predict – without trying it in the REPL – what `((g 1) 2)` yields? $\ddot{y}$

TASKS

Task 9.16

Write a function ``any?`` that accepts a list and a function that returns either ``#t`` or ``#f`` for each list element. ``Any?`` should return ``#t`` if the function returns ``#t`` for at least one element of the list, otherwise ``#f``.

Write a function analogous to ``every?`` that returns ``#t`` if the function returns ``#t`` for all elements of the list, otherwise ``#f``. First, write a version following the pattern of ``any?``. Then write a second version that simply calls ``any?`` and does not use any recursion itself.

Task 9.17

Write functions ``feed-animals`` and ``run-over-animals`` that use the functions in Section 4.2 on page 123 feeds or runs over all the animals on a list.

Task 9.18

Write the following functions using `filter!`

- Write a function ``evens`` that removes the odd numbers from a list, • a function ``count-zeroes`` that counts the zeros in a list of numbers, • and a function ``multiples`` that accepts a number ``n`` and a list of numbers, and a
The list provides all multiples of the number `n`.

Task 9.19

Program a function ``filter-map`` that accepts as arguments a function ``p`` with one argument and a list ``l``. The result of ``filter-map`` should be the list of return values of ``p`` for those elements of ``l`` for which ``p`` does not return ``#f``.

Example:

```
(filter-map (lambda (x)
              (if (even? x)
                  (+ x 1)
                  #f))
            (list 1 2 5 17 24 13))
⇒ #<list 3 25>
```

Task 9.20

Use the `list-fold` function to write the following functions:

1. Write a new version of the `list-map` function that performs a function on each element of a list. The list is applied and returns the list of results.
2. Write a function called `list-or` that applies a function to all elements of a list. each returns a Boolean value, and the results are linked with `or`.
3. Write a function ``count-trues`` that returns a Boolean value for all true values. applies elements of a list and counts how often `#t` is returned.
4. Write a function ``contains`` that returns ``#t`` if an element is contained in a list. is, otherwise `#f`.
5. Write a function called `remove-duplicates` that removes all duplicate elements from a list. filters.

Task 9.21

Football reporters often cite obscure facts about past games when they have little to say about the course of the game.

Here's an example:

"In the 2009/2010 season, there were at least four matchdays where the largest goal difference was less than 3."

1. Provide the football reporter with helpful functions that will allow him to find an answer to such a question:

On how many matchdays of the 2009/2010 season was the largest goal difference less than 3?

Using these functions, write an expression that tests whether the statement about goal differences in the 2009/2010 season, quoted at the beginning of this task, is true!

- (a) Write a function that removes duplicates from a list. This function accepts a list and a function that accepts two elements of the list and returns whether they are equal. (For example, `=` for lists of numbers.) It then returns a list in which each element "occurs only once," meaning no other element is identical to it. (b) Write a function that extracts a list of all match

schedule numbers from the list of all matches.
tracted.

- (c) Write a function that turns the list of all games into a list of lists of all games –

so that each sublist contains all the matches of a matchday. (d) Write a

function that returns the goal difference of a match. (e) Write a function that

calculates the maximum element of a list. The function should accept, in addition to the list, a

function that calculates whether one element is less than or equal to another. (f) Finally, write an expression that returns

the number of matchdays of the 2009/2010 season.

fert, where the largest goal difference was less than 3.

2. Write expressions that answer the following questions – develop further ones if necessary.

Auxiliary functions:

- On which matchday were there the most home wins? •

On which matchday were the most goals scored? •

Were there more wins for home teams on odd-numbered matchdays than on even-numbered ones?

Task 9.22

Consider the following program:

```
(define x 2)           ; --> two
(define y -1)          ; --> minus one
```

```

(define z -3)                ; --> minus three
(define f
  (lambda (xz)
    (+ (* xx) zy))) (f 4 -2)

```

Rename the variables *x*, *y*, and *z*, which are defined in the first three lines of the program, throughout the entire program: *x* to *two*, *y* to *minus one*, and *z* to *minus three*. When renaming, pay attention to lexical binding. Do not rename any parameters of the function *f*.

After you have performed the renaming, what result does the expression return?

(f 4 -2)?

Task 9.23

Consider the following program:

```

(define x 2)                  ; --> two
(define y 4)                  ; --> four
(define z                      ; --> f
  (lambda (xyz) (+ x
                    (zy))))

(zyx (lambda (z) (+ xz)))

```

Rename the variables *x*, *y*, and *z*, which are defined in the first three lines of the program, throughout the entire program. The new name of the variable should appear as a comment after the arrow (*-->*) in the program. When renaming, pay attention to lexical binding. Do not rename any parameters of the function *z*.

After you have performed the renaming, calculate (zyx (lambda (z) (+ xz))) by hand and record the intermediate steps.

Task 9.24

Consider the following program:

```

(define x 1)
(define y 3)
(define z 5)

```

```
(define f
  (lambda (x)
    ((lambda (y)
      ((lambda (z)
        (+ z (* xy))) (+ xz)))

      (+ xy))))
(fy)
```

Rename all local variables bound within the function `f`. Do not change the names of the variables `x`, `y`, and `z` from the first three lines of the program. What result does the expression `(fy)` produce after renaming?

Task 9.25

Program a simple phone book: A phone book is represented as a function that accepts a person's name and returns that person's phone number.

1. First, define a signature ``phonebook-result`` for the result of a lookup in a phone book. Such a result is either a phone number or a value representing "not found". The phone book signature is `(string -> phonebook-result)`.

2. Define a value called `empty-phonebook` to represent the empty phonebook.

3. Define a function `add-to-phonebook` that expects a phone book, a name and a phone number and returns the phone book expanded with the new entry.

4. Write a function ``lookup-in-phonebook`` that expects a phone book and a person's name and returns the person's phone number from the phone book. Examples:
 - `(lookup-in-phonebook empty-phonebook "Hans")`` returns "not found".
 - `(lookup-in-phonebook (add-to-phonebook empty-phonebook "Hans"`

```
"754829")`
"Hans")
```

returns the number 754829.

- `(lookup-in-phonebook (add-to-phonebook empty-phonebook "Hans" "754829") "Lea")`

returns "not found".

Task 9.26

This task is suitable for you if you possess the relevant mathematical knowledge:

A polynomial is a function from $\mathbb{R}$ to $\mathbb{R}$ and has the following form:

$$p(x) = a_0 + a_1 \times x + a_2 \times x^2 + \dots + a_n \times x^n$$

A polynomial is therefore uniquely determined by the coefficients a_0 to a_n .

1. Write the data definition for polynomials.
2. Program a function ``polynomial+`` that adds two polynomials. Write down any applicable properties and check them!
3. Program a `polynomial` function that multiplies two polynomials. For example, the polynomials $a_0 + a_1x$ and $b_0 + b_1x$ are multiplied according to the following scheme:

$$\begin{aligned} & a_0 \times (b_0 + b_1 \times x) + \\ & a_1x \times (b_0 + b_1 \times x) \\ & = a_0 \times b_0 + a_0 \times b_1x + a_1 \times b_0x + a_1 \times b_1x^2 \end{aligned}$$

Write down any applicable characteristics and check them!

4. Write a `polynomial-function` that accepts a polynomial and provides a function that evaluates a polynomial at a specific point, i.e., exactly corresponds to the function p from the definition.
5. The derivative of a polynomial p as above is known to be given by

$$p'(x) = a_1 + 2 \times a_2 \times x + 3 \times a_3 \times x^2 + \dots + n \times a_n \times x^{n-1}$$

Given a polynomial, write the `polynomial-derivative` function that computes the derivative polynomial of a given polynomial.

Task 9.27

Lists can be sorted according to various criteria: for example, ascending, descending, or based on a field of elements. A list of football matches, for instance, can be sorted by the total number of goals, the number of goals scored by the home team, the number of goals scored by the away team, or the number of goals scored by the winning team. The sorting criterion is determined by a function that calculates whether one element should precede another.

1. Write a function that sorts a list according to an arbitrary criterion. The function should have the following signature:

```
(: list-sort ((%a %a -> boolean) (list-of %a) -> (list-of %a)))
```

The first argument is a function that implements an *ordering*, i.e., it compares two elements and returns *#t* if they are already in the correct order and *#f* if not. For example, *<=* or *>=* can be used for sorting lists of numbers.

2. Use this function to generate a list of football matches according to the following criteria.
sort:

- Total number of goals, •
- Number of goals scored by the home team,
- Number of goals scored by the away team or
- Number of goals scored by the winning team

Task 9.28

Sequences of infinite length can be represented as composite data with two components: The first component is the first element of the sequence, and the second is a function without parameters that, when applied, returns a sequence with the remaining elements, excluding the first. Such infinite sequences are called *streams*. Write a data and record definition for streams!

The following function should return a stream of natural numbers, starting from a number *n*:

```
; Create a stream with numbers starting from n (:
from (natural -> stream))
(define from
  (lambda (n)
    (make-stream n
      (lambda () (from (+ n 1))))))
```

(It is assumed that the constructor of the record definition is called ``make-stream``.) The following function is useful for viewing streams, which extracts the first *n* elements of a stream as a list:

```
; extract the first elements of a stream into a list
(: stream-take (natural stream -> (list-of %a)))
```

```
(define stream-take
  (lambda (n stream) (if (= n
    0)
    empty
    (cons (stream-first stream)
      (stream-take (- n 1)
        ((stream-rest-function stream)))))))
```

(We assumed that the selectors were named `stream-first` and `stream-rest-function`.)

`Stream-take` can be applied, for example, to the result of `from` :

```
(stream-take 17 (from 4))
ȳ #<list 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20>
```

Program some intellectual challenges using Streams!

1. Program a function `stream-drop` that accepts a natural number `n` and a stream, and returns a new stream created from the old one by dropping the first `n` elements:

```
(stream-take 17 (stream-drop 3 (from 4))) ȳ #<list 7 8 9 10 11 12 13 14
15 16 17 18 19 20 21 22 23>
```

2. Program a function `stream-filter` analogous to `filter`:

```
(stream-take 10 (stream-filter odd? (from 1)))
ȳ #<list 1 3 5 7 9 11 13 15 17 19>
```

3. Program a function `drop-multiples` that accepts a number `n` and a stream of numbers `s`. `drop-multiples` should return a stream in which all multiples of `n` have been removed relative to `s`.

```
(stream-take 10 (drop-multiples 3 (from 1)))
ȳ #<list 1 2 4 5 7 8 10 11 13 14>
```

4. Write a function `sieve` that extracts all the numbers that are in a stream of numbers. `fernt`, which are multiples of predecessors in the stream:

```
(stream-take 10 (sieve (from 2)))
ȳ #<list 2 3 5 7 11 13 17 19 23 29>
```

What kind of numbers are used in the example call and why?

5. Write a function called `powers` that returns a stream of the powers of a number `n`:

```
(stream-take 10 (powers 2))
ȳ #<list 2 4 8 16 32 64 128 256 512 1024>
```

6. Write a function `stream-map` analogous to `list-map`:

```
(stream-take 10 (stream-map (lambda (x) (+ x 1)) (from 1)))
ȳ #<list 2 3 4 5 6 7 8 9 10 11>
```

7. Write a function `merge` that combines two ascending streams of numbers into one ascending stream.
the stream of elements from both streams combined:

```
(stream-take 10 (merge (powers 2) (powers 3)))
ȳ #<list 2 3 4 8 9 16 27 32 64 81>
```

8. Write a definition for a stream of ascendingly sorted powers of prime numbers:

```
(stream-take 10 prime-powers)
ȳ #<list 2 3 4 5 7 8 9 11 13 16>
```

To do this, first define a stream of streams of powers.

```
(define prime-powers-stream (stream-map powers (sieve (from 2))))
```

Define a function ``merge-streams`` that accepts this stream and sorts the elements of the streams from ``prime-powers-stream`` in ascending order using ``merge``.

Task 9.29

Consider the following mysterious function:

```
(: // ((%a -> (%b -> %b)) (list-of %a) -> (%b -> %b)))
```

```
(define //
```

```
(lambda (proc lis) (cond
```

```
((empty? lis) (lambda
  (y) y)) ((cons?
  lis)
```

```
(lambda (y)
```

```
((proc (first lis)) ((// proc
  (rest lis)) y))))))
```

Notes: Pay attention to the signatures! Several subtasks offer opportunities to use ``curry`` and

``uncurry``. • Compare the function

with ``list-fold`` and describe how ``//`` and ``list-fold`` are related. If possible, write a definition of ``//`` that uses ``list-fold`` and vice versa.

- Write a ``list-sum`` function that adds the elements of a list using ``//``. • Write an ``insert`` function that accepts a real number ``n`` and returns a function that consumes an ascendingly sorted list of real numbers and returns a list in which ``n`` has been inserted at the corresponding position in the list. • Write a function that sorts a list of real numbers in ascending order using ``//``.

Task 9.30

Consider the following mysterious program:

```
(define y
  (lambda (f)
    ((lambda (x)
      (f (lambda (z) ((xx) z)))) (lambda (x)

        (f (lambda (z) ((xx) z)))))))
```

```
(define m
  (y
   (lambda (f)
     (lambda (x)
       (cond
        ((= x 1)
         1)
        (> x 1)
        (* x (f (- x 1))))))))))
```

Try out `m` – what does the function do?

The function `y` is also called *the fixed-point combinator*. This topic is discussed in detail in section 14.8.3 on page 458.

Task 9.31

Prove that for functions `p1` with one parameter that return one-parameter functions, and functions `p2` with two parameters:

$$\begin{aligned} (\text{curry } (\text{uncurry } p1)) &= p1 \text{ (uncurry} \\ (\text{curry } p2)) &= p2 \end{aligned}$$

Task 9.32

A function `f` is *idempotent* if:

$$(\text{compose } ff) = f$$

Show that the following function is idempotent:

```
(define abs
  (lambda (x)
    (if (negative? x) (- x)
        x)))
```

What other idempotent functions do you know?